

TFG_UAV_TELEMETRY_v4

4.0.0

Generado por Doxygen 1.17.0

1 Topic Index	1
1.1 Topics	1
2 Jerarquía de directorios	3
2.1 Directorios	3
3 Índice de estructuras de datos	5
3.1 Estructuras de datos	5
4 Índice de archivos	7
4.1 Lista de archivos	7
5 Topic Documentation	9
5.1 Estado interno del driver GPS	9
5.1.1 Descripción detallada	9
5.1.2 Documentación de variables	9
5.1.2.1 gps_data	9
5.1.2.2 gps_rx_buffer	10
5.1.2.3 gps_uart	10
5.1.2.4 rx_head	10
5.1.2.5 rx_tail	10
5.1.2.6 sentence	10
5.1.2.7 sentence_index	10
5.1.2.8 uart_rx_byte	10
5.2 Direcciones I2C	10
5.2.1 Descripción detallada	11
5.2.2 Documentación de «define»	11
5.2.2.1 ACC_ADDR	11
5.2.2.2 BMP_ADDR	11
5.2.2.3 GYR_ADDR	11
5.2.2.4 MAG_ADDR	11
5.3 Constantes de conversión	11
5.3.1 Descripción detallada	12
5.3.2 Documentación de «define»	12
5.3.2.1 ACC_SENS_2G	12
5.3.2.2 BMP180_OSS	12
5.3.2.3 BMP180_SEA_HPA	12
5.3.2.4 G_CONST	12
5.3.2.5 GAUSS_TO_UT	12
5.3.2.6 GYRO_SENS_250DPS	13
5.3.2.7 MAG_XY_LSB_GAUSS	13
5.3.2.8 MAG_Z_LSB_GAUSS	13
6 Documentación de directorios	15

6.1 Referencia del directorio Core	15
6.2 Referencia del directorio Core/Inc	15
6.3 Referencia del directorio Core/Src	16
7 Documentación de estructuras de datos	17
7.1 Referencia de la estructura EnvDataPacket_t	17
7.1.1 Descripción detallada	17
7.1.2 Documentación de campos	17
7.1.2.1 checksum	17
7.1.2.2 env_timestamp	18
7.1.2.3 env_updated	18
7.1.2.4 heading	18
7.1.2.5 pressure	18
7.1.2.6 temperature	18
7.2 Referencia de la estructura gps_data_t	18
7.2.1 Descripción detallada	19
7.2.2 Documentación de campos	19
7.2.2.1 altitude	19
7.2.2.2 fix	19
7.2.2.3 latitude	19
7.2.2.4 longitude	20
7.2.2.5 satellites	20
7.2.2.6 speed_kmh	20
7.2.2.7 speed_knots	20
7.2.2.8 valid	20
7.3 Referencia de la estructura GpsDataPacket_t	20
7.3.1 Descripción detallada	21
7.3.2 Documentación de campos	21
7.3.2.1 altitude	21
7.3.2.2 checksum	21
7.3.2.3 gps_timestamp	21
7.3.2.4 gps_updated	21
7.3.2.5 latitude	22
7.3.2.6 longitude	22
7.4 Referencia de la estructura ImuDataPacket_t	22
7.4.1 Descripción detallada	22
7.4.2 Documentación de campos	23
7.4.2.1 accel	23
7.4.2.2 checksum	23
7.4.2.3 gyro	23
7.4.2.4 imu_timestamp	23
7.4.2.5 imu_updated	23

7.4.2.6 mag	23
7.5 Referencia de la estructura Sensor10DoF_t	23
7.5.1 Descripción detallada	24
7.5.2 Documentación de campos	25
7.5.2.1 accel_x_ms2	25
7.5.2.2 accel_y_ms2	25
7.5.2.3 accel_z_ms2	25
7.5.2.4 altitude_m	25
7.5.2.5 gyro_x_dps	25
7.5.2.6 gyro_y_dps	25
7.5.2.7 gyro_z_dps	25
7.5.2.8 heading_deg	26
7.5.2.9 mag_x_uT	26
7.5.2.10 mag_y_uT	26
7.5.2.11 mag_z_uT	26
7.5.2.12 pitch_deg	26
7.5.2.13 pressure_atm	26
7.5.2.14 pressure_hpa	26
7.5.2.15 pressure_pa	26
7.5.2.16 roll_deg	27
7.5.2.17 temperature_c	27
7.5.2.18 timestamp_ms	27
8 Documentación de archivos	29
8.1 Referencia del archivo Core/Inc/dma.h	29
8.1.1 Descripción detallada	29
8.1.2 Documentación de funciones	29
8.1.2.1 MX_DMA_Init()	29
8.2 dma.h	30
8.3 Referencia del archivo Core/Inc/formatter_data.h	30
8.3.1 Descripción detallada	31
8.3.2 Documentación de funciones	31
8.3.2.1 EnvData_to_json()	31
8.3.2.2 EnvDataPacket_Init()	32
8.3.2.3 GpsData_to_json()	33
8.3.2.4 GpsDataPacket_Init()	34
8.3.2.5 ImuData_to_json()	35
8.3.2.6 ImuDataPacket_Init()	36
8.4 formatter_data.h	36
8.5 Referencia del archivo Core/Inc/FreeRTOSConfig.h	37
8.5.1 Documentación de «define»	38
8.5.1.1 CMSIS_device_header	38

8.5.1.2 configASSERT	39
8.5.1.3 configCPU_CLOCK_HZ	39
8.5.1.4 configENABLE_FPU	39
8.5.1.5 configENABLE_MPU	39
8.5.1.6 configKERNEL_INTERRUPT_PRIORITY	39
8.5.1.7 configLIBRARY_LOWEST_INTERRUPT_PRIORITY	39
8.5.1.8 configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY	39
8.5.1.9 configMAX_CO_ROUTINE_PRIORITIES	39
8.5.1.10 configMAX_PRIORITIES	39
8.5.1.11 configMAX_SYSCALL_INTERRUPT_PRIORITY	40
8.5.1.12 configMAX_TASK_NAME_LEN	40
8.5.1.13 configMESSAGE_BUFFER_LENGTH_TYPE	40
8.5.1.14 configMINIMAL_STACK_SIZE	40
8.5.1.15 configPRIO_BITS	40
8.5.1.16 configQUEUE_REGISTRY_SIZE	40
8.5.1.17 configSUPPORT_DYNAMIC_ALLOCATION	40
8.5.1.18 configSUPPORT_STATIC_ALLOCATION	40
8.5.1.19 configTICK_RATE_HZ	40
8.5.1.20 configTIMER_QUEUE_LENGTH	40
8.5.1.21 configTIMER_TASK_PRIORITY	41
8.5.1.22 configTIMER_TASK_STACK_DEPTH	41
8.5.1.23 configTOTAL_HEAP_SIZE	41
8.5.1.24 configUSE_16_BIT_TICKS	41
8.5.1.25 configUSE_CO_ROUTINES	41
8.5.1.26 configUSE_COUNTING_SEMAPHORES	41
8.5.1.27 configUSE_IDLE_HOOK	41
8.5.1.28 configUSE_MUTEXES	41
8.5.1.29 configUSE_NEWLIB_REENTRANT	41
8.5.1.30 configUSE_OS2_EVENTFLAGS_FROM_ISR	41
8.5.1.31 configUSE_OS2_MUTEX	42
8.5.1.32 configUSE_OS2_THREAD_ENUMERATE	42
8.5.1.33 configUSE_OS2_THREAD_FLAGS	42
8.5.1.34 configUSE_OS2_THREAD_SUSPEND_RESUME	42
8.5.1.35 configUSE_OS2_TIMER	42
8.5.1.36 configUSE_PORT_OPTIMISED_TASK_SELECTION	42
8.5.1.37 configUSE_PREEMPTION	42
8.5.1.38 configUSE_RECURSIVE_MUTEXES	42
8.5.1.39 configUSE_TICK_HOOK	42
8.5.1.40 configUSE_TIMERS	42
8.5.1.41 configUSE_TRACE_FACILITY	43
8.5.1.42 INCLUDE_eTaskGetState	43
8.5.1.43 INCLUDE_uxTaskGetStackHighWaterMark	43

8.5.1.44 INCLUDE_uxTaskPriorityGet	43
8.5.1.45 INCLUDE_vTaskCleanUpResources	43
8.5.1.46 INCLUDE_vTaskDelay	43
8.5.1.47 INCLUDE_vTaskDelayUntil	43
8.5.1.48 INCLUDE_vTaskDelete	43
8.5.1.49 INCLUDE_vTaskPrioritySet	43
8.5.1.50 INCLUDE_vTaskSuspend	43
8.5.1.51 INCLUDE_xQueueGetMutexHolder	44
8.5.1.52 INCLUDE_xTaskGetCurrentTaskHandle	44
8.5.1.53 INCLUDE_xTaskGetSchedulerState	44
8.5.1.54 INCLUDE_xTimerPendFunctionCall	44
8.5.1.55 USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION	44
8.5.1.56 USE_FreeRTOS_HEAP_4	44
8.5.1.57 vPortSVCHandler	44
8.5.1.58 xPortPendSVHandler	44
8.6 FreeRTOSConfig.h	45
8.7 Referencia del archivo Core/Inc/gpio.h	47
8.7.1 Descripción detallada	47
8.7.2 Documentación de funciones	47
8.7.2.1 MX_GPIO_Init()	47
8.8 gpio.h	48
8.9 Referencia del archivo Core/Inc/gps.h	48
8.9.1 Descripción detallada	49
8.9.2 Documentación de «define»	49
8.9.2.1 GPS_RX_BUFFER_SIZE	49
8.9.2.2 GPS_SENTENCE_MAX_LENGTH	49
8.9.3 Documentación de enumeraciones	50
8.9.3.1 gps_fix_t	50
8.9.3.2 gps_status_t	50
8.9.4 Documentación de funciones	50
8.9.4.1 GPS_DataValid()	50
8.9.4.2 GPS_GetData()	51
8.9.4.3 GPS_Init()	51
8.9.4.4 GPS_Process()	52
8.9.4.5 GPS_RestartReceive()	52
8.9.4.6 GPS_UARTCallback()	53
8.10 gps.h	53
8.11 Referencia del archivo Core/Inc/i2c.h	54
8.11.1 Descripción detallada	54
8.11.2 Documentación de funciones	55
8.11.2.1 MX_I2C1_Init()	55
8.11.3 Documentación de variables	55

8.11.3.1 hi2c1	55
8.12 i2c.h	55
8.13 Referencia del archivo Core/Inc/main.h	55
8.13.1 Descripción detallada	57
8.13.2 Documentación de «define»	57
8.13.2.1 Audio_RST_GPIO_Port	57
8.13.2.2 Audio_RST_Pin	57
8.13.2.3 B1_GPIO_Port	57
8.13.2.4 B1_Pin	58
8.13.2.5 BOOT1_GPIO_Port	58
8.13.2.6 BOOT1_Pin	58
8.13.2.7 CLK_IN_GPIO_Port	58
8.13.2.8 CLK_IN_Pin	58
8.13.2.9 CS_I2C_SPI_GPIO_Port	58
8.13.2.10 CS_I2C_SPI_Pin	58
8.13.2.11 I2C1_SCL_IMU_GPIO_Port	58
8.13.2.12 I2C1_SCL_IMU_Pin	58
8.13.2.13 I2C1_SDA_IMU_GPIO_Port	58
8.13.2.14 I2C1_SDA_IMU_Pin	59
8.13.2.15 I2S3_MCK_GPIO_Port	59
8.13.2.16 I2S3_MCK_Pin	59
8.13.2.17 I2S3_SCK_GPIO_Port	59
8.13.2.18 I2S3_SCK_Pin	59
8.13.2.19 I2S3_SD_GPIO_Port	59
8.13.2.20 I2S3_SD_Pin	59
8.13.2.21 I2S3_WS_GPIO_Port	59
8.13.2.22 I2S3_WS_Pin	59
8.13.2.23 LD3_GPIO_Port	59
8.13.2.24 LD3_Pin	60
8.13.2.25 LD4_GPIO_Port	60
8.13.2.26 LD4_Pin	60
8.13.2.27 LD5_GPIO_Port	60
8.13.2.28 LD5_Pin	60
8.13.2.29 LD6_GPIO_Port	60
8.13.2.30 LD6_Pin	60
8.13.2.31 MEMS_INT2_GPIO_Port	60
8.13.2.32 MEMS_INT2_Pin	60
8.13.2.33 OTG_FS_DM_GPIO_Port	60
8.13.2.34 OTG_FS_DM_Pin	61
8.13.2.35 OTG_FS_DP_GPIO_Port	61
8.13.2.36 OTG_FS_DP_Pin	61
8.13.2.37 OTG_FS_ID_GPIO_Port	61

8.13.2.38 OTG_FS_ID_Pin	61
8.13.2.39 OTG_FS_OverCurrent_GPIO_Port	61
8.13.2.40 OTG_FS_OverCurrent_Pin	61
8.13.2.41 OTG_FS_PowerSwitchOn_GPIO_Port	61
8.13.2.42 OTG_FS_PowerSwitchOn_Pin	61
8.13.2.43 PC14_OSC32_IN_GPIO_Port	61
8.13.2.44 PC14_OSC32_IN_Pin	62
8.13.2.45 PC15_OSC32_OUT_GPIO_Port	62
8.13.2.46 PC15_OSC32_OUT_Pin	62
8.13.2.47 PDM_OUT_GPIO_Port	62
8.13.2.48 PDM_OUT_Pin	62
8.13.2.49 PH0_OSC_IN_GPIO_Port	62
8.13.2.50 PH0_OSC_IN_Pin	62
8.13.2.51 PH1_OSC_OUT_GPIO_Port	62
8.13.2.52 PH1_OSC_OUT_Pin	62
8.13.2.53 SPI1_MISO_GPIO_Port	62
8.13.2.54 SPI1_MISO_Pin	63
8.13.2.55 SPI1_MOSI_GPIO_Port	63
8.13.2.56 SPI1_MOSI_Pin	63
8.13.2.57 SPI1_SCK_GPIO_Port	63
8.13.2.58 SPI1_SCK_Pin	63
8.13.2.59 SWCLK_GPIO_Port	63
8.13.2.60 SWCLK_Pin	63
8.13.2.61 SWDIO_GPIO_Port	63
8.13.2.62 SWDIO_Pin	63
8.13.2.63 SWO_GPIO_Port	63
8.13.2.64 SWO_Pin	64
8.13.2.65 VBUS_FS_GPIO_Port	64
8.13.2.66 VBUS_FS_Pin	64
8.13.3 Documentación de funciones	64
8.13.3.1 Error_Handler()	64
8.14 main.h	64
8.15 Referencia del archivo Core/Inc/sensor10DoF.h	65
8.15.1 Descripción detallada	66
8.15.2 Documentación de enumeraciones	66
8.15.2.1 SensorStatus_t	66
8.15.3 Documentación de funciones	67
8.15.3.1 Sensor10DoF_Calibrate()	67
8.15.3.2 Sensor10DoF_I2CRecover()	67
8.15.3.3 Sensor10DoF_Init()	68
8.15.3.4 Sensor10DoF_Read()	68
8.15.3.5 Sensor10DoF_Reset()	69

8.16 sensor10DoF.h	70
8.17 Referencia del archivo Core/Inc/stm32f4xx_hal_conf.h	70
8.17.1 Documentación de «define»	73
8.17.1.1 assert_param	73
8.17.1.2 DATA_CACHE_ENABLE	73
8.17.1.3 DP83848_PHY_ADDRESS	73
8.17.1.4 ETH_RX_BUF_SIZE	73
8.17.1.5 ETH_RXBUFNB	73
8.17.1.6 ETH_TX_BUF_SIZE	74
8.17.1.7 ETH_TXBUFNB	74
8.17.1.8 EXTERNAL_CLOCK_VALUE	74
8.17.1.9 HAL_CORTEX_MODULE_ENABLED	74
8.17.1.10 HAL_DMA_MODULE_ENABLED	74
8.17.1.11 HAL_EXTI_MODULE_ENABLED	74
8.17.1.12 HAL_FLASH_MODULE_ENABLED	74
8.17.1.13 HAL_GPIO_MODULE_ENABLED	74
8.17.1.14 HAL_I2C_MODULE_ENABLED	74
8.17.1.15 HAL_MODULE_ENABLED	75
8.17.1.16 HAL_PWR_MODULE_ENABLED	75
8.17.1.17 HAL_RCC_MODULE_ENABLED	75
8.17.1.18 HAL_TIM_MODULE_ENABLED	75
8.17.1.19 HAL_UART_MODULE_ENABLED	75
8.17.1.20 HSE_STARTUP_TIMEOUT	75
8.17.1.21 HSE_VALUE	75
8.17.1.22 HSI_VALUE	75
8.17.1.23 INSTRUCTION_CACHE_ENABLE	76
8.17.1.24 LSE_STARTUP_TIMEOUT	76
8.17.1.25 LSE_VALUE	76
8.17.1.26 LSI_VALUE	76
8.17.1.27 MAC_ADDR0	76
8.17.1.28 MAC_ADDR1	76
8.17.1.29 MAC_ADDR2	76
8.17.1.30 MAC_ADDR3	76
8.17.1.31 MAC_ADDR4	77
8.17.1.32 MAC_ADDR5	77
8.17.1.33 PHY_AUTONEGO_COMPLETE	77
8.17.1.34 PHY_AUTONEGOTIATION	77
8.17.1.35 PHY_BCR	77
8.17.1.36 PHY_BSR	77
8.17.1.37 PHY_CONFIG_DELAY	77
8.17.1.38 PHY_DUPLEX_STATUS	77
8.17.1.39 PHY_FULLDUPLEX_100M	78

8.17.1.40 PHY_FULLDUPLEX_10M	78
8.17.1.41 PHY_HALFDUPLEX_100M	78
8.17.1.42 PHY_HALFDUPLEX_10M	78
8.17.1.43 PHY_ISOLATE	78
8.17.1.44 PHY_JABBER_DETECTION	78
8.17.1.45 PHY_LINKED_STATUS	78
8.17.1.46 PHY_LOOPBACK	78
8.17.1.47 PHY_POWERDOWN	79
8.17.1.48 PHY_READ_TO	79
8.17.1.49 PHY_RESET	79
8.17.1.50 PHY_RESET_DELAY	79
8.17.1.51 PHY_RESTART_AUTONEGOTIATION	79
8.17.1.52 PHY_SPEED_STATUS	79
8.17.1.53 PHY_SR	79
8.17.1.54 PHY_WRITE_TO	79
8.17.1.55 PREFETCH_ENABLE	80
8.17.1.56 TICK_INT_PRIORITY	80
8.17.1.57 USE_HAL_ADC_REGISTER_CALLBACKS	80
8.17.1.58 USE_HAL_CAN_REGISTER_CALLBACKS	80
8.17.1.59 USE_HAL_CEC_REGISTER_CALLBACKS	80
8.17.1.60 USE_HAL_CRYP_REGISTER_CALLBACKS	80
8.17.1.61 USE_HAL_DAC_REGISTER_CALLBACKS	80
8.17.1.62 USE_HAL_DCMI_REGISTER_CALLBACKS	80
8.17.1.63 USE_HAL_DFSDM_REGISTER_CALLBACKS	80
8.17.1.64 USE_HAL_DMA2D_REGISTER_CALLBACKS	80
8.17.1.65 USE_HAL_DSI_REGISTER_CALLBACKS	81
8.17.1.66 USE_HAL_ETH_REGISTER_CALLBACKS	81
8.17.1.67 USE_HAL_FMPI2C_REGISTER_CALLBACKS	81
8.17.1.68 USE_HAL_FMPMBUS_REGISTER_CALLBACKS	81
8.17.1.69 USE_HAL_HASH_REGISTER_CALLBACKS	81
8.17.1.70 USE_HAL_HCD_REGISTER_CALLBACKS	81
8.17.1.71 USE_HAL_I2C_REGISTER_CALLBACKS	81
8.17.1.72 USE_HAL_I2S_REGISTER_CALLBACKS	81
8.17.1.73 USE_HAL_IRDA_REGISTER_CALLBACKS	81
8.17.1.74 USE_HAL_LPTIM_REGISTER_CALLBACKS	81
8.17.1.75 USE_HAL_LTDC_REGISTER_CALLBACKS	82
8.17.1.76 USE_HAL_MMC_REGISTER_CALLBACKS	82
8.17.1.77 USE_HAL_NAND_REGISTER_CALLBACKS	82
8.17.1.78 USE_HAL_NOR_REGISTER_CALLBACKS	82
8.17.1.79 USE_HAL_PCCARD_REGISTER_CALLBACKS	82
8.17.1.80 USE_HAL_PCD_REGISTER_CALLBACKS	82
8.17.1.81 USE_HAL_QSPI_REGISTER_CALLBACKS	82

8.17.1.82 USE_HAL_RNG_REGISTER_CALLBACKS	82
8.17.1.83 USE_HAL_RTC_REGISTER_CALLBACKS	82
8.17.1.84 USE_HAL_SAI_REGISTER_CALLBACKS	82
8.17.1.85 USE_HAL_SD_REGISTER_CALLBACKS	83
8.17.1.86 USE_HAL_SDRAM_REGISTER_CALLBACKS	83
8.17.1.87 USE_HAL_SMARTCARD_REGISTER_CALLBACKS	83
8.17.1.88 USE_HAL_SMBUS_REGISTER_CALLBACKS	83
8.17.1.89 USE_HAL_SPDIFRX_REGISTER_CALLBACKS	83
8.17.1.90 USE_HAL_SPI_REGISTER_CALLBACKS	83
8.17.1.91 USE_HAL_SRAM_REGISTER_CALLBACKS	83
8.17.1.92 USE_HAL_TIM_REGISTER_CALLBACKS	83
8.17.1.93 USE_HAL_UART_REGISTER_CALLBACKS	83
8.17.1.94 USE_HAL_USART_REGISTER_CALLBACKS	83
8.17.1.95 USE_HAL_WWDG_REGISTER_CALLBACKS	84
8.17.1.96 USE_RTOS	84
8.17.1.97 USE_SPI_CRC	84
8.17.1.98 VDD_VALUE	84
8.18 stm32f4xx_hal_conf.h	84
8.19 Referencia del archivo Core/Inc/tim.h	89
8.19.1 Descripción detallada	90
8.19.2 Documentación de funciones	90
8.19.2.1 MX_TIM1_Init()	90
8.19.2.2 MX_TIM3_Init()	90
8.19.3 Documentación de variables	90
8.19.3.1 htim1	90
8.19.3.2 htim3	90
8.20 tim.h	91
8.21 Referencia del archivo Core/Inc/usart.h	91
8.21.1 Descripción detallada	92
8.21.2 Documentación de funciones	92
8.21.2.1 MX_USART2_UART_Init()	92
8.21.2.2 MX_USART3_UART_Init()	92
8.21.3 Documentación de variables	92
8.21.3.1 huart2	92
8.21.3.2 huart3	92
8.22 usart.h	93
8.23 Referencia del archivo Core/Inc/wifi.h	93
8.23.1 Descripción detallada	94
8.23.2 Documentación de funciones	95
8.23.2.1 WIFI_ConnectAP()	95
8.23.2.2 WIFI_DisconnectAP()	96
8.23.2.3 WIFI_GetIP()	96

8.23.2.4	WIFI_GetRSSI()	97
8.23.2.5	WIFI_GetState()	97
8.23.2.6	WIFI_HTTP_PostJSON()	98
8.23.2.7	WIFI_Init()	98
8.23.2.8	WIFI_MQTT_Connect()	99
8.23.2.9	WIFI_MQTT_Disconnect()	100
8.23.2.10	WIFI_MQTT_Publish()	100
8.23.2.11	WIFI_MQTT_Subscribe()	101
8.23.2.12	WIFI_MQTT_UserConfig()	102
8.23.2.13	WIFI_Process()	102
8.23.2.14	WIFI_Reconnect()	103
8.23.2.15	WIFI_Reset()	103
8.23.2.16	WIFI_SendJSON()	104
8.23.2.17	WIFI_SendRaw()	104
8.23.2.18	WIFI_SetStationMode()	105
8.23.2.19	WIFI_TCPClose()	105
8.23.2.20	WIFI_TCPConnect()	105
8.23.2.21	WIFI_Test()	106
8.24	wifi.h	106
8.25	Referencia del archivo Core/Inc/wifi_types.h	107
8.25.1	Descripción detallada	107
8.25.2	Documentación de enumeraciones	108
8.25.2.1	wifi_state_t	108
8.25.2.2	wifi_status_t	108
8.26	wifi_types.h	108
8.27	Referencia del archivo Core/Src/formatter_data.c	109
8.27.1	Descripción detallada	109
8.27.2	Documentación de funciones	110
8.27.2.1	EnvData_to_json()	110
8.27.2.2	EnvDataPacket_Init()	111
8.27.2.3	GpsData_to_json()	111
8.27.2.4	GpsDataPacket_Init()	112
8.27.2.5	ImuData_to_json()	113
8.27.2.6	ImuDataPacket_Init()	114
8.28	Referencia del archivo Core/Src/freertos.c	115
8.28.1	Descripción detallada	117
8.28.2	Documentación de «define»	117
8.28.2.1	PAYLOAD_LENIGHT	117
8.28.2.2	SERVER_HOST	117
8.28.2.3	WIFI_TEST_PASS	118
8.28.2.4	WIFI_TEST_SSID	118
8.28.3	Documentación de funciones	118

8.28.3.1 MX_FREERTOS_Init()	118
8.28.3.2 StartGpsTask()	118
8.28.3.3 StartImuTask()	119
8.28.3.4 StartPacketTask()	119
8.28.3.5 StartWifiTask()	119
8.28.4 Documentación de variables	120
8.28.4.1 envPacket	120
8.28.4.2 gps_st	120
8.28.4.3 gpsDataUpdated	120
8.28.4.4 gpsMutex	120
8.28.4.5 gpsPacket	120
8.28.4.6 gpsPacketToSend	120
8.28.4.7 gpsPacketToUpdate	120
8.28.4.8 gpsTask_attributes	121
8.28.4.9 gpsTaskHandle	121
8.28.4.10 huart2	121
8.28.4.11 huart3	121
8.28.4.12 imu_data	121
8.28.4.13 imuDataUpdated	122
8.28.4.14 imuMutex	122
8.28.4.15 imuPacket	122
8.28.4.16 imuPacketToSend	122
8.28.4.17 imuPacketToUpdate	122
8.28.4.18 imuTask_attributes	122
8.28.4.19 imuTaskHandle	123
8.28.4.20 packetMutex	123
8.28.4.21 packetTask_attributes	123
8.28.4.22 packetTaskHandle	123
8.28.4.23 payload_envPacket	123
8.28.4.24 payload_gpsPacket	123
8.28.4.25 payload_imuPacket	124
8.28.4.26 wifi_st	124
8.28.4.27 wifiTask_attributes	124
8.28.4.28 wifiTaskHandle	124
8.29 Referencia del archivo Core/Src/gps.c	124
8.29.1 Descripción detallada	126
8.29.2 Documentación de funciones	126
8.29.2.1 GPS_DataValid()	126
8.29.2.2 GPS_GetData()	127
8.29.2.3 GPS_Init()	127
8.29.2.4 gps_nmea_to_decimal()	128
8.29.2.5 gps_parse_gga()	128

8.29.2.6 <code>gps_parse_rmc()</code>	129
8.29.2.7 <code>GPS_Process()</code>	129
8.29.2.8 <code>GPS_RestartReceive()</code>	129
8.29.2.9 <code>gps_rx_pop()</code>	130
8.29.2.10 <code>gps_rx_push()</code>	130
8.29.2.11 <code>GPS_UARTCallback()</code>	130
8.30 Referencia del archivo <code>Core/Src/sensor10DoF.c</code>	131
8.30.1 Descripción detallada	132
8.30.2 Documentación de funciones	133
8.30.2.1 <code>ACC_Init()</code>	133
8.30.2.2 <code>BMP180_ReadCalibration()</code>	133
8.30.2.3 <code>BMP180_ReadUP()</code>	133
8.30.2.4 <code>BMP180_ReadUT()</code>	134
8.30.2.5 <code>GYR_Init()</code>	134
8.30.2.6 <code>I2C_Read()</code>	134
8.30.2.7 <code>I2C_Write()</code>	135
8.30.2.8 <code>MAG_Init()</code>	135
8.30.2.9 <code>Sensor10DoF_Calibrate()</code>	136
8.30.2.10 <code>Sensor10DoF_CalibrateGyro_Internal()</code>	136
8.30.2.11 <code>Sensor10DoF_I2CRecover()</code>	136
8.30.2.12 <code>Sensor10DoF_Init()</code>	137
8.30.2.13 <code>Sensor10DoF_Read()</code>	137
8.30.2.14 <code>Sensor10DoF_Reset()</code>	138
8.30.3 Documentación de variables	138
8.30.3.1 <code>AC1</code>	138
8.30.3.2 <code>AC2</code>	139
8.30.3.3 <code>AC3</code>	139
8.30.3.4 <code>AC4</code>	139
8.30.3.5 <code>AC5</code>	139
8.30.3.6 <code>AC6</code>	139
8.30.3.7 <code>B1</code>	139
8.30.3.8 <code>B2</code>	139
8.30.3.9 <code>gyro_x_offset</code>	139
8.30.3.10 <code>gyro_y_offset</code>	139
8.30.3.11 <code>gyro_z_offset</code>	140
8.30.3.12 <code>hi2c1</code>	140
8.30.3.13 <code>MB</code>	140
8.30.3.14 <code>MC</code>	140
8.30.3.15 <code>MD</code>	140
8.31 Referencia del archivo <code>Core/Src/stm32f4xx_hal_msp.c</code>	140
8.31.1 Descripción detallada	140
8.31.2 Documentación de funciones	141

8.31.2.1 HAL_MspInit()	141
8.32 Referencia del archivo Core/Src/stm32f4xx_hal_timebase_tim.c	141
8.32.1 Descripción detallada	141
8.32.2 Documentación de funciones	142
8.32.2.1 HAL_InitTick()	142
8.32.2.2 HAL_ResumeTick()	142
8.32.2.3 HAL_SuspendTick()	143
8.32.3 Documentación de variables	143
8.32.3.1 htim2	143
8.33 Referencia del archivo Core/Src/syscalls.c	143
8.33.1 Descripción detallada	144
8.33.2 Documentación de funciones	144
8.33.2.1 __attribute__()	144
8.33.2.2 __io_getchar()	145
8.33.2.3 __io_putchar()	145
8.33.2.4 _close()	145
8.33.2.5 _execve()	145
8.33.2.6 _exit()	145
8.33.2.7 _fork()	145
8.33.2.8 _fstat()	145
8.33.2.9 _getpid()	145
8.33.2.10 _isatty()	146
8.33.2.11 _kill()	146
8.33.2.12 _link()	146
8.33.2.13 _lseek()	146
8.33.2.14 _open()	146
8.33.2.15 _stat()	146
8.33.2.16 _times()	146
8.33.2.17 _unlink()	147
8.33.2.18 _wait()	147
8.33.2.19 initialise_monitor_handles()	147
8.33.3 Documentación de variables	147
8.33.3.1 environ	147
8.34 Referencia del archivo Core/Src/wifi.c	147
8.34.1 Descripción detallada	149
8.34.2 Documentación de «define»	149
8.34.2.1 RING_BUFFER_SIZE	149
8.34.3 Documentación de funciones	149
8.34.3.1 flush_ring_buffer()	149
8.34.3.2 get_dma_write_index()	149
8.34.3.3 HAL_UART_TxCpltCallback()	150
8.34.3.4 search_in_ring_buffer()	150

8.34.3.5	WIFI_ConnectAP()	150
8.34.3.6	WIFI_DisconnectAP()	151
8.34.3.7	WIFI_GetState()	152
8.34.3.8	WIFI_Init()	152
8.34.3.9	WIFI_MQTT_Connect()	153
8.34.3.10	WIFI_MQTT_Disconnect()	154
8.34.3.11	WIFI_MQTT_Publish()	154
8.34.3.12	WIFI_MQTT_UserConfig()	155
8.34.3.13	WIFI_Reconnect()	156
8.34.3.14	WIFI_Reset()	156
8.34.3.15	wifi_send_cmd()	157
8.34.3.16	WIFI_SetStationMode()	158
8.34.3.17	WIFI_Test()	158
8.34.4	Documentación de variables	159
8.34.4.1	cached_broker	159
8.34.4.2	cached_client_id	159
8.34.4.3	cached_pass	159
8.34.4.4	cached_port	159
8.34.4.5	cached_ssid	159
8.34.4.6	read_index	159
8.34.4.7	ring_buffer	159
8.34.4.8	wifi_state	160
8.34.4.9	wifi_tx_sem	160
8.34.4.10	wifi_uart	160
Índice alfabético		161

Capítulo 1

Topic Index

1.1. Topics

Here is a list of all topics with brief descriptions:

Estado interno del driver GPS	9
Direcciones I2C	10
Constantes de conversión	11

Capítulo 2

Jerarquía de directorios

2.1. Directorios

Core	15
Inc	15
dma.h	29
formatter_data.h	30
FreeRTOSConfig.h	37
gpio.h	47
gps.h	48
i2c.h	54
main.h	55
sensor10DoF.h	65
stm32f4xx_hal_conf.h	70
tim.h	89
usart.h	91
wifi.h	93
wifi_types.h	107
Src	16
formatter_data.c	109
freertos.c	115
gps.c	124
sensor10DoF.c	131
stm32f4xx_hal_msp.c	140
stm32f4xx_hal_timebase_tim.c	141
syscalls.c	143
wifi.c	147
Inc	15
dma.h	29
formatter_data.h	30
FreeRTOSConfig.h	37
gpio.h	47
gps.h	48
i2c.h	54
main.h	55
sensor10DoF.h	65
stm32f4xx_hal_conf.h	70
tim.h	89
usart.h	91
wifi.h	93
wifi_types.h	107

Src	16
formatter_data.c	109
freertos.c	115
gps.c	124
sensor10DoF.c	131
stm32f4xx_hal_msp.c	140
stm32f4xx_hal_timebase_tim.c	141
syscalls.c	143
wifi.c	147

Capítulo 3

Índice de estructuras de datos

3.1. Estructuras de datos

Lista de estructuras con breves descripciones:

EnvDataPacket_t	
Paquete normalizado con las magnitudes ambientales	17
gps_data_t	
Última solución de navegación calculada por el módulo GPS	18
GpsDataPacket_t	
Paquete normalizado con la posición procedente del receptor GPS	20
ImuDataPacket_t	
Paquete normalizado con las magnitudes procedentes de la IMU	22
Sensor10DoF_t	
Conjunto completo de medidas calculadas por el módulo 10DoF	23

Capítulo 4

Índice de archivos

4.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

Core/Inc/ dma.h	29
This file contains all the function prototypes for the dma.c file	
Core/Inc/ formatter_data.h	30
Define los paquetes de datos normalizados y la API de conversión a JSON	
Core/Inc/ FreeRTOSConfig.h	37
Core/Inc/ gpio.h	47
This file contains all the function prototypes for the gpio.c file	
Core/Inc/ gps.h	48
Interfaz pública del driver GPS basado en recepción UART	
Core/Inc/ i2c.h	54
This file contains all the function prototypes for the i2c.c file	
Core/Inc/ main.h	55
: Header for main.c file. This file contains the common defines of the application	
Core/Inc/ sensor10DoF.h	65
Interfaz pública del módulo de adquisición del sensor 10DoF	
Core/Inc/ stm32f4xx_hal_conf.h	70
Core/Inc/ tim.h	89
This file contains all the function prototypes for the tim.c file	
Core/Inc/ usart.h	91
This file contains all the function prototypes for the usart.c file	
Core/Inc/ wifi.h	93
Interfaz pública del driver WiFi basado en comandos AT	
Core/Inc/ wifi_types.h	107
Tipos compartidos por la interfaz WiFi y las capas de aplicación	
Core/Src/ formatter_data.c	109
Conversión de paquetes de telemetría a cadenas de texto para publicación	
Core/Src/ freertos.c	115
Planificación FreeRTOS de adquisición, empaquetado y envío MQTT de telemetría	
Core/Src/ gps.c	124
Driver no bloqueante para recepción y procesamiento de tramas NMEA GPS	
Core/Src/ sensor10DoF.c	131
Driver de adquisición para IMU 10DoF mediante bus I2C	
Core/Src/ stm32f4xx_hal_msp.c	140
This file provides code for the MSP Initialization and de-Initialization codes	
Core/Src/ stm32f4xx_hal_timebase_tim.c	141
HAL time base based on the hardware TIM	

Core/Src/ syscalls.c	
STM32CubeIDE Minimal System calls file	143
Core/Src/ wifi.c	
Driver WiFi/MQTT basado en comandos AT con RX DMA circular y TX DMA sincronizado . . .	147

Capítulo 5

Topic Documentation

5.1. Estado interno del driver GPS

Variables privadas usadas para desacoplar recepción UART y parsing.

Variables

- static UART_HandleTypeDef * `gps_uart`
UART asociada al módulo GPS inicializado.
- static uint8_t `uart_rx_byte`
Byte temporal usado por la recepción UART por interrupción.
- static uint8_t `gps_rx_buffer` [GPS_RX_BUFFER_SIZE]
Búfer circular de recepción para no perder bytes entre llamadas a `GPS_Process()`.
- static volatile uint16_t `rx_head` = 0
Índice de escritura del búfer circular, actualizado desde la ISR.
- static volatile uint16_t `rx_tail` = 0
Índice de lectura del búfer circular, consumido por el parser.
- static char `sentence` [GPS_SENTENCE_MAX_LENGTH]
Sentencia NMEA en construcción antes de identificar su tipo.
- static uint16_t `sentence_index` = 0
Posición actual dentro de `sentence`.
- static `gps_data_t` `gps_data`
Último estado GPS decodificado y disponible para el resto del sistema.

5.1.1. Descripción detallada

Variables privadas usadas para desacoplar recepción UART y parsing.

5.1.2. Documentación de variables

5.1.2.1. `gps_data`

```
gps_data_t gps_data [static]
```

Último estado GPS decodificado y disponible para el resto del sistema.

5.1.2.2. `gps_rx_buffer`

```
uint8_t gps_rx_buffer[GPS_RX_BUFFER_SIZE] [static]
```

Búfer circular de recepción para no perder bytes entre llamadas a `GPS_Process()`.

5.1.2.3. `gps_uart`

```
UART_HandleTypeDef* gps_uart [static]
```

UART asociada al módulo GPS inicializado.

5.1.2.4. `rx_head`

```
volatile uint16_t rx_head = 0 [static]
```

Índice de escritura del búfer circular, actualizado desde la ISR.

5.1.2.5. `rx_tail`

```
volatile uint16_t rx_tail = 0 [static]
```

Índice de lectura del búfer circular, consumido por el parser.

5.1.2.6. `sentence`

```
char sentence[GPS_SENTENCE_MAX_LENGTH] [static]
```

Sentencia NMEA en construcción antes de identificar su tipo.

5.1.2.7. `sentence_index`

```
uint16_t sentence_index = 0 [static]
```

Posición actual dentro de `sentence`.

5.1.2.8. `uart_rx_byte`

```
uint8_t uart_rx_byte [static]
```

Byte temporal usado por la recepción UART por interrupción.

5.2. Direcciones I2C

Direcciones de 7 bits desplazadas para el formato esperado por la HAL.

defines

- #define ACC_ADDR (0x19 << 1)
- #define MAG_ADDR (0x1E << 1)
- #define GYR_ADDR (0x6B << 1)
- #define BMP_ADDR (0x77 << 1)

5.2.1. Descripción detallada

Direcciones de 7 bits desplazadas para el formato esperado por la HAL.

5.2.2. Documentación de «define»**5.2.2.1. ACC_ADDR**

```
#define ACC_ADDR (0x19 << 1)
```

Dirección I2C del acelerómetro.

5.2.2.2. BMP_ADDR

```
#define BMP_ADDR (0x77 << 1)
```

Dirección I2C del barómetro BMP180.

5.2.2.3. GYR_ADDR

```
#define GYR_ADDR (0x6B << 1)
```

Dirección I2C del giróscopo.

5.2.2.4. MAG_ADDR

```
#define MAG_ADDR (0x1E << 1)
```

Dirección I2C del magnetómetro.

5.3. Constantes de conversión

Factores usados para convertir cuentas crudas a magnitudes físicas.

defines

- #define `G_CONST` 9.80665f
- #define `ACC_SENS_2G` 0.001f
- #define `GYRO_SENS_250DPS` 0.00875f
- #define `MAG_XY_LSB_GAUSS` 1100.0f
- #define `MAG_Z_LSB_GAUSS` 980.0f
- #define `GAUSS_TO_UT` 100.0f
- #define `BMP180_OSS` 3
- #define `BMP180_SEA_HPA` 1013.25f

5.3.1. Descripción detallada

Factores usados para convertir cuentas crudas a magnitudes físicas.

5.3.2. Documentación de «define»

5.3.2.1. ACC_SENS_2G

```
#define ACC_SENS_2G 0.001f
```

Sensibilidad del acelerómetro en g/LSB para ± 2 g.

5.3.2.2. BMP180_OSS

```
#define BMP180_OSS 3
```

Sobremuestreo del BMP180 priorizando precisión frente a latencia.

5.3.2.3. BMP180_SEA_HPA

```
#define BMP180_SEA_HPA 1013.25f
```

Presión estándar al nivel del mar para estimar altitud relativa.

5.3.2.4. G_CONST

```
#define G_CONST 9.80665f
```

Gravedad estándar usada para convertir g a m/s^2 .

5.3.2.5. GAUSS_TO_UT

```
#define GAUSS_TO_UT 100.0f
```

Factor de conversión de gauss a microteslas.

5.3.2.6. GYRO_SENS_250DPS

```
#define GYRO_SENS_250DPS 0.00875f
```

Sensibilidad del gir6scopo en dps/LSB para ± 250 dps.

5.3.2.7. MAG_XY_LSB_GAUSS

```
#define MAG_XY_LSB_GAUSS 1100.0f
```

Sensibilidad X/Y del magnet6metro en LSB/gauss.

5.3.2.8. MAG_Z_LSB_GAUSS

```
#define MAG_Z_LSB_GAUSS 980.0f
```

Sensibilidad Z del magnet6metro en LSB/gauss.

Capítulo 6

Documentación de directorios

6.1. Referencia del directorio Core

Gráfico de dependencias de directorios de Core:

Directorios

- directorio [Inc](#)
- directorio [Src](#)

6.2. Referencia del directorio Core/Inc

Gráfico de dependencias de directorios de Inc:

Archivos

- archivo [dma.h](#)
This file contains all the function prototypes for the dma.c file.
- archivo [formatter_data.h](#)
Define los paquetes de datos normalizados y la API de conversión a JSON.
- archivo [FreeRTOSConfig.h](#)
- archivo [gpio.h](#)
This file contains all the function prototypes for the gpio.c file.
- archivo [gps.h](#)
Interfaz pública del driver GPS basado en recepción UART.
- archivo [i2c.h](#)
This file contains all the function prototypes for the i2c.c file.
- archivo [main.h](#)
: Header for main.c file. This file contains the common defines of the application.
- archivo [sensor10DoF.h](#)
Interfaz pública del módulo de adquisición del sensor 10DoF.
- archivo [stm32f4xx_hal_conf.h](#)
- archivo [tim.h](#)

This file contains all the function prototypes for the tim.c file.

- archivo [usart.h](#)

This file contains all the function prototypes for the usart.c file.

- archivo [wifi.h](#)

Interfaz pública del driver WiFi basado en comandos AT.

- archivo [wifi_types.h](#)

Tipos compartidos por la interfaz WiFi y las capas de aplicación.

6.3. Referencia del directorio Core/Src

Gráfico de dependencias de directorios de Src:

Archivos

- archivo [formatter_data.c](#)

Conversión de paquetes de telemetría a cadenas de texto para publicación.

- archivo [freertos.c](#)

Planificación FreeRTOS de adquisición, empaquetado y envío MQTT de telemetría.

- archivo [gps.c](#)

Driver no bloqueante para recepción y procesamiento de tramas NMEA GPS.

- archivo [sensor10DoF.c](#)

Driver de adquisición para IMU 10DoF mediante bus I2C.

- archivo [stm32f4xx_hal_msp.c](#)

This file provides code for the MSP Initialization and de-Initialization codes.

- archivo [stm32f4xx_hal_timebase_tim.c](#)

HAL time base based on the hardware TIM.

- archivo [syscalls.c](#)

STM32CubeIDE Minimal System calls file.

- archivo [wifi.c](#)

Driver WiFi/MQTT basado en comandos AT con RX DMA circular y TX DMA sincronizado.

Capítulo 7

Documentación de estructuras de datos

7.1. Referencia de la estructura EnvDataPacket_t

Paquete normalizado con las magnitudes ambientales.

```
#include <formatter_data.h>
```

Campos de datos

- float [heading](#)
Rumbo estimado a partir del magnetómetro.
- float [temperature](#)
Temperatura medida por el sensor barométrico.
- float [pressure](#)
Presión atmosférica medida o calculada.
- uint32_t [env_timestamp](#)
Marca temporal asociada a la muestra ambiental.
- uint32_t [checksum](#)
Valor de comprobación usado para detectar paquetes inconsistentes.
- int [env_updated](#)
Indica si el paquete contiene datos ambientales pendientes de consumir.

7.1.1. Descripción detallada

Paquete normalizado con las magnitudes ambientales.

Mantiene juntas las variables ambientales calculadas por el módulo 10DoF para evitar que temperatura, presión y rumbo pertenezcan a instantes distintos al ser enviados por la capa de comunicación.

7.1.2. Documentación de campos

7.1.2.1. checksum

```
uint32_t EnvDataPacket_t::checksum
```

Valor de comprobación usado para detectar paquetes inconsistentes.

7.1.2.2. env_timestamp

```
uint32_t EnvDataPacket_t::env_timestamp
```

Marca temporal asociada a la muestra ambiental.

7.1.2.3. env_updated

```
int EnvDataPacket_t::env_updated
```

Indica si el paquete contiene datos ambientales pendientes de consumir.

7.1.2.4. heading

```
float EnvDataPacket_t::heading
```

Rumbo estimado a partir del magnetómetro.

7.1.2.5. pressure

```
float EnvDataPacket_t::pressure
```

Presión atmosférica medida o calculada.

7.1.2.6. temperature

```
float EnvDataPacket_t::temperature
```

Temperatura medida por el sensor barométrico.

La documentación de esta estructura está generada del siguiente archivo:

- [Core/Inc/formatter_data.h](#)

7.2. Referencia de la estructura gps_data_t

Última solución de navegación calculada por el módulo GPS.

```
#include <gps.h>
```

Campos de datos

- float `latitude`
Latitud en grados decimales.
- float `longitude`
Longitud en grados decimales.
- float `altitude`
Altitud proporcionada por el receptor GPS.
- float `speed_knots`
Velocidad en nudos, mantenida por compatibilidad con NMEA.
- float `speed_kmh`
Velocidad convertida a km/h para telemetría de aplicación.
- uint8_t `satellites`
Número de satélites usados o visibles según la sentencia procesada.
- `gps_fix_t` `fix`
Estado de fix asociado a la posición almacenada.
- bool `valid`
Indica si la última muestra procesada es apta para uso externo.

7.2.1. Descripción detallada

Última solución de navegación calculada por el módulo GPS.

La estructura conserva tanto la información de posición como metadatos de calidad. Esto permite que la aplicación decida si publica o descarta una muestra sin tener que volver a analizar la sentencia NMEA original.

7.2.2. Documentación de campos

7.2.2.1. `altitude`

```
float gps_data_t::altitude
```

Altitud proporcionada por el receptor GPS.

7.2.2.2. `fix`

```
gps_fix_t gps_data_t::fix
```

Estado de fix asociado a la posición almacenada.

7.2.2.3. `latitude`

```
float gps_data_t::latitude
```

Latitud en grados decimales.

7.2.2.4. longitude

```
float gps_data_t::longitude
```

Longitud en grados decimales.

7.2.2.5. satellites

```
uint8_t gps_data_t::satellites
```

Número de satélites usados o visibles según la sentencia procesada.

7.2.2.6. speed_kmh

```
float gps_data_t::speed_kmh
```

Velocidad convertida a km/h para telemetría de aplicación.

7.2.2.7. speed_knots

```
float gps_data_t::speed_knots
```

Velocidad en nudos, mantenida por compatibilidad con NMEA.

7.2.2.8. valid

```
bool gps_data_t::valid
```

Indica si la última muestra procesada es apta para uso externo.

La documentación de esta estructura está generada del siguiente archivo:

- [Core/Inc/gps.h](#)

7.3. Referencia de la estructura GpsDataPacket_t

Paquete normalizado con la posición procedente del receptor GPS.

```
#include <formatter_data.h>
```

Campos de datos

- double [altitude](#)
Altitud calculada por el receptor GPS.
- double [latitude](#)
Latitud geográfica en grados decimales.
- double [longitude](#)
Longitud geográfica en grados decimales.
- uint32_t [gps_timestamp](#)
Marca temporal asociada a la muestra GPS.
- uint32_t [checksum](#)
Valor de comprobación usado para detectar paquetes inconsistentes.
- int [gps_updated](#)
Indica si el paquete contiene una lectura GPS pendiente de consumir.

7.3.1. Descripción detallada

Paquete normalizado con la posición procedente del receptor GPS.

Se separa del paquete IMU porque la frecuencia y la validez de las tramas GPS son independientes de las lecturas inerciales. El flag de actualización evita publicar repetidamente una posición antigua cuando no llega una nueva trama válida.

7.3.2. Documentación de campos

7.3.2.1. altitude

```
double GpsDataPacket_t::altitude
```

Altitud calculada por el receptor GPS.

7.3.2.2. checksum

```
uint32_t GpsDataPacket_t::checksum
```

Valor de comprobación usado para detectar paquetes inconsistentes.

7.3.2.3. gps_timestamp

```
uint32_t GpsDataPacket_t::gps_timestamp
```

Marca temporal asociada a la muestra GPS.

7.3.2.4. gps_updated

```
int GpsDataPacket_t::gps_updated
```

Indica si el paquete contiene una lectura GPS pendiente de consumir.

7.3.2.5. latitude

```
double GpsDataPacket_t::latitude
```

Latitud geográfica en grados decimales.

7.3.2.6. longitude

```
double GpsDataPacket_t::longitude
```

Longitud geográfica en grados decimales.

La documentación de esta estructura está generada del siguiente archivo:

- [Core/Inc/formatter_data.h](#)

7.4. Referencia de la estructura ImuDataPacket_t

Paquete normalizado con las magnitudes procedentes de la IMU.

```
#include <formatter_data.h>
```

Campos de datos

- float [accel](#) [3]
Aceleración en los tres ejes.
- float [gyro](#) [3]
Velocidad angular en los tres ejes.
- float [mag](#) [3]
Campo magnético en los tres ejes.
- uint32_t [imu_timestamp](#)
Marca temporal asociada a la muestra IMU.
- uint32_t [checksum](#)
Valor de comprobación usado para detectar paquetes inconsistentes.
- int [imu_updated](#)
Indica si el paquete contiene una lectura IMU pendiente de consumir.

7.4.1. Descripción detallada

Paquete normalizado con las magnitudes procedentes de la IMU.

Agrupar acelerómetro, giróscopo y magnetómetro en un único bloque para que la tarea de publicación pueda tratar la lectura inercial como una muestra atómica. El timestamp permite ordenar las muestras aunque se reciban con distinta latencia dentro del pipeline.

7.4.2. Documentación de campos

7.4.2.1. accel

```
float ImuDataPacket_t::accel[3]
```

Aceleración en los tres ejes.

7.4.2.2. checksum

```
uint32_t ImuDataPacket_t::checksum
```

Valor de comprobación usado para detectar paquetes inconsistentes.

7.4.2.3. gyro

```
float ImuDataPacket_t::gyro[3]
```

Velocidad angular en los tres ejes.

7.4.2.4. imu_timestamp

```
uint32_t ImuDataPacket_t::imu_timestamp
```

Marca temporal asociada a la muestra IMU.

7.4.2.5. imu_updated

```
int ImuDataPacket_t::imu_updated
```

Indica si el paquete contiene una lectura IMU pendiente de consumir.

7.4.2.6. mag

```
float ImuDataPacket_t::mag[3]
```

Campo magnético en los tres ejes.

La documentación de esta estructura está generada del siguiente archivo:

- [Core/Inc/formatter_data.h](#)

7.5. Referencia de la estructura Sensor10DoF_t

Conjunto completo de medidas calculadas por el módulo 10DoF.

```
#include <sensor10DoF.h>
```

Campos de datos

- uint32_t [timestamp_ms](#)
Marca temporal de la muestra en milisegundos.
- float [accel_x_ms2](#)
Aceleración en el eje X expresada en m/s^2 .
- float [accel_y_ms2](#)
Aceleración en el eje Y expresada en m/s^2 .
- float [accel_z_ms2](#)
Aceleración en el eje Z expresada en m/s^2 .
- float [gyro_x_dps](#)
Velocidad angular en el eje X expresada en grados/s.
- float [gyro_y_dps](#)
Velocidad angular en el eje Y expresada en grados/s.
- float [gyro_z_dps](#)
Velocidad angular en el eje Z expresada en grados/s.
- float [mag_x_uT](#)
Campo magnético en el eje X expresado en microteslas.
- float [mag_y_uT](#)
Campo magnético en el eje Y expresado en microteslas.
- float [mag_z_uT](#)
Campo magnético en el eje Z expresado en microteslas.
- float [heading_deg](#)
Rumbo horizontal estimado en grados.
- float [roll_deg](#)
Ángulo de roll estimado en grados.
- float [pitch_deg](#)
Ángulo de pitch estimado en grados.
- float [temperature_c](#)
Temperatura medida por el barómetro en grados Celsius.
- float [pressure_pa](#)
Presión atmosférica en pascales.
- float [pressure_hpa](#)
Presión atmosférica en hectopascales.
- float [pressure_atm](#)
Presión atmosférica expresada en atmósferas.
- float [altitude_m](#)
Altitud estimada a partir de la presión barométrica en metros.

7.5.1. Descripción detallada

Conjunto completo de medidas calculadas por el módulo 10DoF.

La estructura agrupa todas las magnitudes derivadas de una lectura para que la aplicación pueda mover una muestra coherente entre tareas. Las unidades se expresan en formato físico de alto nivel y no en cuentas ADC o registros del sensor.

7.5.2. Documentación de campos

7.5.2.1. accel_x_ms2

```
float Sensor10DoF_t::accel_x_ms2
```

Aceleración en el eje X expresada en m/s^2 .

7.5.2.2. accel_y_ms2

```
float Sensor10DoF_t::accel_y_ms2
```

Aceleración en el eje Y expresada en m/s^2 .

7.5.2.3. accel_z_ms2

```
float Sensor10DoF_t::accel_z_ms2
```

Aceleración en el eje Z expresada en m/s^2 .

7.5.2.4. altitude_m

```
float Sensor10DoF_t::altitude_m
```

Altitud estimada a partir de la presión barométrica en metros.

7.5.2.5. gyro_x_dps

```
float Sensor10DoF_t::gyro_x_dps
```

Velocidad angular en el eje X expresada en grados/s.

7.5.2.6. gyro_y_dps

```
float Sensor10DoF_t::gyro_y_dps
```

Velocidad angular en el eje Y expresada en grados/s.

7.5.2.7. gyro_z_dps

```
float Sensor10DoF_t::gyro_z_dps
```

Velocidad angular en el eje Z expresada en grados/s.

7.5.2.8. heading_deg

```
float Sensor10DoF_t::heading_deg
```

Rumbo horizontal estimado en grados.

7.5.2.9. mag_x_uT

```
float Sensor10DoF_t::mag_x_uT
```

Campo magnético en el eje X expresado en microteslas.

7.5.2.10. mag_y_uT

```
float Sensor10DoF_t::mag_y_uT
```

Campo magnético en el eje Y expresado en microteslas.

7.5.2.11. mag_z_uT

```
float Sensor10DoF_t::mag_z_uT
```

Campo magnético en el eje Z expresado en microteslas.

7.5.2.12. pitch_deg

```
float Sensor10DoF_t::pitch_deg
```

Ángulo de pitch estimado en grados.

7.5.2.13. pressure_atm

```
float Sensor10DoF_t::pressure_atm
```

Presión atmosférica expresada en atmósferas.

7.5.2.14. pressure_hpa

```
float Sensor10DoF_t::pressure_hpa
```

Presión atmosférica en hectopascales.

7.5.2.15. pressure_pa

```
float Sensor10DoF_t::pressure_pa
```

Presión atmosférica en pascales.

7.5.2.16. roll_deg

```
float Sensor10DoF_t::roll_deg
```

Ángulo de roll estimado en grados.

7.5.2.17. temperature_c

```
float Sensor10DoF_t::temperature_c
```

Temperatura medida por el barómetro en grados Celsius.

7.5.2.18. timestamp_ms

```
uint32_t Sensor10DoF_t::timestamp_ms
```

Marca temporal de la muestra en milisegundos.

La documentación de esta estructura está generada del siguiente archivo:

- Core/Inc/[sensor10DoF.h](#)

Capítulo 8

Documentación de archivos

8.1. Referencia del archivo Core/Inc/dma.h

This file contains all the function prototypes for the dma.c file.

```
#include "main.h"
```

Gráfico de dependencias incluidas en dma.h:

Funciones

- void [MX_DMA_Init](#) (void)

8.1.1. Descripción detallada

This file contains all the function prototypes for the dma.c file.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.1.2. Documentación de funciones

8.1.2.1. MX_DMA_Init()

```
void MX_DMA_Init (  
    void )
```

8.2. dma.h

[Ir a la documentación de este archivo.](#)

```

00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020 /* Define to prevent recursive inclusion -----*/
00021 #ifndef __DMA_H__
00022 #define __DMA_H__
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00028 /* Includes -----*/
00029 #include "main.h"
00030
00031 /* DMA memory to memory transfer handles -----*/
00032
00033 /* USER CODE BEGIN Includes */
00034
00035 /* USER CODE END Includes */
00036
00037 /* USER CODE BEGIN Private defines */
00038
00039 /* USER CODE END Private defines */
00040
00041 void MX_DMA_Init(void);
00042
00043 /* USER CODE BEGIN Prototypes */
00044
00045 /* USER CODE END Prototypes */
00046
00047 #ifdef __cplusplus
00048 }
00049 #endif
00050
00051 #endif /* __DMA_H__ */
00052

```

8.3. Referencia del archivo Core/Inc/formatter_data.h

Define los paquetes de datos normalizados y la API de conversión a JSON.

```

#include <stdint.h>
#include <stddef.h>

```

Gráfico de dependencias incluidas en formatter_data.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Estructuras de datos

- struct [ImuDataPacket_t](#)
Paquete normalizado con las magnitudes procedentes de la IMU.
- struct [EnvDataPacket_t](#)
Paquete normalizado con las magnitudes ambientales.
- struct [GpsDataPacket_t](#)
Paquete normalizado con la posición procedente del receptor GPS.

Funciones

- void `GpsData_to_json` (char *buffer, size_t buffer_size, `GpsDataPacket_t` *data)
Serializa un paquete GPS en formato JSON.
- void `ImuData_to_json` (char *buffer, size_t buffer_size, `ImuDataPacket_t` *data)
Serializa un paquete IMU en formato JSON.
- void `EnvData_to_json` (char *buffer, size_t buffer_size, `EnvDataPacket_t` *data)
Serializa un paquete ambiental en formato JSON.
- void `GpsDataPacket_Init` (`GpsDataPacket_t` *packet)
Inicializa un paquete GPS con valores seguros.
- void `ImuDataPacket_Init` (`ImuDataPacket_t` *packet)
Inicializa un paquete IMU con valores seguros.
- void `EnvDataPacket_Init` (`EnvDataPacket_t` *packet)
Inicializa un paquete ambiental con valores seguros.

8.3.1. Descripción detallada

Define los paquetes de datos normalizados y la API de conversión a JSON.

Autor

Lorenzo

Fecha

15 may 2026

Este módulo actúa como frontera entre las tareas de adquisición y la capa de comunicación. Las estructuras agrupan los datos por origen funcional —IMU, entorno y GPS— para que FreeRTOS pueda intercambiar muestras completas sin acoplar directamente los drivers de sensores con el formato usado por MQTT.

Los campos de actualización y checksum permiten distinguir muestras nuevas y validar la coherencia básica del paquete antes de serializarlo.

8.3.2. Documentación de funciones

8.3.2.1. `EnvData_to_json()`

```
void EnvData_to_json (  
    char * buffer,  
    size_t buffer_size,  
    EnvDataPacket_t * data)
```

Serializa un paquete ambiental en formato JSON.

Serializa el paquete ambiental en el formato compacto de telemetría.

Parámetros

out	<i>buffer</i>	Buffer de salida donde se escribe la cadena JSON.
in	<i>buffer_size</i>	Tamaño disponible en el buffer de salida.
in	<i>data</i>	Paquete ambiental que contiene la última muestra normalizada.

Precondición

buffer debe apuntar a memoria válida y *buffer_size* debe ser suficiente para almacenar el JSON generado.

data debe apuntar a un paquete inicializado.

Serializa un paquete ambiental en formato JSON.

Parámetros

out	<i>buffer</i>	Buffer donde se escribe la cadena generada.
in	<i>buffer_size</i>	Tamaño total disponible en <i>buffer</i> .
in	<i>data</i>	Paquete ambiental con rumbo, temperatura y presión.

El formato se mantiene coherente con el resto de tramas del sistema para que el receptor pueda aplicar una única lógica de separación de campos, sin tratar este paquete como un caso especial.

Precondición

buffer debe apuntar a memoria válida con al menos *buffer_size* bytes.

data debe apuntar a un paquete ambiental válido.

Gráfico de llamadas a esta función:

8.3.2.2. EnvDataPacket_Init()

```
void EnvDataPacket_Init (  
    EnvDataPacket_t * packet)
```

Inicializa un paquete ambiental con valores seguros.

Inicializa un paquete ambiental a un estado conocido.

Parámetros

out	<i>packet</i>	Paquete ambiental a inicializar.
-----	---------------	----------------------------------

Postcondición

El paquete queda preparado para ser usado por el pipeline antes de la primera lectura barométrica o magnética.

Inicializa un paquete ambiental con valores seguros.

Parámetros

out	<i>packet</i>	Paquete ambiental que se reinicia.
-----	---------------	------------------------------------

Se utiliza el mismo patrón que en el resto de paquetes para mantener una semántica uniforme entre productores y consumidores de datos en FreeRTOS.

Precondición

packet debe apuntar a una estructura válida.

Gráfico de llamadas a esta función:

8.3.2.3. GpsData_to_json()

```
void GpsData_to_json (
    char * buffer,
    size_t buffer_size,
    GpsDataPacket_t * data)
```

Serializa un paquete GPS en formato JSON.

Serializa el paquete GPS en el formato compacto de telemetría.

Parámetros

out	<i>buffer</i>	Buffer de salida donde se escribe la cadena JSON.
in	<i>buffer_size</i>	Tamaño disponible en el buffer de salida.
in	<i>data</i>	Paquete GPS que contiene la última muestra normalizada.

Precondición

buffer debe apuntar a memoria válida y *buffer_size* debe ser suficiente para almacenar el JSON generado.

data debe apuntar a un paquete inicializado.

Nota

La función está pensada para preparar payloads MQTT sin exponer al publicador los detalles internos de la estructura GPS.

Serializa un paquete GPS en formato JSON.

Parámetros

out	<i>buffer</i>	Buffer donde se escribe la cadena generada.
in	<i>buffer_size</i>	Tamaño total disponible en <i>buffer</i> .
in	<i>data</i>	Paquete GPS con altitud, latitud y longitud.

Las coordenadas se limitan a cuatro decimales para contener el tamaño del mensaje y porque la precisión resultante es suficiente para la visualización general de posición usada por la interfaz del proyecto.

Precondición

buffer debe apuntar a memoria válida con al menos *buffer_size* bytes.
data debe apuntar a un paquete GPS válido.

Gráfico de llamadas a esta función:

8.3.2.4. GpsDataPacket_Init()

```
void GpsDataPacket_Init (  
    GpsDataPacket_t * packet)
```

Inicializa un paquete GPS con valores seguros.

Inicializa un paquete GPS a un estado conocido.

Parámetros

out	<i>packet</i>	Paquete GPS a inicializar.
-----	---------------	----------------------------

Postcondición

El paquete queda preparado para ser usado por el pipeline antes de que llegue la primera trama GPS válida.

Inicializa un paquete GPS con valores seguros.

Parámetros

out	<i>packet</i>	Paquete GPS que se reinicia.
-----	---------------	------------------------------

Se borra toda la estructura antes de fijar los flags de control para evitar que una lectura parcial o antigua se interprete como nueva telemetría válida.

Precondición

packet debe apuntar a una estructura válida.

Gráfico de llamadas a esta función:

8.3.2.5. ImuData_to_json()

```
void ImuData_to_json (
    char * buffer,
    size_t buffer_size,
    ImuDataPacket_t * data)
```

Serializa un paquete IMU en formato JSON.

Serializa el paquete inercial en el formato compacto de telemetría.

Parámetros

out	<i>buffer</i>	Buffer de salida donde se escribe la cadena JSON.
in	<i>buffer_size</i>	Tamaño disponible en el buffer de salida.
in	<i>data</i>	Paquete IMU que contiene la última muestra normalizada.

Precondición

buffer debe apuntar a memoria válida y *buffer_size* debe ser suficiente para almacenar el JSON generado.

data debe apuntar a un paquete inicializado.

Nota

Centralizar la conversión evita duplicar el formato de telemetría en las tareas de FreeRTOS.

Serializa un paquete IMU en formato JSON.

Parámetros

out	<i>buffer</i>	Buffer donde se escribe la cadena generada.
in	<i>buffer_size</i>	Tamaño total disponible en <i>buffer</i> .
in	<i>data</i>	Paquete IMU con acelerómetro, giróscopo y magnetómetro.

El formato usa claves delimitadas con | y campos separados por / para evitar comillas dobles dentro del payload. Esta decisión reduce problemas al insertar la cadena dentro de comandos AT, donde las comillas forman parte de la sintaxis del propio comando.

Precondición

buffer debe apuntar a memoria válida con al menos *buffer_size* bytes.

data debe apuntar a un paquete IMU válido.

Nota

La función no devuelve estado; si el buffer no es suficiente, `snprintf` truncará la salida manteniendo la terminación nula cuando el tamaño lo permita.

Gráfico de llamadas a esta función:

8.3.2.6. ImuDataPacket_Init()

```
void ImuDataPacket_Init (
    ImuDataPacket_t * packet)
```

Inicializa un paquete IMU con valores seguros.

Inicializa un paquete IMU a un estado conocido.

Parámetros

out	<i>packet</i>	Paquete IMU a inicializar.
-----	---------------	----------------------------

Postcondición

El paquete queda preparado para ser usado por el pipeline antes de la primera lectura del sensor.

Inicializa un paquete IMU con valores seguros.

Parámetros

out	<i>packet</i>	Paquete IMU que se reinicia.
-----	---------------	------------------------------

El reinicio explícito del flag de actualización permite que las tareas consumidoras distingan entre datos recién publicados y memoria simplemente reservada o reutilizada.

Precondición

packet debe apuntar a una estructura válida.

Gráfico de llamadas a esta función:

8.4. formatter_data.h

[Ir a la documentación de este archivo.](#)

```
00001
00016
00017 #ifndef FORMATTER_DATA_H
00018 #define FORMATTER_DATA_H
00019
00020 #include <stdint.h>
00021 #include <stddef.h>
00022
00032 typedef struct
00033 {
00035     float accel[3];
00036
00038     float gyro[3];
00039
00041     float mag[3];
00042
00044     uint32_t imu_timestamp;
00045
00047     uint32_t checksum;
00048
00050     int imu_updated;
00051
00052
00053 } ImuDataPacket_t;
```

```

00054
00063 typedef struct
00064 {
00066     float heading;
00067
00069     float temperature;
00070
00072     float pressure;
00073
00075     uint32_t env_timestamp;
00076
00078     uint32_t checksum;
00079
00081     int env_updated;
00082
00083
00084 } EnvDataPacket_t;
00085
00095 typedef struct
00096 {
00098     double altitude;
00099
00101     double latitude;
00102
00104     double longitude;
00105
00107     uint32_t gps_timestamp;
00108
00110     uint32_t checksum;
00111
00113     int gps_updated;
00114 } GpsDataPacket_t;
00115
00117 void GpsData_to_json(char *buffer,
00133                     size_t buffer_size,
00134                     GpsDataPacket_t *data);
00135
00150 void ImuData_to_json(char *buffer,
00151                     size_t buffer_size,
00152                     ImuDataPacket_t *data);
00153
00165 void EnvData_to_json(char *buffer,
00166                     size_t buffer_size,
00167                     EnvDataPacket_t *data);
00168
00177 void GpsDataPacket_Init (GpsDataPacket_t *packet);
00178
00187 void ImuDataPacket_Init (ImuDataPacket_t *packet);
00188
00197 void EnvDataPacket_Init (EnvDataPacket_t *packet);
00198
00199 #endif

```

8.5. Referencia del archivo Core/Inc/FreeRTOSConfig.h

defines

- #define CMSIS_device_header "stm32f4xx.h"
- #define configENABLE_FPU 0
- #define configENABLE_MPU 0
- #define configUSE_PREEMPTION 1
- #define configSUPPORT_STATIC_ALLOCATION 1
- #define configSUPPORT_DYNAMIC_ALLOCATION 1
- #define configUSE_IDLE_HOOK 0
- #define configUSE_TICK_HOOK 0
- #define configCPU_CLOCK_HZ (SystemCoreClock)
- #define configTICK_RATE_HZ ((TickType_t)1000)
- #define configMAX_PRIORITIES (56)
- #define configMINIMAL_STACK_SIZE ((uint16_t)128)
- #define configTOTAL_HEAP_SIZE ((size_t)20000)

- #define configMAX_TASK_NAME_LEN (16)
- #define configUSE_TRACE_FACILITY 1
- #define configUSE_16_BIT_TICKS 0
- #define configUSE_MUTEXES 1
- #define configQUEUE_REGISTRY_SIZE 8
- #define configUSE_RECURSIVE_MUTEXES 1
- #define configUSE_COUNTING_SEMAPHORES 1
- #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
- #define configMESSAGE_BUFFER_LENGTH_TYPE size_t
- #define configUSE_CO_ROUTINES 0
- #define configMAX_CO_ROUTINE_PRIORITIES (2)
- #define configUSE_TIMERS 1
- #define configTIMER_TASK_PRIORITY (2)
- #define configTIMER_QUEUE_LENGTH 10
- #define configTIMER_TASK_STACK_DEPTH 256
- #define configUSE_NEWLIB_REENTRANT 1
- #define configUSE_OS2_THREAD_SUSPEND_RESUME 1
- #define configUSE_OS2_THREAD_ENUMERATE 1
- #define configUSE_OS2_EVENTFLAGS_FROM_ISR 1
- #define configUSE_OS2_THREAD_FLAGS 1
- #define configUSE_OS2_TIMER 1
- #define configUSE_OS2_MUTEX 1
- #define INCLUDE_vTaskPrioritySet 1
- #define INCLUDE_uxTaskPriorityGet 1
- #define INCLUDE_vTaskDelete 1
- #define INCLUDE_vTaskCleanUpResources 0
- #define INCLUDE_vTaskSuspend 1
- #define INCLUDE_vTaskDelayUntil 1
- #define INCLUDE_vTaskDelay 1
- #define INCLUDE_xTaskGetSchedulerState 1
- #define INCLUDE_xTimerPendFunctionCall 1
- #define INCLUDE_xQueueGetMutexHolder 1
- #define INCLUDE_uxTaskGetStackHighWaterMark 1
- #define INCLUDE_xTaskGetCurrentTaskHandle 1
- #define INCLUDE_eTaskGetState 1
- #define USE_FreeRTOS_HEAP_4
- #define configPRIO_BITS 4
- #define configLIBRARY_LOWEST_INTERRUPT_PRIORITY 15
- #define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5
- #define configKERNEL_INTERRUPT_PRIORITY (configLIBRARY_LOWEST_INTERRUPT_PRIORITY << (8 - configPRIO_BITS))
- #define configMAX_SYSCALL_INTERRUPT_PRIORITY (configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS))
- #define configASSERT(x)
- #define vPortSVCHandler SVC_Handler
- #define xPortPendSVHandler PendSV_Handler
- #define USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION 0

8.5.1. Documentación de «define»

8.5.1.1. CMSIS_device_header

```
#define CMSIS_device_header "stm32f4xx.h"
```

8.5.1.2. configASSERT

```
#define configASSERT(  
    x)
```

Valor:

```
if ( (x) == 0 ) {taskDISABLE_INTERRUPTS(); for( ;; );}
```

8.5.1.3. configCPU_CLOCK_HZ

```
#define configCPU_CLOCK_HZ ( SystemCoreClock )
```

8.5.1.4. configENABLE_FPU

```
#define configENABLE_FPU 0
```

8.5.1.5. configENABLE_MPU

```
#define configENABLE_MPU 0
```

8.5.1.6. configKERNEL_INTERRUPT_PRIORITY

```
#define configKERNEL_INTERRUPT_PRIORITY ( configLIBRARY_LOWEST_INTERRUPT_PRIORITY << (8 -  
configPRIO_BITS) )
```

8.5.1.7. configLIBRARY_LOWEST_INTERRUPT_PRIORITY

```
#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY 15
```

8.5.1.8. configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY

```
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5
```

8.5.1.9. configMAX_CO_ROUTINE_PRIORITIES

```
#define configMAX_CO_ROUTINE_PRIORITIES ( 2 )
```

8.5.1.10. configMAX_PRIORITIES

```
#define configMAX_PRIORITIES ( 56 )
```

8.5.1.11. configMAX_SYSCALL_INTERRUPT_PRIORITY

```
#define configMAX_SYSCALL_INTERRUPT_PRIORITY ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY <<  
(8 - configPRIO_BITS) )
```

8.5.1.12. configMAX_TASK_NAME_LEN

```
#define configMAX_TASK_NAME_LEN ( 16 )
```

8.5.1.13. configMESSAGE_BUFFER_LENGTH_TYPE

```
#define configMESSAGE_BUFFER_LENGTH_TYPE size_t
```

8.5.1.14. configMINIMAL_STACK_SIZE

```
#define configMINIMAL_STACK_SIZE ((uint16_t)128)
```

8.5.1.15. configPRIO_BITS

```
#define configPRIO_BITS 4
```

8.5.1.16. configQUEUE_REGISTRY_SIZE

```
#define configQUEUE_REGISTRY_SIZE 8
```

8.5.1.17. configSUPPORT_DYNAMIC_ALLOCATION

```
#define configSUPPORT_DYNAMIC_ALLOCATION 1
```

8.5.1.18. configSUPPORT_STATIC_ALLOCATION

```
#define configSUPPORT_STATIC_ALLOCATION 1
```

8.5.1.19. configTICK_RATE_HZ

```
#define configTICK_RATE_HZ ((TickType_t)1000)
```

8.5.1.20. configTIMER_QUEUE_LENGTH

```
#define configTIMER_QUEUE_LENGTH 10
```

8.5.1.21. configTIMER_TASK_PRIORITY

```
#define configTIMER_TASK_PRIORITY ( 2 )
```

8.5.1.22. configTIMER_TASK_STACK_DEPTH

```
#define configTIMER_TASK_STACK_DEPTH 256
```

8.5.1.23. configTOTAL_HEAP_SIZE

```
#define configTOTAL_HEAP_SIZE ((size_t)20000)
```

8.5.1.24. configUSE_16_BIT_TICKS

```
#define configUSE_16_BIT_TICKS 0
```

8.5.1.25. configUSE_CO_ROUTINES

```
#define configUSE_CO_ROUTINES 0
```

8.5.1.26. configUSE_COUNTING_SEMAPHORES

```
#define configUSE_COUNTING_SEMAPHORES 1
```

8.5.1.27. configUSE_IDLE_HOOK

```
#define configUSE_IDLE_HOOK 0
```

8.5.1.28. configUSE_MUTEXES

```
#define configUSE_MUTEXES 1
```

8.5.1.29. configUSE_NEWLIB_REENTRANT

```
#define configUSE_NEWLIB_REENTRANT 1
```

8.5.1.30. configUSE_OS2_EVENTFLAGS_FROM_ISR

```
#define configUSE_OS2_EVENTFLAGS_FROM_ISR 1
```

8.5.1.31. configUSE_OS2_MUTEX

```
#define configUSE_OS2_MUTEX 1
```

8.5.1.32. configUSE_OS2_THREAD_ENUMERATE

```
#define configUSE_OS2_THREAD_ENUMERATE 1
```

8.5.1.33. configUSE_OS2_THREAD_FLAGS

```
#define configUSE_OS2_THREAD_FLAGS 1
```

8.5.1.34. configUSE_OS2_THREAD_SUSPEND_RESUME

```
#define configUSE_OS2_THREAD_SUSPEND_RESUME 1
```

8.5.1.35. configUSE_OS2_TIMER

```
#define configUSE_OS2_TIMER 1
```

8.5.1.36. configUSE_PORT_OPTIMISED_TASK_SELECTION

```
#define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
```

8.5.1.37. configUSE_PREEMPTION

```
#define configUSE_PREEMPTION 1
```

8.5.1.38. configUSE_RECURSIVE_MUTEXES

```
#define configUSE_RECURSIVE_MUTEXES 1
```

8.5.1.39. configUSE_TICK_HOOK

```
#define configUSE_TICK_HOOK 0
```

8.5.1.40. configUSE_TIMERS

```
#define configUSE_TIMERS 1
```

8.5.1.41. configUSE_TRACE_FACILITY

```
#define configUSE_TRACE_FACILITY 1
```

8.5.1.42. INCLUDE_eTaskGetState

```
#define INCLUDE_eTaskGetState 1
```

8.5.1.43. INCLUDE_uxTaskGetStackHighWaterMark

```
#define INCLUDE_uxTaskGetStackHighWaterMark 1
```

8.5.1.44. INCLUDE_uxTaskPriorityGet

```
#define INCLUDE_uxTaskPriorityGet 1
```

8.5.1.45. INCLUDE_vTaskCleanUpResources

```
#define INCLUDE_vTaskCleanUpResources 0
```

8.5.1.46. INCLUDE_vTaskDelay

```
#define INCLUDE_vTaskDelay 1
```

8.5.1.47. INCLUDE_vTaskDelayUntil

```
#define INCLUDE_vTaskDelayUntil 1
```

8.5.1.48. INCLUDE_vTaskDelete

```
#define INCLUDE_vTaskDelete 1
```

8.5.1.49. INCLUDE_vTaskPrioritySet

```
#define INCLUDE_vTaskPrioritySet 1
```

8.5.1.50. INCLUDE_vTaskSuspend

```
#define INCLUDE_vTaskSuspend 1
```

8.5.1.51. INCLUDE_xQueueGetMutexHolder

```
#define INCLUDE_xQueueGetMutexHolder 1
```

8.5.1.52. INCLUDE_xTaskGetCurrentTaskHandle

```
#define INCLUDE_xTaskGetCurrentTaskHandle 1
```

8.5.1.53. INCLUDE_xTaskGetSchedulerState

```
#define INCLUDE_xTaskGetSchedulerState 1
```

8.5.1.54. INCLUDE_xTimerPendFunctionCall

```
#define INCLUDE_xTimerPendFunctionCall 1
```

8.5.1.55. USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION

```
#define USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION 0
```

8.5.1.56. USE_FreeRTOS_HEAP_4

```
#define USE_FreeRTOS_HEAP_4
```

8.5.1.57. vPortSVCHandler

```
#define vPortSVCHandler SVC_Handler
```

8.5.1.58. xPortPendSVHandler

```
#define xPortPendSVHandler PendSV_Handler
```

8.6. FreeRTOSConfig.h

[Ir a la documentación de este archivo.](#)

```

00001 /* USER CODE BEGIN Header */
00002 /*
00003  * FreeRTOS Kernel V10.3.1
00004  * Portion Copyright (C) 2017 Amazon.com, Inc. or its affiliates. All Rights Reserved.
00005  * Portion Copyright (C) 2019 STMicroelectronics, Inc. All Rights Reserved.
00006  *
00007  * Permission is hereby granted, free of charge, to any person obtaining a copy of
00008  * this software and associated documentation files (the "Software"), to deal in
00009  * the Software without restriction, including without limitation the rights to
00010  * use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of
00011  * the Software, and to permit persons to whom the Software is furnished to do so,
00012  * subject to the following conditions:
00013  *
00014  * The above copyright notice and this permission notice shall be included in all
00015  * copies or substantial portions of the Software.
00016  *
00017  * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00018  * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS
00019  * FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR
00020  * COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER
00021  * IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00022  * CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00023  *
00024  * http://www.FreeRTOS.org
00025  * http://aws.amazon.com/freertos
00026  *
00027  * 1 tab == 4 spaces!
00028  */
00029 /* USER CODE END Header */
00030
00031 #ifndef FREERTOS_CONFIG_H
00032 #define FREERTOS_CONFIG_H
00033
00034 /*-----
00035  * Application specific definitions.
00036  *
00037  * These definitions should be adjusted for your particular hardware and
00038  * application requirements.
00039  *
00040  * These parameters and more are described within the 'configuration' section of the
00041  * FreeRTOS API documentation available on the FreeRTOS.org web site.
00042  *
00043  * See http://www.freertos.org/a00110.html
00044  *-----*/
00045
00046 /* USER CODE BEGIN Includes */
00047 /* Section where include file can be added */
00048 /* USER CODE END Includes */
00049
00050 /* Ensure definitions are only used by the compiler, and not by the assembler. */
00051 #if defined(__ICCARM__) || defined(__CC_ARM) || defined(__GNUC__)
00052     #include <stdint.h>
00053     extern uint32_t SystemCoreClock;
00054 #endif
00055 #ifndef CMSIS_device_header
00056 #define CMSIS_device_header "stm32f4xx.h"
00057 #endif /* CMSIS_device_header */
00058
00059 #define configENABLE_FPU 0
00060 #define configENABLE_MPU 0
00061
00062 #define configUSE_PREEMPTION 1
00063 #define configSUPPORT_STATIC_ALLOCATION 1
00064 #define configSUPPORT_DYNAMIC_ALLOCATION 1
00065 #define configUSE_IDLE_HOOK 0
00066 #define configUSE_TICK_HOOK 0
00067 #define configCPU_CLOCK_HZ ( SystemCoreClock )
00068 #define configTICK_RATE_HZ ((TickType_t)1000)
00069 #define configMAX_PRIORITIES ( 56 )
00070 #define configMINIMAL_STACK_SIZE ((uint16_t)128)
00071 #define configTOTAL_HEAP_SIZE ((size_t)20000)
00072 #define configMAX_TASK_NAME_LEN ( 16 )
00073 #define configUSE_TRACE_FACILITY 1
00074 #define configUSE_16_BIT_TICKS 0
00075 #define configUSE_MUTEXES 1
00076 #define configQUEUE_REGISTRY_SIZE 8
00077 #define configUSE_RECURSIVE_MUTEXES 1
00078 #define configUSE_COUNTING_SEMAPHORES 1
00079 #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
00080 /* USER CODE BEGIN MESSAGE_BUFFER_LENGTH_TYPE */
00081 /* Defaults to size_t for backward compatibility, but can be changed
00082    if lengths will always be less than the number of bytes in a size_t. */

```

```

00083 #define configMESSAGE_BUFFER_LENGTH_TYPE      size_t
00084 /* USER CODE END MESSAGE_BUFFER_LENGTH_TYPE */
00085
00086 /* Co-routine definitions. */
00087 #define configUSE_CO_ROUTINES                    0
00088 #define configMAX_CO_ROUTINE_PRIORITIES          ( 2 )
00089
00090 /* Software timer definitions. */
00091 #define configUSE_TIMERS                          1
00092 #define configTIMER_TASK_PRIORITY                ( 2 )
00093 #define configTIMER_QUEUE_LENGTH                10
00094 #define configTIMER_TASK_STACK_DEPTH            256
00095
00096 /* The following flag must be enabled only when using newlib */
00097 #define configUSE_NEWLIB_REENTRANT               1
00098
00099 /* CMSIS-RTOS V2 flags */
00100 #define configUSE_OS2_THREAD_SUSPEND_RESUME      1
00101 #define configUSE_OS2_THREAD_ENUMERATE          1
00102 #define configUSE_OS2_EVENTFLAGS_FROM_ISR        1
00103 #define configUSE_OS2_THREAD_FLAGS              1
00104 #define configUSE_OS2_TIMER                      1
00105 #define configUSE_OS2_MUTEX                      1
00106
00107 /* Set the following definitions to 1 to include the API function, or zero
00108 to exclude the API function. */
00109 #define INCLUDE_vTaskPrioritySet                  1
00110 #define INCLUDE_uxTaskPriorityGet                 1
00111 #define INCLUDE_vTaskDelete                      1
00112 #define INCLUDE_vTaskCleanUpResources            0
00113 #define INCLUDE_vTaskSuspend                     1
00114 #define INCLUDE_vTaskDelayUntil                  1
00115 #define INCLUDE_vTaskDelay                       1
00116 #define INCLUDE_xTaskGetSchedulerState           1
00117 #define INCLUDE_xTimerPendFunctionCall           1
00118 #define INCLUDE_xQueueGetMutexHolder             1
00119 #define INCLUDE_uxTaskGetStackHighWaterMark      1
00120 #define INCLUDE_xTaskGetCurrentTaskHandle        1
00121 #define INCLUDE_eTaskGetState                    1
00122
00123 /*
00124 * The CMSIS-RTOS V2 FreeRTOS wrapper is dependent on the heap implementation used
00125 * by the application thus the correct define need to be enabled below
00126 */
00127 #define USE_FreeRTOS_HEAP_4
00128
00129 /* Cortex-M specific definitions. */
00130 #ifndef __NVIC_PRIO_BITS
00131 /* __BVIC_PRIO_BITS will be specified when CMSIS is being used. */
00132 #define configPRIO_BITS                          __NVIC_PRIO_BITS
00133 #else
00134 #define configPRIO_BITS                          4
00135 #endif
00136
00137 /* The lowest interrupt priority that can be used in a call to a "set priority"
00138 function. */
00139 #define configLIBRARY_LOWEST_INTERRUPT_PRIORITY   15
00140
00141 /* The highest interrupt priority that can be used by any interrupt service
00142 routine that makes calls to interrupt safe FreeRTOS API functions. DO NOT CALL
00143 INTERRUPT SAFE FREERTOS API FUNCTIONS FROM ANY INTERRUPT THAT HAS A HIGHER
00144 PRIORITY THAN THIS! (higher priorities are lower numeric values. */
00145 #define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5
00146
00147 /* Interrupt priorities used by the kernel port layer itself. These are generic
00148 to all Cortex-M ports, and do not rely on any particular library functions. */
00149 #define configKERNEL_INTERRUPT_PRIORITY           ( configLIBRARY_LOWEST_INTERRUPT_PRIORITY « (8 - configPRIO_BITS) )
00150 /* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to zero !!!!
00151 See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
00152 #define configMAX_SYSCALL_INTERRUPT_PRIORITY      ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY « (8 - configPRIO_BITS) )
00153
00154 /* Normal assert() semantics without relying on the provision of an assert.h
00155 header file. */
00156 /* USER CODE BEGIN 1 */
00157 #define configASSERT( x ) if ((x) == 0) {taskDISABLE_INTERRUPTS(); for( ;; );}
00158 /* USER CODE END 1 */
00159
00160 /* Definitions that map the FreeRTOS port interrupt handlers to their CMSIS
00161 standard names. */
00162 #define vPortSVCHandler      SVC_Handler
00163 #define xPortPendSVHandler   PendSV_Handler
00164
00165 /* IMPORTANT: After 10.3.1 update, SysTick_Handler comes from NVIC (if SYS timebase = systick),
00166 otherwise from cmsis_os2.c */

```

```
00167 #define USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION 0
00168
00169 /* USER CODE BEGIN Defines */
00170 /* Section where parameter definitions can be added (for instance, to override default ones in
   FreeRTOS.h) */
00171 /* USER CODE END Defines */
00172
00173 #endif /* FREERTOS_CONFIG_H */
```

8.7. Referencia del archivo Core/Inc/gpio.h

This file contains all the function prototypes for the gpio.c file.

```
#include "main.h"
```

Gráfico de dependencias incluidas en gpio.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Funciones

- void [MX_GPIO_Init](#) (void)

8.7.1. Descripción detallada

This file contains all the function prototypes for the gpio.c file.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.7.2. Documentación de funciones

8.7.2.1. MX_GPIO_Init()

```
void MX_GPIO_Init (
    void )
```

8.8. gpio.h

[Ir a la documentación de este archivo.](#)

```
00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020 /* Define to prevent recursive inclusion -----*/
00021 #ifndef __GPIO_H__
00022 #define __GPIO_H__
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00028 /* Includes -----*/
00029 #include "main.h"
00030
00031 /* USER CODE BEGIN Includes */
00032
00033 /* USER CODE END Includes */
00034
00035 /* USER CODE BEGIN Private defines */
00036
00037 /* USER CODE END Private defines */
00038
00039 void MX_GPIO_Init(void);
00040
00041 /* USER CODE BEGIN Prototypes */
00042
00043 /* USER CODE END Prototypes */
00044
00045 #ifdef __cplusplus
00046 }
00047 #endif
00048 #endif /*__ GPIO_H__ */
00049
```

8.9. Referencia del archivo Core/Inc/gps.h

Interfaz pública del driver GPS basado en recepción UART.

```
#include "stm32f4xx_hal.h"
#include <stdint.h>
#include <stdbool.h>
```

Gráfico de dependencias incluidas en gps.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Estructuras de datos

- struct [gps_data_t](#)
Última solución de navegación calculada por el módulo GPS.

defines

- #define [GPS_RX_BUFFER_SIZE](#) 512
Tamaño del buffer circular usado para desacoplar la UART del parser.
- #define [GPS_SENTENCE_MAX_LENGTH](#) 128
Longitud máxima esperada para una sentencia NMEA completa.

Enumeraciones

- enum [gps_status_t](#) { [GPS_OK](#) = 0 , [GPS_ERROR](#) , [GPS_TIMEOUT](#) }
Códigos de estado devueltos por la API GPS.
- enum [gps_fix_t](#) { [GPS_NO_FIX](#) = 0 , [GPS_FIX](#) }
Estado de fix del receptor GPS.

Funciones

- `gps_status_t GPS_Init` (UART_HandleTypeDef *huart)
Inicializa el módulo GPS y asocia la UART usada por el receptor.
- `void GPS_UARTCallback` (UART_HandleTypeDef *huart)
Callback que debe invocarse desde la interrupción de recepción UART.
- `void GPS_Process` (void)
Procesa las tramas GPS acumuladas por la recepción UART.
- `gps_data_t GPS_GetData` (void)
Devuelve una copia de la última muestra GPS procesada.
- `bool GPS_DataValid` (void)
Indica si la última muestra GPS disponible es válida.
- `void GPS_RestartReceive` (void)
Reinicia la recepción UART del GPS.

8.9.1. Descripción detallada

Interfaz pública del driver GPS basado en recepción UART.

Autor

Lorenzo

Fecha

13 may 2026

El módulo encapsula la recepción y el procesamiento de tramas NMEA para ofrecer al resto del sistema una estructura de datos estable. La separación entre callback UART, procesamiento y consulta de datos permite integrarlo en FreeRTOS sin bloquear las tareas de adquisición o comunicación.

8.9.2. Documentación de «define»

8.9.2.1. GPS_RX_BUFFER_SIZE

```
#define GPS_RX_BUFFER_SIZE 512
```

Tamaño del buffer circular usado para desacoplar la UART del parser.

Se dimensiona por encima de una única sentencia NMEA para absorber ráfagas o pequeñas latencias de planificación sin perder caracteres.

8.9.2.2. GPS_SENTENCE_MAX_LENGTH

```
#define GPS_SENTENCE_MAX_LENGTH 128
```

Longitud máxima esperada para una sentencia NMEA completa.

Limita el almacenamiento temporal de una trama individual y protege al parser frente a entradas corruptas o sin terminador.

8.9.3. Documentación de enumeraciones

8.9.3.1. `gps_fix_t`

enum `gps_fix_t`

Estado de fix del receptor GPS.

Se mantiene como enum para que la validez de posición sea explícita y no dependa de interpretar directamente campos numéricos de la trama NMEA.

Valores de enumeraciones

GPS_NO_FIX	El receptor todavía no dispone de posición válida.
GPS_FIX	El receptor dispone de posición válida.

8.9.3.2. `gps_status_t`

enum `gps_status_t`

Códigos de estado devueltos por la API GPS.

Permiten diferenciar fallos de inicialización, ausencia temporal de datos y errores generales sin obligar a las capas superiores a conocer detalles de la UART o del parser.

Valores de enumeraciones

GPS_OK	Operación completada correctamente.
GPS_ERROR	Error genérico durante una operación del módulo GPS.
GPS_TIMEOUT	La operación no recibió datos dentro del tiempo esperado.

8.9.4. Documentación de funciones

8.9.4.1. `GPS_DataValid()`

```
bool GPS_DataValid (
    void )
```

Indica si la última muestra GPS disponible es válida.

Indica si ya se ha decodificado una sentencia GPS válida.

Valores devueltos

<i>true</i>	Existe una muestra válida con fix aceptable.
<i>false</i>	La muestra actual no debe usarse como posición fiable.

Indica si la última muestra GPS disponible es válida.

Valores devueltos

<i>true</i>	Hay datos GPS procesados.
<i>false</i>	Aún no hay datos GPS válidos disponibles.

8.9.4.2. GPS_GetData()

```
gps_data_t GPS_GetData (
    void )
```

Devuelve una copia de la última muestra GPS procesada.

Devuelve una copia del último estado GPS decodificado.

Devuelve

Estructura `gps_data_t` con el último estado conocido del receptor.

Nota

La devolución por valor evita que el consumidor modifique accidentalmente el estado interno del driver.

Devuelve una copia de la última muestra GPS procesada.

Devuelve

Estructura con los últimos datos GPS disponibles.

Se devuelve por valor para evitar que otros módulos modifiquen directamente el estado interno del driver. Gráfico de llamadas a esta función:

8.9.4.3. GPS_Init()

```
gps_status_t GPS_Init (
    UART_HandleTypeDef * huart)
```

Inicializa el módulo GPS y asocia la UART usada por el receptor.

Inicializa el driver GPS y arranca la recepción UART por interrupción.

Parámetros

in	<i>huart</i>	Handler HAL de la UART conectada al módulo GPS.
----	--------------	---

Devuelve

GPS_OK si la inicialización se completa correctamente.

GPS_ERROR si no es posible preparar la recepción.

Precondición

`huart` debe estar inicializado por la capa HAL antes de llamar a esta función.

Inicializa el módulo GPS y asocia la UART usada por el receptor.

Parámetros

<code>in</code>	<code>huart</code>	UART conectada al módulo GPS.
-----------------	--------------------	-------------------------------

Valores devueltos

<code>GPS_OK</code>	La recepción inicial se configuró correctamente.
<code>GPS_ERROR</code>	No se pudo iniciar la recepción por interrupción.

La recepción se inicia byte a byte para poder reconstruir sentencias NMEA sin depender de DMA ni de longitudes fijas, ya que el GPS emite tramas ASCII de tamaño variable. Gráfico de llamadas a esta función:

8.9.4.4. GPS_Process()

```
void GPS_Process (
    void )
```

Procesa las tramas GPS acumuladas por la recepción UART.

Procesa los bytes pendientes y actualiza los datos GPS disponibles.

Debe ejecutarse desde contexto de tarea o bucle principal para convertir las sentencias recibidas en una muestra `gps_data_t` consultable por la aplicación.

Procesa las tramas GPS acumuladas por la recepción UART.

Esta función debe ejecutarse periódicamente desde contexto de tarea o bucle principal. Separar el parser de la ISR evita bloquear otras interrupciones y reduce el riesgo de perder datos en sistemas con FreeRTOS. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.9.4.5. GPS_RestartReceive()

```
void GPS_RestartReceive (
    void )
```

Reinicia la recepción UART del GPS.

Rearma manualmente la recepción UART del GPS.

Se expone para recuperar la recepción tras errores o condiciones de parada sin reinicializar toda la aplicación.

Reinicia la recepción UART del GPS.

Esta función permite recuperar la escucha si una condición externa cancela o interrumpe la recepción por interrupción. Se reutiliza el byte interno para mantener encapsulado el estado de recepción.

8.9.4.6. GPS_UARTCallback()

```
void GPS_UARTCallback (
    UART_HandleTypeDef * huart)
```

Callback que debe invocarse desde la interrupción de recepción UART.

Callback que debe llamarse al completarse la recepción UART del GPS.

Parámetros

in	<i>huart</i>	Handler HAL que ha generado el evento de recepción.
----	--------------	---

La callback mantiene la captura de caracteres separada del procesado completo de sentencias para reducir el trabajo realizado en contexto de interrupción.

Callback que debe invocarse desde la interrupción de recepción UART.

Parámetros

in	<i>huart</i>	UART que ha generado la interrupción.
----	--------------	---------------------------------------

El callback filtra la UART recibida para que el driver ignore interrupciones de otros periféricos. Tras guardar el byte, rearma inmediatamente la escucha para mantener una recepción continua. Gráfico de llamadas de esta función:

8.10. gps.h

[Ir a la documentación de este archivo.](#)

```
00001
00013
00014 #ifndef GPS_H
00015 #define GPS_H
00016
00017 #include "stm32f4xx_hal.h"
00018 #include <stdint.h>
00019 #include <stdbool.h>
00020
00029 #define GPS_RX_BUFFER_SIZE          512
00030
00039 #define GPS_SENTENCE_MAX_LENGTH     128
00040
00049 typedef enum
00050 {
00052     GPS_OK = 0,
00053
00055     GPS_ERROR,
00056
00058     GPS_TIMEOUT
00059 } gps_status_t;
00061
00069 typedef enum
00070 {
00072     GPS_NO_FIX = 0,
00073
00075     GPS_FIX
00076 } gps_fix_t;
00078
00087 typedef struct
00088 {
00090     float latitude;
00091
00093     float longitude;
00094
```

```

00096     float altitude;
00097
00099     float speed_knots;
00100
00102     float speed_kmh;
00103
00105     uint8_t satellites;
00106
00108     gps_fix_t fix;
00109
00111     bool valid;
00112
00113 } gps_data_t;
00114
00126 gps_status_t GPS_Init(UART_HandleTypeDef *huart);
00127
00137 void GPS_UARTCallback(UART_HandleTypeDef *huart);
00138
00146 void GPS_Process(void);
00147
00156 gps_data_t GPS_GetData(void);
00157
00164 bool GPS_DataValid(void);
00165
00173 void GPS_RestartReceive(void);
00174
00175 #endif

```

8.11. Referencia del archivo Core/Inc/i2c.h

This file contains all the function prototypes for the i2c.c file.

```
#include "main.h"
```

Gráfico de dependencias incluidas en i2c.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Funciones

- void [MX_I2C1_Init](#) (void)

Variables

- I2C_HandleTypeDef [hi2c1](#)

8.11.1. Descripción detallada

This file contains all the function prototypes for the i2c.c file.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.11.2. Documentación de funciones

8.11.2.1. MX_I2C1_Init()

```
void MX_I2C1_Init (
    void )
```

Gráfico de llamadas a esta función:

8.11.3. Documentación de variables

8.11.3.1. hi2c1

```
I2C_HandleTypeDef hi2c1 [extern]
```

8.12. i2c.h

[Ir a la documentación de este archivo.](#)

```
00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020 /* Define to prevent recursive inclusion -----*/
00021 #ifndef __I2C_H__
00022 #define __I2C_H__
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00028 /* Includes -----*/
00029 #include "main.h"
00030
00031 /* USER CODE BEGIN Includes */
00032
00033 /* USER CODE END Includes */
00034
00035 extern I2C_HandleTypeDef hi2c1;
00036
00037 /* USER CODE BEGIN Private defines */
00038
00039 /* USER CODE END Private defines */
00040
00041 void MX_I2C1_Init(void);
00042
00043 /* USER CODE BEGIN Prototypes */
00044
00045 /* USER CODE END Prototypes */
00046
00047 #ifdef __cplusplus
00048 }
00049 #endif
00050
00051 #endif /* __I2C_H__ */
00052
```

8.13. Referencia del archivo Core/Inc/main.h

: Header for main.c file. This file contains the common defines of the application.

```
#include "stm32f4xx_hal.h"
```

Gráfico de dependencias incluidas en main.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

defines

- #define CS_I2C_SPI_Pin GPIO_PIN_3
- #define CS_I2C_SPI_GPIO_Port GPIOE
- #define PC14_OSC32_IN_Pin GPIO_PIN_14
- #define PC14_OSC32_IN_GPIO_Port GPIOC
- #define PC15_OSC32_OUT_Pin GPIO_PIN_15
- #define PC15_OSC32_OUT_GPIO_Port GPIOC
- #define PH0_OSC_IN_Pin GPIO_PIN_0
- #define PH0_OSC_IN_GPIO_Port GPIOH
- #define PH1_OSC_OUT_Pin GPIO_PIN_1
- #define PH1_OSC_OUT_GPIO_Port GPIOH
- #define OTG_FS_PowerSwitchOn_Pin GPIO_PIN_0
- #define OTG_FS_PowerSwitchOn_GPIO_Port GPIOC
- #define PDM_OUT_Pin GPIO_PIN_3
- #define PDM_OUT_GPIO_Port GPIOC
- #define B1_Pin GPIO_PIN_0
- #define B1_GPIO_Port GPIOA
- #define I2S3_WS_Pin GPIO_PIN_4
- #define I2S3_WS_GPIO_Port GPIOA
- #define SPI1_SCK_Pin GPIO_PIN_5
- #define SPI1_SCK_GPIO_Port GPIOA
- #define SPI1_MISO_Pin GPIO_PIN_6
- #define SPI1_MISO_GPIO_Port GPIOA
- #define SPI1_MOSI_Pin GPIO_PIN_7
- #define SPI1_MOSI_GPIO_Port GPIOA
- #define BOOT1_Pin GPIO_PIN_2
- #define BOOT1_GPIO_Port GPIOB
- #define CLK_IN_Pin GPIO_PIN_10
- #define CLK_IN_GPIO_Port GPIOB
- #define LD4_Pin GPIO_PIN_12
- #define LD4_GPIO_Port GPIOD
- #define LD3_Pin GPIO_PIN_13
- #define LD3_GPIO_Port GPIOD
- #define LD5_Pin GPIO_PIN_14
- #define LD5_GPIO_Port GPIOD
- #define LD6_Pin GPIO_PIN_15
- #define LD6_GPIO_Port GPIOD
- #define I2S3_MCK_Pin GPIO_PIN_7
- #define I2S3_MCK_GPIO_Port GPIOC
- #define VBUS_FS_Pin GPIO_PIN_9
- #define VBUS_FS_GPIO_Port GPIOA
- #define OTG_FS_ID_Pin GPIO_PIN_10
- #define OTG_FS_ID_GPIO_Port GPIOA
- #define OTG_FS_DM_Pin GPIO_PIN_11
- #define OTG_FS_DM_GPIO_Port GPIOA
- #define OTG_FS_DP_Pin GPIO_PIN_12
- #define OTG_FS_DP_GPIO_Port GPIOA
- #define SWDIO_Pin GPIO_PIN_13
- #define SWDIO_GPIO_Port GPIOA
- #define SWCLK_Pin GPIO_PIN_14
- #define SWCLK_GPIO_Port GPIOA
- #define I2S3_SCK_Pin GPIO_PIN_10
- #define I2S3_SCK_GPIO_Port GPIOC
- #define I2S3_SD_Pin GPIO_PIN_12

- #define `I2S3_SD_GPIO_Port` GPIOC
- #define `Audio_RST_Pin` GPIO_PIN_4
- #define `Audio_RST_GPIO_Port` GPIOD
- #define `OTG_FS_OverCurrent_Pin` GPIO_PIN_5
- #define `OTG_FS_OverCurrent_GPIO_Port` GPIOD
- #define `SWO_Pin` GPIO_PIN_3
- #define `SWO_GPIO_Port` GPIOB
- #define `I2C1_SCL_IMU_Pin` GPIO_PIN_6
- #define `I2C1_SCL_IMU_GPIO_Port` GPIOB
- #define `I2C1_SDA_IMU_Pin` GPIO_PIN_7
- #define `I2C1_SDA_IMU_GPIO_Port` GPIOB
- #define `MEMS_INT2_Pin` GPIO_PIN_1
- #define `MEMS_INT2_GPIO_Port` GPIOE

Funciones

- void `Error_Handler` (void)

8.13.1. Descripción detallada

: Header for main.c file. This file contains the common defines of the application.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.13.2. Documentación de «define»

8.13.2.1. `Audio_RST_GPIO_Port`

```
#define Audio_RST_GPIO_Port GPIOD
```

8.13.2.2. `Audio_RST_Pin`

```
#define Audio_RST_Pin GPIO_PIN_4
```

8.13.2.3. `B1_GPIO_Port`

```
#define B1_GPIO_Port GPIOA
```

8.13.2.4. B1_Pin

```
#define B1_Pin GPIO_PIN_0
```

8.13.2.5. BOOT1_GPIO_Port

```
#define BOOT1_GPIO_Port GPIOB
```

8.13.2.6. BOOT1_Pin

```
#define BOOT1_Pin GPIO_PIN_2
```

8.13.2.7. CLK_IN_GPIO_Port

```
#define CLK_IN_GPIO_Port GPIOB
```

8.13.2.8. CLK_IN_Pin

```
#define CLK_IN_Pin GPIO_PIN_10
```

8.13.2.9. CS_I2C_SPI_GPIO_Port

```
#define CS_I2C_SPI_GPIO_Port GPIOE
```

8.13.2.10. CS_I2C_SPI_Pin

```
#define CS_I2C_SPI_Pin GPIO_PIN_3
```

8.13.2.11. I2C1_SCL_IMU_GPIO_Port

```
#define I2C1_SCL_IMU_GPIO_Port GPIOB
```

8.13.2.12. I2C1_SCL_IMU_Pin

```
#define I2C1_SCL_IMU_Pin GPIO_PIN_6
```

8.13.2.13. I2C1_SDA_IMU_GPIO_Port

```
#define I2C1_SDA_IMU_GPIO_Port GPIOB
```

8.13.2.14. I2C1_SDA_IMU_Pin

```
#define I2C1_SDA_IMU_Pin GPIO_PIN_7
```

8.13.2.15. I2S3_MCK_GPIO_Port

```
#define I2S3_MCK_GPIO_Port GPIOC
```

8.13.2.16. I2S3_MCK_Pin

```
#define I2S3_MCK_Pin GPIO_PIN_7
```

8.13.2.17. I2S3_SCK_GPIO_Port

```
#define I2S3_SCK_GPIO_Port GPIOC
```

8.13.2.18. I2S3_SCK_Pin

```
#define I2S3_SCK_Pin GPIO_PIN_10
```

8.13.2.19. I2S3_SD_GPIO_Port

```
#define I2S3_SD_GPIO_Port GPIOC
```

8.13.2.20. I2S3_SD_Pin

```
#define I2S3_SD_Pin GPIO_PIN_12
```

8.13.2.21. I2S3_WS_GPIO_Port

```
#define I2S3_WS_GPIO_Port GPIOA
```

8.13.2.22. I2S3_WS_Pin

```
#define I2S3_WS_Pin GPIO_PIN_4
```

8.13.2.23. LD3_GPIO_Port

```
#define LD3_GPIO_Port GPIOD
```

8.13.2.24. LD3_Pin

```
#define LD3_Pin GPIO_PIN_13
```

8.13.2.25. LD4_GPIO_Port

```
#define LD4_GPIO_Port GPIOD
```

8.13.2.26. LD4_Pin

```
#define LD4_Pin GPIO_PIN_12
```

8.13.2.27. LD5_GPIO_Port

```
#define LD5_GPIO_Port GPIOD
```

8.13.2.28. LD5_Pin

```
#define LD5_Pin GPIO_PIN_14
```

8.13.2.29. LD6_GPIO_Port

```
#define LD6_GPIO_Port GPIOD
```

8.13.2.30. LD6_Pin

```
#define LD6_Pin GPIO_PIN_15
```

8.13.2.31. MEMS_INT2_GPIO_Port

```
#define MEMS_INT2_GPIO_Port GPIOE
```

8.13.2.32. MEMS_INT2_Pin

```
#define MEMS_INT2_Pin GPIO_PIN_1
```

8.13.2.33. OTG_FS_DM_GPIO_Port

```
#define OTG_FS_DM_GPIO_Port GPIOA
```

8.13.2.34. OTG_FS_DM_Pin

```
#define OTG_FS_DM_Pin GPIO_PIN_11
```

8.13.2.35. OTG_FS_DP_GPIO_Port

```
#define OTG_FS_DP_GPIO_Port GPIOA
```

8.13.2.36. OTG_FS_DP_Pin

```
#define OTG_FS_DP_Pin GPIO_PIN_12
```

8.13.2.37. OTG_FS_ID_GPIO_Port

```
#define OTG_FS_ID_GPIO_Port GPIOA
```

8.13.2.38. OTG_FS_ID_Pin

```
#define OTG_FS_ID_Pin GPIO_PIN_10
```

8.13.2.39. OTG_FS_OverCurrent_GPIO_Port

```
#define OTG_FS_OverCurrent_GPIO_Port GPIOD
```

8.13.2.40. OTG_FS_OverCurrent_Pin

```
#define OTG_FS_OverCurrent_Pin GPIO_PIN_5
```

8.13.2.41. OTG_FS_PowerSwitchOn_GPIO_Port

```
#define OTG_FS_PowerSwitchOn_GPIO_Port GPIOC
```

8.13.2.42. OTG_FS_PowerSwitchOn_Pin

```
#define OTG_FS_PowerSwitchOn_Pin GPIO_PIN_0
```

8.13.2.43. PC14_OSC32_IN_GPIO_Port

```
#define PC14_OSC32_IN_GPIO_Port GPIOC
```

8.13.2.44. PC14_OSC32_IN_Pin

```
#define PC14_OSC32_IN_Pin GPIO_PIN_14
```

8.13.2.45. PC15_OSC32_OUT_GPIO_Port

```
#define PC15_OSC32_OUT_GPIO_Port GPIOC
```

8.13.2.46. PC15_OSC32_OUT_Pin

```
#define PC15_OSC32_OUT_Pin GPIO_PIN_15
```

8.13.2.47. PDM_OUT_GPIO_Port

```
#define PDM_OUT_GPIO_Port GPIOC
```

8.13.2.48. PDM_OUT_Pin

```
#define PDM_OUT_Pin GPIO_PIN_3
```

8.13.2.49. PH0_OSC_IN_GPIO_Port

```
#define PH0_OSC_IN_GPIO_Port GPIOH
```

8.13.2.50. PH0_OSC_IN_Pin

```
#define PH0_OSC_IN_Pin GPIO_PIN_0
```

8.13.2.51. PH1_OSC_OUT_GPIO_Port

```
#define PH1_OSC_OUT_GPIO_Port GPIOH
```

8.13.2.52. PH1_OSC_OUT_Pin

```
#define PH1_OSC_OUT_Pin GPIO_PIN_1
```

8.13.2.53. SPI1_MISO_GPIO_Port

```
#define SPI1_MISO_GPIO_Port GPIOA
```

8.13.2.54. SPI1_MISO_Pin

```
#define SPI1_MISO_Pin GPIO_PIN_6
```

8.13.2.55. SPI1_MOSI_GPIO_Port

```
#define SPI1_MOSI_GPIO_Port GPIOA
```

8.13.2.56. SPI1_MOSI_Pin

```
#define SPI1_MOSI_Pin GPIO_PIN_7
```

8.13.2.57. SPI1_SCK_GPIO_Port

```
#define SPI1_SCK_GPIO_Port GPIOA
```

8.13.2.58. SPI1_SCK_Pin

```
#define SPI1_SCK_Pin GPIO_PIN_5
```

8.13.2.59. SWCLK_GPIO_Port

```
#define SWCLK_GPIO_Port GPIOA
```

8.13.2.60. SWCLK_Pin

```
#define SWCLK_Pin GPIO_PIN_14
```

8.13.2.61. SWDIO_GPIO_Port

```
#define SWDIO_GPIO_Port GPIOA
```

8.13.2.62. SWDIO_Pin

```
#define SWDIO_Pin GPIO_PIN_13
```

8.13.2.63. SWO_GPIO_Port

```
#define SWO_GPIO_Port GPIOB
```

8.13.2.64. SWO_Pin

```
#define SWO_Pin GPIO_PIN_3
```

8.13.2.65. VBUS_FS_GPIO_Port

```
#define VBUS_FS_GPIO_Port GPIOA
```

8.13.2.66. VBUS_FS_Pin

```
#define VBUS_FS_Pin GPIO_PIN_9
```

8.13.3. Documentación de funciones

8.13.3.1. Error_Handler()

```
void Error_Handler (
    void )
```

8.14. main.h

[Ir a la documentación de este archivo.](#)

```
00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020
00021 /* Define to prevent recursive inclusion -----*/
00022 #ifndef __MAIN_H
00023 #define __MAIN_H
00024
00025 #ifdef __cplusplus
00026 extern "C" {
00027 #endif
00028
00029 /* Includes -----*/
00030 #include "stm32f4xx_hal.h"
00031
00032 /* Private includes -----*/
00033 /* USER CODE BEGIN Includes */
00034
00035 /* USER CODE END Includes */
00036
00037 /* Exported types -----*/
00038 /* USER CODE BEGIN ET */
00039
00040 /* USER CODE END ET */
00041
00042 /* Exported constants -----*/
00043 /* USER CODE BEGIN EC */
00044
00045 /* USER CODE END EC */
00046
00047 /* Exported macro -----*/
00048 /* USER CODE BEGIN EM */
00049
00050 /* USER CODE END EM */
00051
00052 /* Exported functions prototypes -----*/
00053 void Error_Handler(void);
00054
00055 /* USER CODE BEGIN EFP */
00056
00057 /* USER CODE END EFP */
```

```

00058
00059 /* Private defines -----*/
00060 #define CS_I2C_SPI_Pin GPIO_PIN_3
00061 #define CS_I2C_SPI_GPIO_Port GPIOE
00062 #define PC14_OSC32_IN_Pin GPIO_PIN_14
00063 #define PC14_OSC32_IN_GPIO_Port GPIOC
00064 #define PC15_OSC32_OUT_Pin GPIO_PIN_15
00065 #define PC15_OSC32_OUT_GPIO_Port GPIOC
00066 #define PH0_OSC_IN_Pin GPIO_PIN_0
00067 #define PH0_OSC_IN_GPIO_Port GPIOH
00068 #define PH1_OSC_OUT_Pin GPIO_PIN_1
00069 #define PH1_OSC_OUT_GPIO_Port GPIOH
00070 #define OTG_FS_PowerSwitchOn_Pin GPIO_PIN_0
00071 #define OTG_FS_PowerSwitchOn_GPIO_Port GPIOC
00072 #define PDM_OUT_Pin GPIO_PIN_3
00073 #define PDM_OUT_GPIO_Port GPIOC
00074 #define B1_Pin GPIO_PIN_0
00075 #define B1_GPIO_Port GPIOA
00076 #define I2S3_WS_Pin GPIO_PIN_4
00077 #define I2S3_WS_GPIO_Port GPIOA
00078 #define SPI1_SCK_Pin GPIO_PIN_5
00079 #define SPI1_SCK_GPIO_Port GPIOA
00080 #define SPI1_MISO_Pin GPIO_PIN_6
00081 #define SPI1_MISO_GPIO_Port GPIOA
00082 #define SPI1_MOSI_Pin GPIO_PIN_7
00083 #define SPI1_MOSI_GPIO_Port GPIOA
00084 #define BOOT1_Pin GPIO_PIN_2
00085 #define BOOT1_GPIO_Port GPIOB
00086 #define CLK_IN_Pin GPIO_PIN_10
00087 #define CLK_IN_GPIO_Port GPIOB
00088 #define LD4_Pin GPIO_PIN_12
00089 #define LD4_GPIO_Port GPIOD
00090 #define LD3_Pin GPIO_PIN_13
00091 #define LD3_GPIO_Port GPIOD
00092 #define LD5_Pin GPIO_PIN_14
00093 #define LD5_GPIO_Port GPIOD
00094 #define LD6_Pin GPIO_PIN_15
00095 #define LD6_GPIO_Port GPIOD
00096 #define I2S3_MCK_Pin GPIO_PIN_7
00097 #define I2S3_MCK_GPIO_Port GPIOC
00098 #define VBUS_FS_Pin GPIO_PIN_9
00099 #define VBUS_FS_GPIO_Port GPIOA
00100 #define OTG_FS_ID_Pin GPIO_PIN_10
00101 #define OTG_FS_ID_GPIO_Port GPIOA
00102 #define OTG_FS_DM_Pin GPIO_PIN_11
00103 #define OTG_FS_DM_GPIO_Port GPIOA
00104 #define OTG_FS_DP_Pin GPIO_PIN_12
00105 #define OTG_FS_DP_GPIO_Port GPIOA
00106 #define SWDIO_Pin GPIO_PIN_13
00107 #define SWDIO_GPIO_Port GPIOA
00108 #define SWCLK_Pin GPIO_PIN_14
00109 #define SWCLK_GPIO_Port GPIOA
00110 #define I2S3_SCK_Pin GPIO_PIN_10
00111 #define I2S3_SCK_GPIO_Port GPIOC
00112 #define I2S3_SD_Pin GPIO_PIN_12
00113 #define I2S3_SD_GPIO_Port GPIOC
00114 #define Audio_RST_Pin GPIO_PIN_4
00115 #define Audio_RST_GPIO_Port GPIOD
00116 #define OTG_FS_OverCurrent_Pin GPIO_PIN_5
00117 #define OTG_FS_OverCurrent_GPIO_Port GPIOD
00118 #define SWO_Pin GPIO_PIN_3
00119 #define SWO_GPIO_Port GPIOB
00120 #define I2C1_SCL_IMU_Pin GPIO_PIN_6
00121 #define I2C1_SCL_IMU_GPIO_Port GPIOB
00122 #define I2C1_SDA_IMU_Pin GPIO_PIN_7
00123 #define I2C1_SDA_IMU_GPIO_Port GPIOB
00124 #define MEMS_INT2_Pin GPIO_PIN_1
00125 #define MEMS_INT2_GPIO_Port GPIOE
00126
00127 /* USER CODE BEGIN Private defines */
00128
00129 /* USER CODE END Private defines */
00130
00131 #ifndef __cplusplus
00132 }
00133 #endif
00134
00135 #endif /* __MAIN_H */

```

8.15. Referencia del archivo Core/Inc/sensor10DoF.h

Interfaz pública del módulo de adquisición del sensor 10DoF.

```
#include "main.h"
#include <stdint.h>
```

Gráfico de dependencias incluidas en sensor10DoF.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Estructuras de datos

- struct `Sensor10DoF_t`
Conjunto completo de medidas calculadas por el módulo 10DoF.

Enumeraciones

- enum `SensorStatus_t` { `SENSOR_OK` = 0 , `SENSOR_ERROR` , `SENSOR_TIMEOUT` , `SENSOR_BUSY` }
Estado genérico de las operaciones del módulo 10DoF.

Funciones

- `SensorStatus_t` `Sensor10DoF_Init` (void)
Inicializa los sensores y prepara el bus de comunicación.
- `SensorStatus_t` `Sensor10DoF_Read` (`Sensor10DoF_t` *data)
Lee una muestra completa del módulo 10DoF.
- void `Sensor10DoF_Reset` (void)
Reinicia el estado interno del módulo 10DoF.
- void `Sensor10DoF_I2CRecover` (void)
Ejecuta una recuperación manual del bus I2C usado por el módulo.
- void `Sensor10DoF_Calibrate` (void)
Ejecuta la calibración del módulo 10DoF.

8.15.1. Descripción detallada

Interfaz pública del módulo de adquisición del sensor 10DoF.

Este driver concentra la inicialización, recuperación y lectura de las magnitudes inerciales, magnéticas y ambientales usadas por el sistema. La API devuelve valores ya convertidos a unidades de aplicación para que las tareas de FreeRTOS y la capa de telemetría no dependan de registros ni escalas internas de cada sensor.

8.15.2. Documentación de enumeraciones

8.15.2.1. `SensorStatus_t`

```
enum SensorStatus_t
```

Estado genérico de las operaciones del módulo 10DoF.

La distinción entre error, timeout y bus ocupado permite aplicar estrategias de recuperación distintas sin acoplar la aplicación al detalle del bus I2C.

Valores de enumeraciones

SENSOR_OK	Operación completada correctamente.
SENSOR_ERROR	Error genérico durante la operación del sensor.
SENSOR_TIMEOUT	No se recibió respuesta dentro del tiempo esperado.
SENSOR_BUSY	El recurso compartido o bus todavía no está disponible.

8.15.3. Documentación de funciones**8.15.3.1. Sensor10DoF_Calibrate()**

```
void Sensor10DoF_Calibrate (
    void )
```

Ejecuta la calibración del módulo 10DoF.

Ejecuta una nueva calibración del giróscopo.

La calibración reduce offsets sistemáticos, especialmente en el giróscopo, y permite que las magnitudes publicadas sean estables cuando el sistema está en reposo.

Ejecuta la calibración del módulo 10DoF.

Permite recalcular los offsets si cambia la temperatura, la alimentación o se observa deriva tras el arranque. El sensor debe permanecer inmóvil durante la llamada para que el promedio represente únicamente el sesgo estático. Gráfico de llamadas de esta función:

8.15.3.2. Sensor10DoF_I2CRecover()

```
void Sensor10DoF_I2CRecover (
    void )
```

Ejecuta una recuperación manual del bus I2C usado por el módulo.

Intenta recuperar el bus I2C generando pulsos manuales en SCL.

Esta operación está pensada para liberar el bus cuando un dispositivo mantiene líneas bloqueadas tras una transacción incompleta o un reinicio inesperado.

Ejecuta una recuperación manual del bus I2C usado por el módulo.

Algunos esclavos I2C pueden dejar SDA retenida tras un reset o una transferencia interrumpida. La recuperación conmuta temporalmente SCL/SDA como GPIO para liberar el bus antes de reinicializar el periférico I2C. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.15.3.3. Sensor10DoF_Init()

```
SensorStatus_t Sensor10DoF_Init (
    void )
```

Inicializa los sensores y prepara el bus de comunicación.

Inicializa todos los sensores del módulo 10DoF.

Devuelve

SENSOR_OK si todos los dispositivos quedan listos para lectura.

SENSOR_ERROR si algún dispositivo no responde o no queda configurado.

SENSOR_TIMEOUT si el bus no responde dentro del tiempo esperado.

Debe llamarse antes de solicitar lecturas para asegurar que las escalas, registros y compensaciones iniciales están configuradas de forma coherente.

Inicializa los sensores y prepara el bus de comunicación.

Valores devueltos

<i>SENSOR_OK</i>	Todos los dispositivos respondieron y quedaron configurados.
<i>SENSOR_ERROR</i>	Algún dispositivo no respondió, no pudo configurarse o falló la calibración.

La función verifica primero la presencia de todos los esclavos I2C antes de configurar registros. Esto permite detectar fallos de cableado o bloqueo de bus antes de producir lecturas parciales incoherentes. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.15.3.4. Sensor10DoF_Read()

```
SensorStatus_t Sensor10DoF_Read (
    Sensor10DoF_t * d)
```

Lee una muestra completa del módulo 10DoF.

Lee todos los sensores y rellena una muestra 10DoF completa.

Parámetros

out	data	Estructura donde se almacenan las magnitudes convertidas.
-----	------	---

Devuelve

SENSOR_OK si la lectura se completa correctamente.

SENSOR_ERROR si alguna magnitud no puede actualizarse.

SENSOR_TIMEOUT si un dispositivo no responde a tiempo.

SENSOR_BUSY si el módulo no puede atender la lectura en ese instante.

Precondición

`data` debe apuntar a una estructura válida.

La función entrega una muestra agregada para reducir el acoplamiento entre las tareas consumidoras y los distintos sensores físicos que forman el módulo.

Lee una muestra completa del módulo 10DoF.

Parámetros

out	<i>d</i>	Estructura de destino con aceleración, orientación, campo magnético, velocidad angular, temperatura, presión y altitud.
-----	----------	---

Valores devueltos

<i>SENSOR_OK</i>	Muestra actualizada correctamente.
<i>SENSOR_ERROR</i>	Error de argumento, comunicación o compensación inválida.

La función entrega magnitudes ya convertidas para aislar al resto del proyecto de escalas de registros. También calcula roll, pitch y heading compensado por inclinación para que la interfaz pueda consumir directamente valores físicos.

Precondición

[Sensor10DoF_Init\(\)](#) debe haberse ejecutado correctamente.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.15.3.5. Sensor10DoF_Reset()

```
void Sensor10DoF_Reset (
    void )
```

Reinicia el estado interno del módulo 10DoF.

Recupera el bus I2C y reinicializa el módulo 10DoF.

Se utiliza como mecanismo de recuperación cuando la secuencia de lectura queda en un estado no fiable y conviene volver a una condición conocida.

Reinicia el estado interno del módulo 10DoF.

Se proporciona como rutina de recuperación ante errores de comunicación. La función intenta dejar el bus y los sensores en un estado operativo sin exponer a las capas superiores los detalles eléctricos de recuperación. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.16. sensor10DoF.h

[Ir a la documentación de este archivo.](#)

```

00001
00012
00013 #ifndef SENSOR10DOF_H
00014 #define SENSOR10DOF_H
00015
00016 #include "main.h"
00017 #include <stdint.h>
00018
00026 typedef enum
00027 {
00029     SENSOR_OK = 0,
00030
00032     SENSOR_ERROR,
00033
00035     SENSOR_TIMEOUT,
00036
00038     SENSOR_BUSY
00039 } SensorStatus_t;
00040
00050 typedef struct
00051 {
00053     uint32_t timestamp_ms;
00054
00056     float accel_x_ms2;
00057
00059     float accel_y_ms2;
00060
00062     float accel_z_ms2;
00063
00065     float gyro_x_dps;
00066
00068     float gyro_y_dps;
00069
00071     float gyro_z_dps;
00072
00074     float mag_x_uT;
00075
00077     float mag_y_uT;
00078
00080     float mag_z_uT;
00081
00083     float heading_deg;
00084
00086     float roll_deg;
00087
00089     float pitch_deg;
00090
00092     float temperature_c;
00093
00095     float pressure_pa;
00096
00098     float pressure_hpa;
00099
00101     float pressure_atm;
00102
00104     float altitude_m;
00105
00106 } Sensor10DoF_t;
00107
00119 SensorStatus_t Sensor10DoF_Init(void);
00120
00137 SensorStatus_t Sensor10DoF_Read(Sensor10DoF_t *data);
00138
00146 void Sensor10DoF_Reset(void);
00147
00155 void Sensor10DoF_I2CRecover(void);
00156
00165 void Sensor10DoF_Calibrate(void);
00166 #endif

```

8.17. Referencia del archivo Core/Inc/stm32f4xx_hal_conf.h

```

#include "stm32f4xx_hal_rcc.h"
#include "stm32f4xx_hal_gpio.h"
#include "stm32f4xx_hal_exti.h"

```

```
#include "stm32f4xx_hal_dma.h"
#include "stm32f4xx_hal_cortex.h"
#include "stm32f4xx_hal_flash.h"
#include "stm32f4xx_hal_i2c.h"
#include "stm32f4xx_hal_pwr.h"
#include "stm32f4xx_hal_tim.h"
#include "stm32f4xx_hal_uart.h"
```

Gráfico de dependencias incluidas en stm32f4xx_hal_conf.h:

defines

- #define HAL_MODULE_ENABLED
This is the list of modules to be used in the HAL driver.
- #define HAL_I2C_MODULE_ENABLED
- #define HAL_TIM_MODULE_ENABLED
- #define HAL_UART_MODULE_ENABLED
- #define HAL_GPIO_MODULE_ENABLED
- #define HAL_EXTI_MODULE_ENABLED
- #define HAL_DMA_MODULE_ENABLED
- #define HAL_RCC_MODULE_ENABLED
- #define HAL_FLASH_MODULE_ENABLED
- #define HAL_PWR_MODULE_ENABLED
- #define HAL_CORTEX_MODULE_ENABLED
- #define HSE_VALUE 8000000U
Adjust the value of External High Speed oscillator (HSE) used in your application. This value is used by the RCC HAL module to compute the system frequency (when HSE is used as system clock source, directly or through the PLL).
- #define HSE_STARTUP_TIMEOUT 100U
- #define HSI_VALUE ((uint32_t)16000000U)
Internal High Speed oscillator (HSI) value. This value is used by the RCC HAL module to compute the system frequency (when HSI is used as system clock source, directly or through the PLL).
- #define LSI_VALUE 32000U
Internal Low Speed oscillator (LSI) value.
- #define LSE_VALUE 32768U
External Low Speed oscillator (LSE) value.
- #define LSE_STARTUP_TIMEOUT 5000U
- #define EXTERNAL_CLOCK_VALUE 12288000U
External clock source for I2S peripheral This value is used by the I2S HAL module to compute the I2S clock source frequency, this source is inserted directly through I2S_CKIN pad.
- #define VDD_VALUE 3300U
This is the HAL system configuration section.
- #define TICK_INT_PRIORITY 15U
- #define USE_RTOS 0U
- #define PREFETCH_ENABLE 1U
- #define INSTRUCTION_CACHE_ENABLE 1U
- #define DATA_CACHE_ENABLE 1U
- #define USE_HAL_ADC_REGISTER_CALLBACKS 0U /* ADC register callback disabled */
- #define USE_HAL_CAN_REGISTER_CALLBACKS 0U /* CAN register callback disabled */
- #define USE_HAL_CEC_REGISTER_CALLBACKS 0U /* CEC register callback disabled */
- #define USE_HAL_CRYPT_REGISTER_CALLBACKS 0U /* CRYPT register callback disabled */
- #define USE_HAL_DAC_REGISTER_CALLBACKS 0U /* DAC register callback disabled */
- #define USE_HAL_DCMI_REGISTER_CALLBACKS 0U /* DCMI register callback disabled */
- #define USE_HAL_DFSDM_REGISTER_CALLBACKS 0U /* DFSDM register callback disabled */
- #define USE_HAL_DMA2D_REGISTER_CALLBACKS 0U /* DMA2D register callback disabled */

```

■ #define USE_HAL_DSI_REGISTER_CALLBACKS 0U /* DSI register callback disabled */
■ #define USE_HAL_ETH_REGISTER_CALLBACKS 0U /* ETH register callback disabled */
■ #define USE_HAL_HASH_REGISTER_CALLBACKS 0U /* HASH register callback disabled */
■ #define USE_HAL_HCD_REGISTER_CALLBACKS 0U /* HCD register callback disabled */
■ #define USE_HAL_I2C_REGISTER_CALLBACKS 0U /* I2C register callback disabled */
■ #define USE_HAL_FMPI2C_REGISTER_CALLBACKS 0U /* FMPI2C register callback disabled */
■ #define USE_HAL_FMPSMBUS_REGISTER_CALLBACKS 0U /* FMPSMBUS register callback disabled
*/
■ #define USE_HAL_I2S_REGISTER_CALLBACKS 0U /* I2S register callback disabled */
■ #define USE_HAL_IRDA_REGISTER_CALLBACKS 0U /* IRDA register callback disabled */
■ #define USE_HAL_LPTIM_REGISTER_CALLBACKS 0U /* LPTIM register callback disabled */
■ #define USE_HAL_LTDC_REGISTER_CALLBACKS 0U /* LTDC register callback disabled */
■ #define USE_HAL_MMC_REGISTER_CALLBACKS 0U /* MMC register callback disabled */
■ #define USE_HAL_NAND_REGISTER_CALLBACKS 0U /* NAND register callback disabled */
■ #define USE_HAL_NOR_REGISTER_CALLBACKS 0U /* NOR register callback disabled */
■ #define USE_HAL_PCCARD_REGISTER_CALLBACKS 0U /* PCCARD register callback disabled */
■ #define USE_HAL_PCD_REGISTER_CALLBACKS 0U /* PCD register callback disabled */
■ #define USE_HAL_QSPI_REGISTER_CALLBACKS 0U /* QSPI register callback disabled */
■ #define USE_HAL_RNG_REGISTER_CALLBACKS 0U /* RNG register callback disabled */
■ #define USE_HAL_RTC_REGISTER_CALLBACKS 0U /* RTC register callback disabled */
■ #define USE_HAL_SAI_REGISTER_CALLBACKS 0U /* SAI register callback disabled */
■ #define USE_HAL_SD_REGISTER_CALLBACKS 0U /* SD register callback disabled */
■ #define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U /* SMARTCARD register callback disabled
*/
■ #define USE_HAL_SDRAM_REGISTER_CALLBACKS 0U /* SDRAM register callback disabled */
■ #define USE_HAL_SRAM_REGISTER_CALLBACKS 0U /* SRAM register callback disabled */
■ #define USE_HAL_SPDIFRX_REGISTER_CALLBACKS 0U /* SPDIFRX register callback disabled */
■ #define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U /* SMBUS register callback disabled */
■ #define USE_HAL_SPI_REGISTER_CALLBACKS 0U /* SPI register callback disabled */
■ #define USE_HAL_TIM_REGISTER_CALLBACKS 0U /* TIM register callback disabled */
■ #define USE_HAL_UART_REGISTER_CALLBACKS 0U /* UART register callback disabled */
■ #define USE_HAL_USART_REGISTER_CALLBACKS 0U /* USART register callback disabled */
■ #define USE_HAL_WWDG_REGISTER_CALLBACKS 0U /* WWDG register callback disabled */
■ #define MAC_ADDR0 2U

    Uncomment the line below to expanse the "assert_param" macro in the HAL drivers code.
■ #define MAC_ADDR1 0U
■ #define MAC_ADDR2 0U
■ #define MAC_ADDR3 0U
■ #define MAC_ADDR4 0U
■ #define MAC_ADDR5 0U
■ #define ETH_RX_BUF_SIZE ETH_MAX_PACKET_SIZE /* buffer size for receive */
■ #define ETH_TX_BUF_SIZE ETH_MAX_PACKET_SIZE /* buffer size for transmit */
■ #define ETH_RXBUFNB 4U /* 4 Rx buffers of size ETH_RX_BUF_SIZE */
■ #define ETH_TXBUFNB 4U /* 4 Tx buffers of size ETH_TX_BUF_SIZE */
■ #define DP83848_PHY_ADDRESS
■ #define PHY_RESET_DELAY 0x000000FFU
■ #define PHY_CONFIG_DELAY 0x000000FFU
■ #define PHY_READ_TO 0x0000FFFFU
■ #define PHY_WRITE_TO 0x0000FFFFU
■ #define PHY_BCR ((uint16_t)0x0000U)
■ #define PHY_BSR ((uint16_t)0x0001U)
■ #define PHY_RESET ((uint16_t)0x8000U)
■ #define PHY_LOOPBACK ((uint16_t)0x4000U)
■ #define PHY_FULLDUPLEX_100M ((uint16_t)0x2100U)

```

- #define PHY_HALFDUPLEX_100M ((uint16_t)0x2000U)
- #define PHY_FULLLDUPLEX_10M ((uint16_t)0x0100U)
- #define PHY_HALFDUPLEX_10M ((uint16_t)0x0000U)
- #define PHY_AUTONEGOTIATION ((uint16_t)0x1000U)
- #define PHY_RESTART_AUTONEGOTIATION ((uint16_t)0x0200U)
- #define PHY_POWERDOWN ((uint16_t)0x0800U)
- #define PHY_ISOLATE ((uint16_t)0x0400U)
- #define PHY_AUTONEGO_COMPLETE ((uint16_t)0x0020U)
- #define PHY_LINKED_STATUS ((uint16_t)0x0004U)
- #define PHY_JABBER_DETECTION ((uint16_t)0x0002U)
- #define PHY_SR ((uint16_t))
- #define PHY_SPEED_STATUS ((uint16_t))
- #define PHY_DUPLEX_STATUS ((uint16_t))
- #define USE_SPI_CRC 0U
- #define assert_param(expr)

Include module's header file.

8.17.1. Documentación de «define»

8.17.1.1. assert_param

```
#define assert_param(  
    expr)
```

Valor:

((void)0U)

Include module's header file.

8.17.1.2. DATA_CACHE_ENABLE

```
#define DATA_CACHE_ENABLE 1U
```

8.17.1.3. DP83848_PHY_ADDRESS

```
#define DP83848_PHY_ADDRESS
```

8.17.1.4. ETH_RX_BUF_SIZE

```
#define ETH_RX_BUF_SIZE ETH_MAX_PACKET_SIZE /* buffer size for receive */
```

8.17.1.5. ETH_RXBUFNB

```
#define ETH_RXBUFNB 4U /* 4 Rx buffers of size ETH_RX_BUF_SIZE */
```

8.17.1.6. ETH_TX_BUF_SIZE

```
#define ETH_TX_BUF_SIZE ETH_MAX_PACKET_SIZE /* buffer size for transmit */
```

8.17.1.7. ETH_TXBUFNB

```
#define ETH_TXBUFNB 4U /* 4 Tx buffers of size ETH_TX_BUF_SIZE */
```

8.17.1.8. EXTERNAL_CLOCK_VALUE

```
#define EXTERNAL_CLOCK_VALUE 12288000U
```

External clock source for I2S peripheral This value is used by the I2S HAL module to compute the I2S clock source frequency, this source is inserted directly through I2S_CKIN pad.

Value of the External audio frequency in Hz

8.17.1.9. HAL_CORTEX_MODULE_ENABLED

```
#define HAL_CORTEX_MODULE_ENABLED
```

8.17.1.10. HAL_DMA_MODULE_ENABLED

```
#define HAL_DMA_MODULE_ENABLED
```

8.17.1.11. HAL_EXTI_MODULE_ENABLED

```
#define HAL_EXTI_MODULE_ENABLED
```

8.17.1.12. HAL_FLASH_MODULE_ENABLED

```
#define HAL_FLASH_MODULE_ENABLED
```

8.17.1.13. HAL_GPIO_MODULE_ENABLED

```
#define HAL_GPIO_MODULE_ENABLED
```

8.17.1.14. HAL_I2C_MODULE_ENABLED

```
#define HAL_I2C_MODULE_ENABLED
```

8.17.1.15. HAL_MODULE_ENABLED

```
#define HAL_MODULE_ENABLED
```

This is the list of modules to be used in the HAL driver.

8.17.1.16. HAL_PWR_MODULE_ENABLED

```
#define HAL_PWR_MODULE_ENABLED
```

8.17.1.17. HAL_RCC_MODULE_ENABLED

```
#define HAL_RCC_MODULE_ENABLED
```

8.17.1.18. HAL_TIM_MODULE_ENABLED

```
#define HAL_TIM_MODULE_ENABLED
```

8.17.1.19. HAL_UART_MODULE_ENABLED

```
#define HAL_UART_MODULE_ENABLED
```

8.17.1.20. HSE_STARTUP_TIMEOUT

```
#define HSE_STARTUP_TIMEOUT 100U
```

Time out for HSE start up, in ms

8.17.1.21. HSE_VALUE

```
#define HSE_VALUE 8000000U
```

Adjust the value of External High Speed oscillator (HSE) used in your application. This value is used by the RCC HAL module to compute the system frequency (when HSE is used as system clock source, directly or through the PLL).

Value of the External oscillator in Hz

8.17.1.22. HSI_VALUE

```
#define HSI_VALUE ((uint32_t)16000000U)
```

Internal High Speed oscillator (HSI) value. This value is used by the RCC HAL module to compute the system frequency (when HSI is used as system clock source, directly or through the PLL).

Value of the Internal oscillator in Hz

8.17.1.23. INSTRUCTION_CACHE_ENABLE

```
#define INSTRUCTION_CACHE_ENABLE 1U
```

8.17.1.24. LSE_STARTUP_TIMEOUT

```
#define LSE_STARTUP_TIMEOUT 5000U
```

Time out for LSE start up, in ms

8.17.1.25. LSE_VALUE

```
#define LSE_VALUE 32768U
```

External Low Speed oscillator (LSE) value.

< Value of the Internal Low Speed oscillator in Hz The real value may vary depending on the variations in voltage and temperature. Value of the External Low Speed oscillator in Hz

8.17.1.26. LSI_VALUE

```
#define LSI_VALUE 32000U
```

Internal Low Speed oscillator (LSI) value.

LSI Typical Value in Hz

8.17.1.27. MAC_ADDR0

```
#define MAC_ADDR0 2U
```

Uncomment the line below to expanse the "assert_param" macro in the HAL drivers code.

8.17.1.28. MAC_ADDR1

```
#define MAC_ADDR1 0U
```

8.17.1.29. MAC_ADDR2

```
#define MAC_ADDR2 0U
```

8.17.1.30. MAC_ADDR3

```
#define MAC_ADDR3 0U
```

8.17.1.31. MAC_ADDR4

```
#define MAC_ADDR4 0U
```

8.17.1.32. MAC_ADDR5

```
#define MAC_ADDR5 0U
```

8.17.1.33. PHY_AUTONEGO_COMPLETE

```
#define PHY_AUTONEGO_COMPLETE ((uint16_t)0x0020U)
```

Auto-Negotiation process completed

8.17.1.34. PHY_AUTONEGOTIATION

```
#define PHY_AUTONEGOTIATION ((uint16_t)0x1000U)
```

Enable auto-negotiation function

8.17.1.35. PHY_BCR

```
#define PHY_BCR ((uint16_t)0x0000U)
```

Transceiver Basic Control Register

8.17.1.36. PHY_BSR

```
#define PHY_BSR ((uint16_t)0x0001U)
```

Transceiver Basic Status Register

8.17.1.37. PHY_CONFIG_DELAY

```
#define PHY_CONFIG_DELAY 0x0000FFU
```

8.17.1.38. PHY_DUPLEX_STATUS

```
#define PHY_DUPLEX_STATUS ((uint16_t))
```

PHY Duplex mask

8.17.1.39. PHY_FULLDUPLEX_100M

```
#define PHY_FULLDUPLEX_100M ((uint16_t)0x2100U)
```

Set the full-duplex mode at 100 Mb/s

8.17.1.40. PHY_FULLDUPLEX_10M

```
#define PHY_FULLDUPLEX_10M ((uint16_t)0x0100U)
```

Set the full-duplex mode at 10 Mb/s

8.17.1.41. PHY_HALFDUPLEX_100M

```
#define PHY_HALFDUPLEX_100M ((uint16_t)0x2000U)
```

Set the half-duplex mode at 100 Mb/s

8.17.1.42. PHY_HALFDUPLEX_10M

```
#define PHY_HALFDUPLEX_10M ((uint16_t)0x0000U)
```

Set the half-duplex mode at 10 Mb/s

8.17.1.43. PHY_ISOLATE

```
#define PHY_ISOLATE ((uint16_t)0x0400U)
```

Isolate PHY from MII

8.17.1.44. PHY_JABBER_DETECTION

```
#define PHY_JABBER_DETECTION ((uint16_t)0x0002U)
```

Jabber condition detected

8.17.1.45. PHY_LINKED_STATUS

```
#define PHY_LINKED_STATUS ((uint16_t)0x0004U)
```

Valid link established

8.17.1.46. PHY_LOOPBACK

```
#define PHY_LOOPBACK ((uint16_t)0x4000U)
```

Select loop-back mode

8.17.1.47. PHY_POWERDOWN

```
#define PHY_POWERDOWN ((uint16_t)0x0800U)
```

Select the power down mode

8.17.1.48. PHY_READ_TO

```
#define PHY_READ_TO 0x0000FFFFU
```

8.17.1.49. PHY_RESET

```
#define PHY_RESET ((uint16_t)0x8000U)
```

PHY Reset

8.17.1.50. PHY_RESET_DELAY

```
#define PHY_RESET_DELAY 0x000000FFU
```

8.17.1.51. PHY_RESTART_AUTONEGOTIATION

```
#define PHY_RESTART_AUTONEGOTIATION ((uint16_t)0x0200U)
```

Restart auto-negotiation function

8.17.1.52. PHY_SPEED_STATUS

```
#define PHY_SPEED_STATUS ((uint16_t))
```

PHY Speed mask

8.17.1.53. PHY_SR

```
#define PHY_SR ((uint16_t))
```

PHY status register Offset

8.17.1.54. PHY_WRITE_TO

```
#define PHY_WRITE_TO 0x0000FFFFU
```

8.17.1.55. PREFETCH_ENABLE

```
#define PREFETCH_ENABLE 1U
```

8.17.1.56. TICK_INT_PRIORITY

```
#define TICK_INT_PRIORITY 15U
```

tick interrupt priority

8.17.1.57. USE_HAL_ADC_REGISTER_CALLBACKS

```
#define USE_HAL_ADC_REGISTER_CALLBACKS 0U /* ADC register callback disabled */
```

8.17.1.58. USE_HAL_CAN_REGISTER_CALLBACKS

```
#define USE_HAL_CAN_REGISTER_CALLBACKS 0U /* CAN register callback disabled */
```

8.17.1.59. USE_HAL_CEC_REGISTER_CALLBACKS

```
#define USE_HAL_CEC_REGISTER_CALLBACKS 0U /* CEC register callback disabled */
```

8.17.1.60. USE_HAL_Cryp_REGISTER_CALLBACKS

```
#define USE_HAL_Cryp_REGISTER_CALLBACKS 0U /* CRYPT register callback disabled */
```

8.17.1.61. USE_HAL_DAC_REGISTER_CALLBACKS

```
#define USE_HAL_DAC_REGISTER_CALLBACKS 0U /* DAC register callback disabled */
```

8.17.1.62. USE_HAL_DCMI_REGISTER_CALLBACKS

```
#define USE_HAL_DCMI_REGISTER_CALLBACKS 0U /* DCMI register callback disabled */
```

8.17.1.63. USE_HAL_DFSDM_REGISTER_CALLBACKS

```
#define USE_HAL_DFSDM_REGISTER_CALLBACKS 0U /* DFSDM register callback disabled */
```

8.17.1.64. USE_HAL_DMA2D_REGISTER_CALLBACKS

```
#define USE_HAL_DMA2D_REGISTER_CALLBACKS 0U /* DMA2D register callback disabled */
```

8.17.1.65. USE_HAL_DSI_REGISTER_CALLBACKS

```
#define USE_HAL_DSI_REGISTER_CALLBACKS 0U /* DSI register callback disabled */
```

8.17.1.66. USE_HAL_ETH_REGISTER_CALLBACKS

```
#define USE_HAL_ETH_REGISTER_CALLBACKS 0U /* ETH register callback disabled */
```

8.17.1.67. USE_HAL_FMPI2C_REGISTER_CALLBACKS

```
#define USE_HAL_FMPI2C_REGISTER_CALLBACKS 0U /* FMPI2C register callback disabled */
```

8.17.1.68. USE_HAL_FMPUSBUS_REGISTER_CALLBACKS

```
#define USE_HAL_FMPUSBUS_REGISTER_CALLBACKS 0U /* FMPUSBUS register callback disabled */
```

8.17.1.69. USE_HAL_HASH_REGISTER_CALLBACKS

```
#define USE_HAL_HASH_REGISTER_CALLBACKS 0U /* HASH register callback disabled */
```

8.17.1.70. USE_HAL_HCD_REGISTER_CALLBACKS

```
#define USE_HAL_HCD_REGISTER_CALLBACKS 0U /* HCD register callback disabled */
```

8.17.1.71. USE_HAL_I2C_REGISTER_CALLBACKS

```
#define USE_HAL_I2C_REGISTER_CALLBACKS 0U /* I2C register callback disabled */
```

8.17.1.72. USE_HAL_I2S_REGISTER_CALLBACKS

```
#define USE_HAL_I2S_REGISTER_CALLBACKS 0U /* I2S register callback disabled */
```

8.17.1.73. USE_HAL_IRDA_REGISTER_CALLBACKS

```
#define USE_HAL_IRDA_REGISTER_CALLBACKS 0U /* IRDA register callback disabled */
```

8.17.1.74. USE_HAL_LPTIM_REGISTER_CALLBACKS

```
#define USE_HAL_LPTIM_REGISTER_CALLBACKS 0U /* LPTIM register callback disabled */
```

8.17.1.75. USE_HAL_LTDC_REGISTER_CALLBACKS

```
#define USE_HAL_LTDC_REGISTER_CALLBACKS 0U /* LTDC register callback disabled */
```

8.17.1.76. USE_HAL_MMC_REGISTER_CALLBACKS

```
#define USE_HAL_MMC_REGISTER_CALLBACKS 0U /* MMC register callback disabled */
```

8.17.1.77. USE_HAL_NAND_REGISTER_CALLBACKS

```
#define USE_HAL_NAND_REGISTER_CALLBACKS 0U /* NAND register callback disabled */
```

8.17.1.78. USE_HAL_NOR_REGISTER_CALLBACKS

```
#define USE_HAL_NOR_REGISTER_CALLBACKS 0U /* NOR register callback disabled */
```

8.17.1.79. USE_HAL_PCCARD_REGISTER_CALLBACKS

```
#define USE_HAL_PCCARD_REGISTER_CALLBACKS 0U /* PCCARD register callback disabled */
```

8.17.1.80. USE_HAL_PCD_REGISTER_CALLBACKS

```
#define USE_HAL_PCD_REGISTER_CALLBACKS 0U /* PCD register callback disabled */
```

8.17.1.81. USE_HAL_QSPI_REGISTER_CALLBACKS

```
#define USE_HAL_QSPI_REGISTER_CALLBACKS 0U /* QSPI register callback disabled */
```

8.17.1.82. USE_HAL_RNG_REGISTER_CALLBACKS

```
#define USE_HAL_RNG_REGISTER_CALLBACKS 0U /* RNG register callback disabled */
```

8.17.1.83. USE_HAL_RTC_REGISTER_CALLBACKS

```
#define USE_HAL_RTC_REGISTER_CALLBACKS 0U /* RTC register callback disabled */
```

8.17.1.84. USE_HAL_SAI_REGISTER_CALLBACKS

```
#define USE_HAL_SAI_REGISTER_CALLBACKS 0U /* SAI register callback disabled */
```

8.17.1.85. USE_HAL_SD_REGISTER_CALLBACKS

```
#define USE_HAL_SD_REGISTER_CALLBACKS 0U /* SD register callback disabled */
```

8.17.1.86. USE_HAL_SDRAM_REGISTER_CALLBACKS

```
#define USE_HAL_SDRAM_REGISTER_CALLBACKS 0U /* SDRAM register callback disabled */
```

8.17.1.87. USE_HAL_SMARTCARD_REGISTER_CALLBACKS

```
#define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U /* SMARTCARD register callback disabled */
```

8.17.1.88. USE_HAL_SMBUS_REGISTER_CALLBACKS

```
#define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U /* SMBUS register callback disabled */
```

8.17.1.89. USE_HAL_SPDIFRX_REGISTER_CALLBACKS

```
#define USE_HAL_SPDIFRX_REGISTER_CALLBACKS 0U /* SPDIFRX register callback disabled */
```

8.17.1.90. USE_HAL_SPI_REGISTER_CALLBACKS

```
#define USE_HAL_SPI_REGISTER_CALLBACKS 0U /* SPI register callback disabled */
```

8.17.1.91. USE_HAL_SRAM_REGISTER_CALLBACKS

```
#define USE_HAL_SRAM_REGISTER_CALLBACKS 0U /* SRAM register callback disabled */
```

8.17.1.92. USE_HAL_TIM_REGISTER_CALLBACKS

```
#define USE_HAL_TIM_REGISTER_CALLBACKS 0U /* TIM register callback disabled */
```

8.17.1.93. USE_HAL_UART_REGISTER_CALLBACKS

```
#define USE_HAL_UART_REGISTER_CALLBACKS 0U /* UART register callback disabled */
```

8.17.1.94. USE_HAL_USART_REGISTER_CALLBACKS

```
#define USE_HAL_USART_REGISTER_CALLBACKS 0U /* USART register callback disabled */
```

8.17.1.95. USE_HAL_WWDG_REGISTER_CALLBACKS

```
#define USE_HAL_WWDG_REGISTER_CALLBACKS 0U /* WWDG register callback disabled */
```

8.17.1.96. USE_RTOS

```
#define USE_RTOS 0U
```

8.17.1.97. USE_SPI_CRC

```
#define USE_SPI_CRC 0U
```

8.17.1.98. VDD_VALUE

```
#define VDD_VALUE 3300U
```

This is the HAL system configuration section.

Value of VDD in mv

8.18. stm32f4xx_hal_conf.h

[Ir a la documentación de este archivo.](#)

```
00001 /* USER CODE BEGIN Header */
00021 /* USER CODE END Header */
00022
00023 /* Define to prevent recursive inclusion -----*/
00024 #ifndef __STM32F4xx_HAL_CONF_H
00025 #define __STM32F4xx_HAL_CONF_H
00026
00027 #ifdef __cplusplus
00028 extern "C" {
00029 #endif
00030
00031 /* Exported types -----*/
00032 /* Exported constants -----*/
00033
00034 /* ##### Module Selection ##### */
00038 #define HAL_MODULE_ENABLED
00039
00040 /* #define HAL_Cryp_MODULE_ENABLED */
00041 /* #define HAL_ADC_MODULE_ENABLED */
00042 /* #define HAL_CAN_MODULE_ENABLED */
00043 /* #define HAL_CRC_MODULE_ENABLED */
00044 /* #define HAL_CAN_LEGACY_MODULE_ENABLED */
00045 /* #define HAL_DAC_MODULE_ENABLED */
00046 /* #define HAL_DCMI_MODULE_ENABLED */
00047 /* #define HAL_DMA2D_MODULE_ENABLED */
00048 /* #define HAL_ETH_MODULE_ENABLED */
00049 /* #define HAL_ETH_LEGACY_MODULE_ENABLED */
00050 /* #define HAL_NAND_MODULE_ENABLED */
00051 /* #define HAL_NOR_MODULE_ENABLED */
00052 /* #define HAL_PCCARD_MODULE_ENABLED */
00053 /* #define HAL_SRAM_MODULE_ENABLED */
00054 /* #define HAL_SDRAM_MODULE_ENABLED */
00055 /* #define HAL_HASH_MODULE_ENABLED */
00056 #define HAL_I2C_MODULE_ENABLED
00057 /* #define HAL_I2S_MODULE_ENABLED */
00058 /* #define HAL_IWDG_MODULE_ENABLED */
00059 /* #define HAL_LTDC_MODULE_ENABLED */
00060 /* #define HAL_RNG_MODULE_ENABLED */
00061 /* #define HAL_RTC_MODULE_ENABLED */
00062 /* #define HAL_SAI_MODULE_ENABLED */
```

```

00063 /* #define HAL_SD_MODULE_ENABLED */
00064 /* #define HAL_MMC_MODULE_ENABLED */
00065 /* #define HAL_SPI_MODULE_ENABLED */
00066 #define HAL_TIM_MODULE_ENABLED
00067 #define HAL_UART_MODULE_ENABLED
00068 /* #define HAL_USART_MODULE_ENABLED */
00069 /* #define HAL_IRDA_MODULE_ENABLED */
00070 /* #define HAL_SMARTCARD_MODULE_ENABLED */
00071 /* #define HAL_SMBUS_MODULE_ENABLED */
00072 /* #define HAL_WWDG_MODULE_ENABLED */
00073 /* #define HAL_PCD_MODULE_ENABLED */
00074 /* #define HAL_HCD_MODULE_ENABLED */
00075 /* #define HAL_DSI_MODULE_ENABLED */
00076 /* #define HAL_QSPI_MODULE_ENABLED */
00077 /* #define HAL_QSPI_MODULE_ENABLED */
00078 /* #define HAL_CEC_MODULE_ENABLED */
00079 /* #define HAL_FMPI2C_MODULE_ENABLED */
00080 /* #define HAL_FMPMBUS_MODULE_ENABLED */
00081 /* #define HAL_SPDIFRX_MODULE_ENABLED */
00082 /* #define HAL_DFSDM_MODULE_ENABLED */
00083 /* #define HAL_LPTIM_MODULE_ENABLED */
00084 #define HAL_GPIO_MODULE_ENABLED
00085 #define HAL_EXTI_MODULE_ENABLED
00086 #define HAL_DMA_MODULE_ENABLED
00087 #define HAL_RCC_MODULE_ENABLED
00088 #define HAL_FLASH_MODULE_ENABLED
00089 #define HAL_PWR_MODULE_ENABLED
00090 #define HAL_CORTEX_MODULE_ENABLED
00091
00092 /* ##### HSE/HSI Values adaptation ##### */
00093 #if !defined (HSE_VALUE)
00094     #define HSE_VALUE      8000000U
00095 #endif /* HSE_VALUE */
00096
00097 #if !defined (HSE_STARTUP_TIMEOUT)
00098     #define HSE_STARTUP_TIMEOUT    100U
00099 #endif /* HSE_STARTUP_TIMEOUT */
00100
00101 #if !defined (HSI_VALUE)
00102     #define HSI_VALUE      ((uint32_t)16000000U)
00103 #endif /* HSI_VALUE */
00104
00105 #if !defined (LSI_VALUE)
00106     #define LSI_VALUE      32000U
00107 #endif /* LSI_VALUE */
00108
00109 #if !defined (LSE_VALUE)
00110     #define LSE_VALUE      32768U
00111 #endif /* LSE_VALUE */
00112
00113 #if !defined (LSE_STARTUP_TIMEOUT)
00114     #define LSE_STARTUP_TIMEOUT    5000U
00115 #endif /* LSE_STARTUP_TIMEOUT */
00116
00117 #if !defined (EXTERNAL_CLOCK_VALUE)
00118     #define EXTERNAL_CLOCK_VALUE    12288000U
00119 #endif /* EXTERNAL_CLOCK_VALUE */
00120
00121 /* Tip: To avoid modifying this file each time you need to use different HSE,
00122    === you can define the HSE value in your toolchain compiler preprocessor. */
00123
00124 /* ##### System Configuration ##### */
00125 #define VDD_VALUE                3300U
00126 #define TICK_INT_PRIORITY        15U
00127 #define USE_RTOS                  0U
00128 #define PREFETCH_ENABLE          1U
00129 #define INSTRUCTION_CACHE_ENABLE 1U
00130 #define DATA_CACHE_ENABLE       1U
00131
00132 #define USE_HAL_ADC_REGISTER_CALLBACKS    0U /* ADC register callback disabled */
00133 #define USE_HAL_CAN_REGISTER_CALLBACKS    0U /* CAN register callback disabled */
00134 #define USE_HAL_CEC_REGISTER_CALLBACKS    0U /* CEC register callback disabled */
00135 #define USE_HAL_Cryp_REGISTER_CALLBACKS    0U /* CRYP register callback disabled */
00136 #define USE_HAL_DAC_REGISTER_CALLBACKS    0U /* DAC register callback disabled */
00137 #define USE_HAL_DCMI_REGISTER_CALLBACKS    0U /* DCMI register callback disabled */
00138 #define USE_HAL_DFSDM_REGISTER_CALLBACKS    0U /* DFSDM register callback disabled */
00139 #define USE_HAL_DMA2D_REGISTER_CALLBACKS    0U /* DMA2D register callback disabled */
00140 #define USE_HAL_DSI_REGISTER_CALLBACKS    0U /* DSI register callback disabled */
00141 #define USE_HAL_ETH_REGISTER_CALLBACKS    0U /* ETH register callback disabled */
00142 #define USE_HAL_HASH_REGISTER_CALLBACKS    0U /* HASH register callback disabled */
00143 #define USE_HAL_HCD_REGISTER_CALLBACKS    0U /* HCD register callback disabled */
00144 #define USE_HAL_I2C_REGISTER_CALLBACKS    0U /* I2C register callback disabled */
00145 #define USE_HAL_FMPI2C_REGISTER_CALLBACKS    0U /* FMPI2C register callback disabled */
00146 #define USE_HAL_FMPMBUS_REGISTER_CALLBACKS    0U /* FMPSMBUS register callback disabled */
00147 #define USE_HAL_I2S_REGISTER_CALLBACKS    0U /* I2S register callback disabled */
00148 #define USE_HAL_IRDA_REGISTER_CALLBACKS    0U /* IRDA register callback disabled */
00149 #define USE_HAL_LPTIM_REGISTER_CALLBACKS    0U /* LPTIM register callback disabled */
00150 #define USE_HAL_LTDC_REGISTER_CALLBACKS    0U /* LTDC register callback disabled */

```

```

00176 #define USE_HAL_MMC_REGISTER_CALLBACKS 0U /* MMC register callback disabled */
00177 #define USE_HAL_NAND_REGISTER_CALLBACKS 0U /* NAND register callback disabled */
00178 #define USE_HAL_NOR_REGISTER_CALLBACKS 0U /* NOR register callback disabled */
00179 #define USE_HAL_PCCARD_REGISTER_CALLBACKS 0U /* PCCARD register callback disabled */
00180 #define USE_HAL_PCD_REGISTER_CALLBACKS 0U /* PCD register callback disabled */
00181 #define USE_HAL_QSPI_REGISTER_CALLBACKS 0U /* QSPI register callback disabled */
00182 #define USE_HAL_RNG_REGISTER_CALLBACKS 0U /* RNG register callback disabled */
00183 #define USE_HAL_RTC_REGISTER_CALLBACKS 0U /* RTC register callback disabled */
00184 #define USE_HAL_SAI_REGISTER_CALLBACKS 0U /* SAI register callback disabled */
00185 #define USE_HAL_SD_REGISTER_CALLBACKS 0U /* SD register callback disabled */
00186 #define USE_HAL_SMARTCARD_REGISTER_CALLBACKS 0U /* SMARTCARD register callback disabled */
00187 #define USE_HAL_SDRAM_REGISTER_CALLBACKS 0U /* SDRAM register callback disabled */
00188 #define USE_HAL_SRAM_REGISTER_CALLBACKS 0U /* SRAM register callback disabled */
00189 #define USE_HAL_SPDIFRX_REGISTER_CALLBACKS 0U /* SPDIFRX register callback disabled */
00190 #define USE_HAL_SMBUS_REGISTER_CALLBACKS 0U /* SMBUS register callback disabled */
00191 #define USE_HAL_SPI_REGISTER_CALLBACKS 0U /* SPI register callback disabled */
00192 #define USE_HAL_TIM_REGISTER_CALLBACKS 0U /* TIM register callback disabled */
00193 #define USE_HAL_UART_REGISTER_CALLBACKS 0U /* UART register callback disabled */
00194 #define USE_HAL_USART_REGISTER_CALLBACKS 0U /* USART register callback disabled */
00195 #define USE_HAL_WWDG_REGISTER_CALLBACKS 0U /* WWDG register callback disabled */
00196
00197 /* ##### Assert Selection ##### */
00202 /* #define USE_FULL_ASSERT 1U */
00203
00204 /* ##### Ethernet peripheral configuration ##### */
00205
00206 /* Section 1 : Ethernet peripheral configuration */
00207
00208 /* MAC ADDRESS: MAC_ADDR0:MAC_ADDR1:MAC_ADDR2:MAC_ADDR3:MAC_ADDR4:MAC_ADDR5 */
00209 #define MAC_ADDR0 2U
00210 #define MAC_ADDR1 0U
00211 #define MAC_ADDR2 0U
00212 #define MAC_ADDR3 0U
00213 #define MAC_ADDR4 0U
00214 #define MAC_ADDR5 0U
00215
00216 /* Definition of the Ethernet driver buffers size and count */
00217 #define ETH_RX_BUF_SIZE ETH_MAX_PACKET_SIZE /* buffer size for receive */
00218 #define ETH_TX_BUF_SIZE ETH_MAX_PACKET_SIZE /* buffer size for transmit */
00219 #define ETH_RXBUFNB 4U /* 4 Rx buffers of size ETH_RX_BUF_SIZE */
00220 #define ETH_TXBUFNB 4U /* 4 Tx buffers of size ETH_TX_BUF_SIZE */
00221
00222 /* Section 2: PHY configuration section */
00223
00224 /* DP83848_PHY_ADDRESS Address*/
00225 #define DP83848_PHY_ADDRESS
00226 /* PHY Reset delay these values are based on a 1 ms Systick interrupt*/
00227 #define PHY_RESET_DELAY 0x000000FFU
00228 /* PHY Configuration delay */
00229 #define PHY_CONFIG_DELAY 0x000000FFU
00230
00231 #define PHY_READ_TO 0x0000FFFFU
00232 #define PHY_WRITE_TO 0x0000FFFFU
00233
00234 /* Section 3: Common PHY Registers */
00235
00236 #define PHY_BCR ((uint16_t)0x0000U)
00237 #define PHY_BSR ((uint16_t)0x0001U)
00238
00239 #define PHY_RESET ((uint16_t)0x8000U)
00240 #define PHY_LOOPBACK ((uint16_t)0x4000U)
00241 #define PHY_FULLDUPLEX_100M ((uint16_t)0x2100U)
00242 #define PHY_HALFDUPLEX_100M ((uint16_t)0x2000U)
00243 #define PHY_FULLDUPLEX_10M ((uint16_t)0x0100U)
00244 #define PHY_HALFDUPLEX_10M ((uint16_t)0x0000U)
00245 #define PHY_AUTONEGOTIATION ((uint16_t)0x1000U)
00246 #define PHY_RESTART_AUTONEGOTIATION ((uint16_t)0x0200U)
00247 #define PHY_POWERDOWN ((uint16_t)0x0800U)
00248 #define PHY_ISOLATE ((uint16_t)0x0400U)
00249
00250 #define PHY_AUTONEGO_COMPLETE ((uint16_t)0x0020U)
00251 #define PHY_LINKED_STATUS ((uint16_t)0x0004U)
00252 #define PHY_JABBER_DETECTION ((uint16_t)0x0002U)
00253
00254 /* Section 4: Extended PHY Registers */
00255 #define PHY_SR ((uint16_t))
00256
00257 #define PHY_SPEED_STATUS ((uint16_t))
00258 #define PHY_DUPLEX_STATUS ((uint16_t))
00259
00260 /* ##### SPI peripheral configuration ##### */
00261
00262 /* CRC FEATURE: Use to activate CRC feature inside HAL SPI Driver
00263 * Activated: CRC code is present inside driver
00264 * Deactivated: CRC code cleaned from driver
00265 */
00266

```

```
00267 #define USE_SPI_CRC                                0U
00268
00269 /* Includes ----- */
00270
00271 #ifndef HAL_RCC_MODULE_ENABLED
00272     #include "stm32f4xx_hal_rcc.h"
00273 #endif /* HAL_RCC_MODULE_ENABLED */
00274
00275 #ifndef HAL_GPIO_MODULE_ENABLED
00276     #include "stm32f4xx_hal_gpio.h"
00277 #endif /* HAL_GPIO_MODULE_ENABLED */
00278
00279 #ifndef HAL_EXTI_MODULE_ENABLED
00280     #include "stm32f4xx_hal_exti.h"
00281 #endif /* HAL_EXTI_MODULE_ENABLED */
00282
00283 #ifndef HAL_DMA_MODULE_ENABLED
00284     #include "stm32f4xx_hal_dma.h"
00285 #endif /* HAL_DMA_MODULE_ENABLED */
00286
00287 #ifndef HAL_CORTEX_MODULE_ENABLED
00288     #include "stm32f4xx_hal_cortex.h"
00289 #endif /* HAL_CORTEX_MODULE_ENABLED */
00290
00291 #ifndef HAL_ADC_MODULE_ENABLED
00292     #include "stm32f4xx_hal_adc.h"
00293 #endif /* HAL_ADC_MODULE_ENABLED */
00294
00295 #ifndef HAL_CAN_MODULE_ENABLED
00296     #include "stm32f4xx_hal_can.h"
00297 #endif /* HAL_CAN_MODULE_ENABLED */
00298
00299 #ifndef HAL_CAN_LEGACY_MODULE_ENABLED
00300     #include "stm32f4xx_hal_can_legacy.h"
00301 #endif /* HAL_CAN_LEGACY_MODULE_ENABLED */
00302
00303 #ifndef HAL_CRC_MODULE_ENABLED
00304     #include "stm32f4xx_hal_crc.h"
00305 #endif /* HAL_CRC_MODULE_ENABLED */
00306
00307 #ifndef HAL_Cryp_MODULE_ENABLED
00308     #include "stm32f4xx_hal_cryp.h"
00309 #endif /* HAL_Cryp_MODULE_ENABLED */
00310
00311 #ifndef HAL_DMA2D_MODULE_ENABLED
00312     #include "stm32f4xx_hal_dma2d.h"
00313 #endif /* HAL_DMA2D_MODULE_ENABLED */
00314
00315 #ifndef HAL_DAC_MODULE_ENABLED
00316     #include "stm32f4xx_hal_dac.h"
00317 #endif /* HAL_DAC_MODULE_ENABLED */
00318
00319 #ifndef HAL_DCMI_MODULE_ENABLED
00320     #include "stm32f4xx_hal_dcmi.h"
00321 #endif /* HAL_DCMI_MODULE_ENABLED */
00322
00323 #ifndef HAL_ETH_MODULE_ENABLED
00324     #include "stm32f4xx_hal_eth.h"
00325 #endif /* HAL_ETH_MODULE_ENABLED */
00326
00327 #ifndef HAL_ETH_LEGACY_MODULE_ENABLED
00328     #include "stm32f4xx_hal_eth_legacy.h"
00329 #endif /* HAL_ETH_LEGACY_MODULE_ENABLED */
00330
00331 #ifndef HAL_FLASH_MODULE_ENABLED
00332     #include "stm32f4xx_hal_flash.h"
00333 #endif /* HAL_FLASH_MODULE_ENABLED */
00334
00335 #ifndef HAL_SRAM_MODULE_ENABLED
00336     #include "stm32f4xx_hal_sram.h"
00337 #endif /* HAL_SRAM_MODULE_ENABLED */
00338
00339 #ifndef HAL_NOR_MODULE_ENABLED
00340     #include "stm32f4xx_hal_nor.h"
00341 #endif /* HAL_NOR_MODULE_ENABLED */
00342
00343 #ifndef HAL_NAND_MODULE_ENABLED
00344     #include "stm32f4xx_hal_nand.h"
00345 #endif /* HAL_NAND_MODULE_ENABLED */
00346
00347 #ifndef HAL_PCCARD_MODULE_ENABLED
00348     #include "stm32f4xx_hal_pccard.h"
00349 #endif /* HAL_PCCARD_MODULE_ENABLED */
00350
00351 #ifndef HAL_SDRAM_MODULE_ENABLED
00352     #include "stm32f4xx_hal_sdram.h"
00353 #endif /* HAL_SDRAM_MODULE_ENABLED */
```

```
00357
00358 #ifndef HAL_HASH_MODULE_ENABLED
00359 #include "stm32f4xx_hal_hash.h"
00360 #endif /* HAL_HASH_MODULE_ENABLED */
00361
00362 #ifndef HAL_I2C_MODULE_ENABLED
00363 #include "stm32f4xx_hal_i2c.h"
00364 #endif /* HAL_I2C_MODULE_ENABLED */
00365
00366 #ifndef HAL_SMBUS_MODULE_ENABLED
00367 #include "stm32f4xx_hal_smbus.h"
00368 #endif /* HAL_SMBUS_MODULE_ENABLED */
00369
00370 #ifndef HAL_I2S_MODULE_ENABLED
00371 #include "stm32f4xx_hal_i2s.h"
00372 #endif /* HAL_I2S_MODULE_ENABLED */
00373
00374 #ifndef HAL_IWDG_MODULE_ENABLED
00375 #include "stm32f4xx_hal_iwdg.h"
00376 #endif /* HAL_IWDG_MODULE_ENABLED */
00377
00378 #ifndef HAL_LTDC_MODULE_ENABLED
00379 #include "stm32f4xx_hal_ltdc.h"
00380 #endif /* HAL_LTDC_MODULE_ENABLED */
00381
00382 #ifndef HAL_PWR_MODULE_ENABLED
00383 #include "stm32f4xx_hal_pwr.h"
00384 #endif /* HAL_PWR_MODULE_ENABLED */
00385
00386 #ifndef HAL_RNG_MODULE_ENABLED
00387 #include "stm32f4xx_hal_rng.h"
00388 #endif /* HAL_RNG_MODULE_ENABLED */
00389
00390 #ifndef HAL_RTC_MODULE_ENABLED
00391 #include "stm32f4xx_hal_rtc.h"
00392 #endif /* HAL_RTC_MODULE_ENABLED */
00393
00394 #ifndef HAL_SAI_MODULE_ENABLED
00395 #include "stm32f4xx_hal_sai.h"
00396 #endif /* HAL_SAI_MODULE_ENABLED */
00397
00398 #ifndef HAL_SD_MODULE_ENABLED
00399 #include "stm32f4xx_hal_sd.h"
00400 #endif /* HAL_SD_MODULE_ENABLED */
00401
00402 #ifndef HAL_SPI_MODULE_ENABLED
00403 #include "stm32f4xx_hal_spi.h"
00404 #endif /* HAL_SPI_MODULE_ENABLED */
00405
00406 #ifndef HAL_TIM_MODULE_ENABLED
00407 #include "stm32f4xx_hal_tim.h"
00408 #endif /* HAL_TIM_MODULE_ENABLED */
00409
00410 #ifndef HAL_UART_MODULE_ENABLED
00411 #include "stm32f4xx_hal_uart.h"
00412 #endif /* HAL_UART_MODULE_ENABLED */
00413
00414 #ifndef HAL_USART_MODULE_ENABLED
00415 #include "stm32f4xx_hal_usart.h"
00416 #endif /* HAL_USART_MODULE_ENABLED */
00417
00418 #ifndef HAL_IRDA_MODULE_ENABLED
00419 #include "stm32f4xx_hal_irda.h"
00420 #endif /* HAL_IRDA_MODULE_ENABLED */
00421
00422 #ifndef HAL_SMARTCARD_MODULE_ENABLED
00423 #include "stm32f4xx_hal_smartcard.h"
00424 #endif /* HAL_SMARTCARD_MODULE_ENABLED */
00425
00426 #ifndef HAL_WWDG_MODULE_ENABLED
00427 #include "stm32f4xx_hal_wwdg.h"
00428 #endif /* HAL_WWDG_MODULE_ENABLED */
00429
00430 #ifndef HAL_PCD_MODULE_ENABLED
00431 #include "stm32f4xx_hal_pcd.h"
00432 #endif /* HAL_PCD_MODULE_ENABLED */
00433
00434 #ifndef HAL_HCD_MODULE_ENABLED
00435 #include "stm32f4xx_hal_hcd.h"
00436 #endif /* HAL_HCD_MODULE_ENABLED */
00437
00438 #ifndef HAL_DSI_MODULE_ENABLED
00439 #include "stm32f4xx_hal_dsi.h"
00440 #endif /* HAL_DSI_MODULE_ENABLED */
00441
00442 #ifndef HAL_QSPI_MODULE_ENABLED
00443 #include "stm32f4xx_hal_qspi.h"
```

```

00444 #endif /* HAL_QSPI_MODULE_ENABLED */
00445
00446 #ifdef HAL_CEC_MODULE_ENABLED
00447     #include "stm32f4xx_hal_cec.h"
00448 #endif /* HAL_CEC_MODULE_ENABLED */
00449
00450 #ifdef HAL_FMPI2C_MODULE_ENABLED
00451     #include "stm32f4xx_hal_fmpi2c.h"
00452 #endif /* HAL_FMPI2C_MODULE_ENABLED */
00453
00454 #ifdef HAL_FMPMBUS_MODULE_ENABLED
00455     #include "stm32f4xx_hal_fmpmbus.h"
00456 #endif /* HAL_FMPMBUS_MODULE_ENABLED */
00457
00458 #ifdef HAL_SPDIFRX_MODULE_ENABLED
00459     #include "stm32f4xx_hal_spdifrx.h"
00460 #endif /* HAL_SPDIFRX_MODULE_ENABLED */
00461
00462 #ifdef HAL_DFSDM_MODULE_ENABLED
00463     #include "stm32f4xx_hal_dfsdm.h"
00464 #endif /* HAL_DFSDM_MODULE_ENABLED */
00465
00466 #ifdef HAL_LPTIM_MODULE_ENABLED
00467     #include "stm32f4xx_hal_lptim.h"
00468 #endif /* HAL_LPTIM_MODULE_ENABLED */
00469
00470 #ifdef HAL_MMC_MODULE_ENABLED
00471     #include "stm32f4xx_hal_mmc.h"
00472 #endif /* HAL_MMC_MODULE_ENABLED */
00473
00474 /* Exported macro -----*/
00475 #ifdef USE_FULL_ASSERT
00484     #define assert_param(expr) ((expr) ? (void)0U : assert_failed((uint8_t *)__FILE__, __LINE__))
00485 /* Exported functions ----- */
00486     void assert_failed(uint8_t* file, uint32_t line);
00487 #else
00488     #define assert_param(expr) ((void)0U)
00489 #endif /* USE_FULL_ASSERT */
00490
00491 #ifdef __cplusplus
00492 }
00493 #endif
00494
00495 #endif /* __STM32F4xx_HAL_CONF_H */

```

8.19. Referencia del archivo Core/Inc/tim.h

This file contains all the function prototypes for the tim.c file.

```
#include "main.h"
```

Gráfico de dependencias incluidas en tim.h:

Funciones

- void [MX_TIM1_Init](#) (void)
- void [MX_TIM3_Init](#) (void)

Variables

- TIM_HandleTypeDef [htim1](#)
- TIM_HandleTypeDef [htim3](#)

8.19.1. Descripción detallada

This file contains all the function prototypes for the tim.c file.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.19.2. Documentación de funciones

8.19.2.1. MX_TIM1_Init()

```
void MX_TIM1_Init (  
    void )
```

8.19.2.2. MX_TIM3_Init()

```
void MX_TIM3_Init (  
    void )
```

8.19.3. Documentación de variables

8.19.3.1. htim1

```
TIM_HandleTypeDef htim1 [extern]
```

8.19.3.2. htim3

```
TIM_HandleTypeDef htim3 [extern]
```

8.20. tim.h

[Ir a la documentación de este archivo.](#)

```

00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020 /* Define to prevent recursive inclusion -----*/
00021 #ifndef __TIM_H__
00022 #define __TIM_H__
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00028 /* Includes -----*/
00029 #include "main.h"
00030
00031 /* USER CODE BEGIN Includes */
00032
00033 /* USER CODE END Includes */
00034
00035 extern TIM_HandleTypeDef htim1;
00036
00037 extern TIM_HandleTypeDef htim3;
00038
00039 /* USER CODE BEGIN Private defines */
00040
00041 /* USER CODE END Private defines */
00042
00043 void MX_TIM1_Init(void);
00044 void MX_TIM3_Init(void);
00045
00046 /* USER CODE BEGIN Prototypes */
00047
00048 /* USER CODE END Prototypes */
00049
00050 #ifdef __cplusplus
00051 }
00052 #endif
00053
00054 #endif /* __TIM_H__ */
00055

```

8.21. Referencia del archivo Core/Inc/usart.h

This file contains all the function prototypes for the usart.c file.

```
#include "main.h"
```

Gráfico de dependencias incluidas en usart.h:

Funciones

- void [MX_USART2_UART_Init](#) (void)
- void [MX_USART3_UART_Init](#) (void)

Variables

- UART_HandleTypeDef [huart2](#)
- UART_HandleTypeDef [huart3](#)

8.21.1. Descripción detallada

This file contains all the function prototypes for the usart.c file.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.21.2. Documentación de funciones

8.21.2.1. MX_USART2_UART_Init()

```
void MX_USART2_UART_Init (  
    void )
```

8.21.2.2. MX_USART3_UART_Init()

```
void MX_USART3_UART_Init (  
    void )
```

8.21.3. Documentación de variables

8.21.3.1. huart2

```
UART_HandleTypeDef huart2  [extern]
```

8.21.3.2. huart3

```
UART_HandleTypeDef huart3  [extern]
```

8.22. usart.h

[Ir a la documentación de este archivo.](#)

```

00001 /* USER CODE BEGIN Header */
00019 /* USER CODE END Header */
00020 /* Define to prevent recursive inclusion -----*/
00021 #ifndef __USART_H__
00022 #define __USART_H__
00023
00024 #ifdef __cplusplus
00025 extern "C" {
00026 #endif
00027
00028 /* Includes -----*/
00029 #include "main.h"
00030
00031 /* USER CODE BEGIN Includes */
00032
00033 /* USER CODE END Includes */
00034
00035 extern UART_HandleTypeDef huart2;
00036
00037 extern UART_HandleTypeDef huart3;
00038
00039 /* USER CODE BEGIN Private defines */
00040
00041 /* USER CODE END Private defines */
00042
00043 void MX_USART2_UART_Init(void);
00044 void MX_USART3_UART_Init(void);
00045
00046 /* USER CODE BEGIN Prototypes */
00047
00048 /* USER CODE END Prototypes */
00049
00050 #ifdef __cplusplus
00051 }
00052 #endif
00053
00054 #endif /* __USART_H__ */
00055

```

8.23. Referencia del archivo Core/Inc/wifi.h

Interfaz pública del driver WiFi basado en comandos AT.

```

#include "main.h"
#include "wifi_types.h"
#include <stdint.h>

```

Gráfico de dependencias incluidas en wifi.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Funciones

- [wifi_status_t WIFI_Init](#) (UART_HandleTypeDef *huart)
Inicializa el driver WiFi y asocia la UART del módulo.
- [wifi_status_t WIFI_Reset](#) (void)
Reinicia el módulo WiFi.
- [wifi_status_t WIFI_Test](#) (void)
Comprueba la comunicación básica con el módulo WiFi.
- [wifi_status_t WIFI_SetStationMode](#) (void)
Configura el módulo WiFi en modo estación.
- [wifi_status_t WIFI_ConnectAP](#) (const char *ssid, const char *pass)
Conecta el módulo WiFi a un punto de acceso.

- `wifi_status_t WIFI_DisconnectAP` (void)
Desconecta el módulo del punto de acceso actual.
- `wifi_state_t WIFI_GetState` (void)
Devuelve el estado lógico actual del driver WiFi.
- `wifi_status_t WIFI_GetIP` (char *ip, uint16_t len)
Solicita la dirección IP asignada al módulo.
- `wifi_status_t WIFI_GetRSSI` (int *rssi)
Solicita el nivel RSSI de la conexión WiFi.
- `wifi_status_t WIFI_TCPConnect` (const char *host, uint16_t port)
Abre una conexión TCP con un host remoto.
- `wifi_status_t WIFI_TCPClose` (void)
Cierra la conexión TCP activa.
- `wifi_status_t WIFI_SendRaw` (const char *data)
Envía datos sin procesar por la conexión activa.
- `wifi_status_t WIFI_SendJSON` (const char *json)
Envía una cadena JSON por la conexión activa.
- `wifi_status_t WIFI_Reconnect` (void)
Ejecuta la secuencia de reconexión WiFi definida por el driver.
- void `WIFI_Process` (void)
Procesa tareas internas periódicas del módulo WiFi.
- `wifi_status_t WIFI_HTTP_PostJSON` (const char *host, const char *path, const char *json)
Realiza una petición HTTP POST con contenido JSON.
- `wifi_status_t WIFI_MQTT_UserConfig` (const char *client_id)
Configura el identificador de cliente MQTT.
- `wifi_status_t WIFI_MQTT_Connect` (const char *broker, uint16_t port)
Establece la conexión MQTT con el broker.
- `wifi_status_t WIFI_MQTT_Publish` (const char *topic, const char *payload, uint8_t qos, uint8_t retain)
Publica un payload en un topic MQTT.
- `wifi_status_t WIFI_MQTT_Subscribe` (const char *topic, uint8_t qos)
Suscribe el cliente MQTT a un topic.
- `wifi_status_t WIFI_MQTT_Disconnect` (void)
Cierra la sesión MQTT activa.

8.23.1. Descripción detallada

Interfaz pública del driver WiFi basado en comandos AT.

Autor

Lorenzo

Fecha

25 abr 2026

El módulo encapsula las operaciones de red necesarias para el proyecto: conexión al punto de acceso, transporte TCP, envío HTTP y publicación MQTT. La API mantiene una interfaz de alto nivel para que las tareas de FreeRTOS no dependan del formato concreto de los comandos AT ni de las respuestas del módulo WiFi.

8.23.2. Documentación de funciones

8.23.2.1. WIFI_ConnectAP()

```
wifi_status_t WIFI_ConnectAP (  
    const char * ssid,  
    const char * pass)
```

Conecta el módulo WiFi a un punto de acceso.

Conecta el módulo a un punto de acceso WiFi.

Parámetros

in	<i>ssid</i>	Nombre de la red WiFi.
in	<i>pass</i>	Contraseña de la red WiFi.

Devuelve

WIFI_OK si la asociación se completa correctamente.

WIFI_TIMEOUT si el punto de acceso no responde a tiempo.

WIFI_ERROR si las credenciales o la negociación fallan.

Precondición

ssid y *pass* deben apuntar a cadenas válidas terminadas en cero.

Conecta el módulo WiFi a un punto de acceso.

Parámetros

in	<i>ssid</i>	Nombre de la red WiFi.
in	<i>pass</i>	Contraseña de la red WiFi.

Valores devueltos

<i>WIFI_OK</i>	Conexión establecida y dirección IP obtenida.
<i>WIFI_ERROR</i>	Error de conexión o transmisión.
<i>WIFI_TIMEOUT</i>	No se obtuvo confirmación dentro del tiempo límite.

Las credenciales se almacenan localmente para que [WIFI_Reconnect\(\)](#) pueda reconstruir la conexión sin depender de que la aplicación vuelva a pasarlas. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.2. WIFI_DisconnectAP()

```
wifi_status_t WIFI_DisconnectAP (  
    void )
```

Desconecta el módulo del punto de acceso actual.

Devuelve

WIFI_OK si la desconexión se completa correctamente.

WIFI_ERROR si el módulo no acepta la orden.

Valores devueltos

<i>WIFI_OK</i>	Desconexión aceptada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Gráfico de llamadas de esta función:

8.23.2.3. WIFI_GetIP()

```
wifi_status_t WIFI_GetIP (  
    char * ip,  
    uint16_t len)
```

Solicita la dirección IP asignada al módulo.

Parámetros

out	<i>ip</i>	Buffer donde se copiará la dirección IP.
in	<i>len</i>	Tamaño disponible en el buffer <i>ip</i> .

Devuelve

WIFI_OK si se obtiene una dirección IP válida.

WIFI_NOT_CONNECTED si el módulo no tiene red activa.

WIFI_ERROR si no puede parsearse la respuesta.

Precondición

ip debe apuntar a un buffer válido.

8.23.2.4. WIFI_GetRSSI()

```
wifi_status_t WIFI_GetRSSI (
    int * rssi)
```

Solicita el nivel RSSI de la conexión WiFi.

Parámetros

out	rssi	Variable donde se almacena la potencia recibida.
-----	------	--

Devuelve

WIFI_OK si el RSSI se obtiene correctamente.

WIFI_NOT_CONNECTED si no hay conexión activa.

WIFI_ERROR si la respuesta no puede interpretarse.

Precondición

rssi debe apuntar a memoria válida.

8.23.2.5. WIFI_GetState()

```
wifi_state_t WIFI_GetState (
    void )
```

Devuelve el estado lógico actual del driver WiFi.

Devuelve el estado lógico actual del módulo WiFi/MQTT.

Devuelve

Estado de conexión almacenado por el módulo.

Nota

El estado permite a la aplicación evitar publicaciones cuando todavía no existe conectividad suficiente.

Devuelve el estado lógico actual del driver WiFi.

Devuelve

Estado interno de conexión.

8.23.2.6. WIFI_HTTP_PostJSON()

```
wifi_status_t WIFI_HTTP_PostJSON (  
    const char * host,  
    const char * path,  
    const char * json)
```

Realiza una petición HTTP POST con contenido JSON.

Parámetros

in	<i>host</i>	Servidor HTTP de destino.
in	<i>path</i>	Ruta del recurso al que se envía la petición.
in	<i>json</i>	Cuerpo JSON de la petición.

Devuelve

WIFI_OK si la petición se envía correctamente.

WIFI_NOT_CONNECTED si no existe conectividad suficiente.

WIFI_ERROR si falla la conexión o el envío.

La función ofrece una vía de telemetría alternativa a MQTT manteniendo el mismo formato de datos de aplicación.

8.23.2.7. WIFI_Init()

```
wifi_status_t WIFI_Init (  
    UART_HandleTypeDef * huart)
```

Inicializa el driver WiFi y asocia la UART del módulo.

Inicializa el driver WiFi y prepara la recepción DMA circular.

Parámetros

in	<i>huart</i>	Handler HAL de la UART conectada al módulo WiFi.
----	--------------	--

Devuelve

WIFI_OK si el módulo queda preparado para recibir comandos.

WIFI_ERROR si la UART o el módulo no pueden inicializarse.

Precondición

`huart` debe estar inicializado por la capa HAL.

Inicializa el driver WiFi y asocia la UART del módulo.

Parámetros

in	<i>huart</i>	UART conectada al módulo WiFi.
----	--------------	--------------------------------

Valores devueltos

<i>WIFI_OK</i>	El módulo respondió y quedó en modo estación.
<i>WIFI_ERROR</i>	No se pudo comunicar o configurar el módulo.

La recepción DMA se arranca antes de probar el módulo para capturar cualquier respuesta o evento que emita el ESP. Se desactivan interrupciones de mitad y fin de recepción porque el consumo se realiza por software consultando el índice del DMA. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.8. WIFI_MQTT_Connect()

```
wifi_status_t WIFI_MQTT_Connect (
    const char * broker,
    uint16_t port)
```

Establece la conexión MQTT con el broker.

Abre la conexión MQTT contra el broker indicado.

Parámetros

in	<i>broker</i>	Nombre DNS o dirección IP del broker MQTT.
in	<i>port</i>	Puerto TCP del broker MQTT.

Devuelve

WIFI_OK si la sesión MQTT queda establecida.

WIFI_TIMEOUT si el broker no responde.

WIFI_ERROR si falla la negociación MQTT.

Establece la conexión MQTT con el broker.

Parámetros

in	<i>broker</i>	Dirección IP o nombre del broker MQTT.
in	<i>port</i>	Puerto TCP del broker MQTT.

Valores devueltos

<i>WIFI_OK</i>	Conexión MQTT establecida.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

El broker y puerto se conservan para recuperar la sesión tras pérdidas de red sin obligar al resto de tareas a conocer la secuencia AT de reconexión. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.9. WIFI_MQTT_Disconnect()

```
wifi_status_t WIFI_MQTT_Disconnect (
    void )
```

Cierra la sesión MQTT activa.

Cierra la sesión MQTT actual.

Devuelve

WIFI_OK si la desconexión se completa correctamente.

WIFI_ERROR si el módulo no confirma el cierre de sesión.

Cierra la sesión MQTT activa.

Valores devueltos

<i>WIFI_OK</i>	Sesión MQTT liberada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.10. WIFI_MQTT_Publish()

```
wifi_status_t WIFI_MQTT_Publish (
    const char * topic,
    const char * payload,
    uint8_t qos,
    uint8_t retain)
```

Publica un payload en un topic MQTT.

Parámetros

in	<i>topic</i>	Topic MQTT de destino.
in	<i>payload</i>	Contenido que se publicará.
in	<i>qos</i>	Nivel QoS solicitado para la publicación.

in	<i>retain</i>	Flag retain enviado al broker.
----	---------------	--------------------------------

Devuelve

WIFI_OK si el módulo acepta la publicación.

WIFI_NOT_CONNECTED si no existe sesión MQTT activa.

WIFI_ERROR si el comando de publicación falla.

La función permite a las tareas publicar telemetría sin conocer el comando AT concreto ni la sintaxis interna del módulo WiFi.

Parámetros

in	<i>topic</i>	Topic MQTT de destino.
in	<i>payload</i>	Cadena que se publica como contenido del mensaje.
in	<i>qos</i>	Nivel QoS solicitado al módulo WiFi.
in	<i>retain</i>	Flag retain del mensaje MQTT.

Valores devueltos

<i>WIFI_OK</i>	Publicación aceptada por el módulo.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo confirmación dentro del tiempo límite.

Si la publicación falla, el estado se degrada a WIFI_CONNECTED porque la red puede seguir activa aunque la sesión MQTT haya quedado inválida. Esta decisión permite que la capa superior intente reconstruir solo MQTT antes de reiniciar toda la conexión WiFi. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.11. WIFI_MQTT_Subscribe()

```
wifi_status_t WIFI_MQTT_Subscribe (
    const char * topic,
    uint8_t qos)
```

Suscribe el cliente MQTT a un topic.

Parámetros

in	<i>topic</i>	Topic MQTT que se desea recibir.
in	<i>qos</i>	Nivel QoS solicitado para la suscripción.

Devuelve

WIFI_OK si la suscripción se completa correctamente.

WIFI_NOT_CONNECTED si no existe sesión MQTT activa.

WIFI_ERROR si el broker o el módulo rechazan la suscripción.

8.23.2.12. WIFI_MQTT_UserConfig()

```
wifi_status_t WIFI_MQTT_UserConfig (
    const char * client_id)
```

Configura el identificador de cliente MQTT.

Parámetros

in	<i>client</i> ↔ <i>_id</i>	Identificador usado por el broker para reconocer el nodo.
----	-------------------------------	---

Devuelve

WIFI_OK si la configuración se acepta correctamente.

WIFI_ERROR si el módulo rechaza el identificador.

Precondición

client_id debe apuntar a una cadena válida terminada en cero.

Parámetros

in	<i>client</i> ↔ <i>_id</i>	Identificador usado por el cliente MQTT del módulo.
----	-------------------------------	---

Valores devueltos

<i>WIFI_OK</i>	Configuración aceptada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

El identificador se guarda para poder repetir la configuración después de una reconexión WiFi o reinicio del módulo.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.13. WIFI_Process()

```
void WIFI_Process (
    void )
```

Procesa tareas internas periódicas del módulo WiFi.

Se expone para que una tarea de FreeRTOS pueda mantener el estado del driver sin bloquear al resto del sistema.

8.23.2.14. WIFI_Reconnect()

```
wifi_status_t WIFI_Reconnect (
    void )
```

Ejecuta la secuencia de reconexión WiFi definida por el driver.

Reconstruye la conexión WiFi y MQTT usando la configuración almacenada.

Devuelve

WIFI_OK si se recupera la conectividad.

WIFI_ERROR si la reconexión falla.

Centralizar la reconexión evita que varias tareas intenten recuperar el módem con secuencias distintas.

Ejecuta la secuencia de reconexión WiFi definida por el driver.

Valores devueltos

WIFI_OK	Reconexión completada hasta MQTT.
WIFI_ERROR	No se pudo restaurar WiFi o MQTT.

La rutina distingue entre pérdida total de WiFi y pérdida solo de MQTT para no reiniciar más capas de las necesarias. Esto reduce tiempo de recuperación y evita cortes innecesarios cuando el punto de acceso sigue disponible. Gráfico de llamadas de esta función:

8.23.2.15. WIFI_Reset()

```
wifi_status_t WIFI_Reset (
    void )
```

Reinicia el módulo WiFi.

Reinicia el módulo WiFi mediante comando AT.

Devuelve

WIFI_OK si el reinicio se acepta correctamente.

WIFI_TIMEOUT si el módulo no responde.

WIFI_ERROR si la secuencia de reinicio falla.

Se usa para devolver el módem a un estado conocido cuando la comunicación AT o la conexión de red dejan de ser fiables.

Reinicia el módulo WiFi.

Valores devueltos

WIFI_OK	El módulo confirmó el reinicio con <code>ready</code> .
---------	---

<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.16. **WIFI_SendJSON()**

```
wifi_status_t WIFI_SendJSON (  
    const char * json)
```

Envía una cadena JSON por la conexión activa.

Parámetros

in	<i>json</i>	Cadena JSON terminada en cero.
----	-------------	--------------------------------

Devuelve

WIFI_OK si el envío se completa correctamente.

WIFI_NOT_CONNECTED si no existe transporte disponible.

WIFI_ERROR si el módulo rechaza los datos.

Esta función mantiene separada la generación del contenido JSON de la mecánica de transmisión por el módem.

8.23.2.17. **WIFI_SendRaw()**

```
wifi_status_t WIFI_SendRaw (  
    const char * data)
```

Envía datos sin procesar por la conexión activa.

Parámetros

in	<i>data</i>	Cadena de datos a transmitir.
----	-------------	-------------------------------

Devuelve

WIFI_OK si el módulo acepta el envío.

WIFI_NOT_CONNECTED si no existe transporte disponible.

WIFI_ERROR si se produce un fallo durante la transmisión.

8.23.2.18. WIFI_SetStationMode()

```
wifi_status_t WIFI_SetStationMode (
    void )
```

Configura el módulo WiFi en modo estación.

Devuelve

WIFI_OK si el modo queda configurado correctamente.

WIFI_ERROR si el módulo rechaza la configuración.

El modo estación es necesario para que el dispositivo se conecte a la red local y pueda alcanzar el broker MQTT o servicios HTTP externos.

Valores devueltos

WIFI_OK	Configuración aceptada.
WIFI_ERROR	Error devuelto por el módulo o fallo de transmisión.
WIFI_TIMEOUT	No hubo respuesta dentro del tiempo límite.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.23.2.19. WIFI_TCPClose()

```
wifi_status_t WIFI_TCPClose (
    void )
```

Cierra la conexión TCP activa.

Devuelve

WIFI_OK si la conexión se cierra correctamente.

WIFI_ERROR si no puede cerrarse la sesión.

8.23.2.20. WIFI_TCPConnect()

```
wifi_status_t WIFI_TCPConnect (
    const char * host,
    uint16_t port)
```

Abre una conexión TCP con un host remoto.

Parámetros

in	host	Nombre DNS o dirección IP del servidor remoto.
in	port	Puerto TCP remoto.

Devuelve

WIFI_OK si la conexión se establece correctamente.

WIFI_TIMEOUT si el host no responde.

WIFI_ERROR si el módulo rechaza la conexión.

8.23.2.21. WIFI_Test()

```
wifi_status_t WIFI_Test (
    void )
```

Comprueba la comunicación básica con el módulo WiFi.

Comprueba que el módulo WiFi responde a comandos AT.

Devuelve

WIFI_OK si el módulo responde al comando de prueba.

WIFI_TIMEOUT si no hay respuesta.

WIFI_ERROR si la respuesta recibida no es válida.

Comprueba la comunicación básica con el módulo WiFi.

Valores devueltos

WIFI_OK	El módulo respondió con OK.
WIFI_ERROR	Error devuelto por el módulo o fallo de transmisión.
WIFI_TIMEOUT	No hubo respuesta dentro del tiempo límite.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.24. wifi.h

[Ir a la documentación de este archivo.](#)

```
00001
00014
00015 #ifndef WIFI_H
00016 #define WIFI_H
00017
00018 #include "main.h"
00019 #include "wifi_types.h"
00020 #include <stdint.h>
00021
00022 #ifdef __cplusplus
00023 extern "C" {
00024 #endif
00025
00036 wifi_status_t WIFI_Init (UART_HandleTypeDef *huart);
00037
00049 wifi_status_t WIFI_Reset(void);
00050
00058 wifi_status_t WIFI_Test(void);
00059
00070 wifi_status_t WIFI_SetStationMode(void);
00071
00084 wifi_status_t WIFI_ConnectAP(const char *ssid, const char *pass);
00085
00092 wifi_status_t WIFI_DisconnectAP(void);
00093
00102 wifi_state_t WIFI_GetState(void);
00103
00116 wifi_status_t WIFI_GetIP(char *ip, uint16_t len);
00117
00129 wifi_status_t WIFI_GetRSSI(int *rssi);
00130
00141 wifi_status_t WIFI_TCPConnect(const char *host, uint16_t port);
00142
00149 wifi_status_t WIFI_TCPClose(void);
00150
```

```

00160 wifi_status_t WIFI_SendRaw(const char *data);
00161
00175 wifi_status_t WIFI_SendJSON(const char *json);
00176
00187 wifi_status_t WIFI_Reconnect(void);
00188
00196 void WIFI_Process(void);
00197
00213 wifi_status_t WIFI_HTTP_PostJSON(const char *host,
00214                                   const char *path,
00215                                   const char *json);
00216
00227 wifi_status_t WIFI_MQTT_UserConfig(const char *client_id);
00228
00239 wifi_status_t WIFI_MQTT_Connect(const char *broker, uint16_t port);
00240
00257 wifi_status_t WIFI_MQTT_Publish(const char *topic,
00258                                 const char *payload,
00259                                 uint8_t qos,
00260                                 uint8_t retain);
00261
00272 wifi_status_t WIFI_MQTT_Subscribe(const char *topic,
00273                                   uint8_t qos);
00274
00281 wifi_status_t WIFI_MQTT_Disconnect(void);
00282
00283 #ifdef __cplusplus
00284 }
00285 #endif
00286
00287 #endif /* WIFI_H */

```

8.25. Referencia del archivo Core/Inc/wifi_types.h

Tipos compartidos por la interfaz WiFi y las capas de aplicación.

```
#include <stdint.h>
```

Gráfico de dependencias incluidas en wifi_types.h: Gráfico de los archivos que directa o indirectamente incluyen a este archivo:

Enumeraciones

- enum `wifi_status_t` {
`WIFI_OK = 0` , `WIFI_ERROR` , `WIFI_TIMEOUT` , `WIFI_NOT_CONNECTED` ,
`WIFI_BUSY` }
Códigos de resultado de las operaciones WiFi.
- enum `wifi_state_t` { `WIFI_DISCONNECTED` = 0 , `WIFI_CONNECTED` , `WIFI_GOT_IP` , `WIFI_MQTT_CONNECTED`
} }
Estado lógico de la conexión WiFi/MQTT.

8.25.1. Descripción detallada

Tipos compartidos por la interfaz WiFi y las capas de aplicación.

Autor

Lorenzo

Fecha

25 abr 2026

Este archivo separa los estados y códigos de retorno de la API principal para que puedan reutilizarse en distintos módulos sin introducir dependencias circulares con el driver WiFi completo.

8.25.2. Documentación de enumeraciones

8.25.2.1. wifi_state_t

```
enum wifi_state_t
```

Estado lógico de la conexión WiFi/MQTT.

El estado se expone a la aplicación para decidir si debe reintentar conexión, publicar telemetría o esperar a que el módulo alcance una fase superior.

Valores de enumeraciones

WIFI_DISCONNECTED	El módulo no está asociado a una red WiFi.
WIFI_CONNECTED	El módulo está asociado al punto de acceso.
WIFI_GOT_IP	El módulo dispone de dirección IP.
WIFI_MQTT_CONNECTED	La sesión MQTT está establecida.

8.25.2.2. wifi_status_t

```
enum wifi_status_t
```

Códigos de resultado de las operaciones WiFi.

Permiten a las tareas de FreeRTOS diferenciar entre fallos recuperables, ausencia de conexión y recursos ocupados, manteniendo una política de recuperación desacoplada de los comandos AT concretos.

Valores de enumeraciones

WIFI_OK	Operación completada correctamente.
WIFI_ERROR	Error genérico durante la operación.
WIFI_TIMEOUT	No se recibió respuesta dentro del tiempo esperado.
WIFI_NOT_CONNECTED	La operación requiere una conexión que todavía no existe.
WIFI_BUSY	El módulo está ocupado o no puede aceptar una nueva operación.

8.26. wifi_types.h

[Ir a la documentación de este archivo.](#)

```
00001
00012 #ifndef WIFI_TYPES_H
00013 #define WIFI_TYPES_H
00014
00015 #include <stdint.h>
00016
00025 typedef enum
00026 {
00028     WIFI_OK = 0,
00029
00031     WIFI_ERROR,
00032
00034     WIFI_TIMEOUT,
00035
00037     WIFI_NOT_CONNECTED,
00038
```

```

00040     WIFI_BUSY
00041
00042 } wifi_status_t;
00043
00051 typedef enum
00052 {
00054     WIFI_DISCONNECTED = 0,
00055
00057     WIFI_CONNECTED,
00058
00060     WIFI_GOT_IP,
00061
00063     WIFI_MQTT_CONNECTED
00064 } wifi_state_t;
00066
00067 #endif /* WIFI_TYPES_H */

```

8.27. Referencia del archivo Core/Src/formatter_data.c

Conversión de paquetes de telemetría a cadenas de texto para publicación.

```

#include "formatter_data.h"
#include <stdio.h>
#include <string.h>

```

Gráfico de dependencias incluidas en formatter_data.c:

Funciones

- void [ImuData_to_json](#) (char *buffer, size_t buffer_size, [ImuDataPacket_t](#) *data)
Serializa el paquete inercial en el formato compacto de telemetría.
- void [EnvData_to_json](#) (char *buffer, size_t buffer_size, [EnvDataPacket_t](#) *data)
Serializa el paquete ambiental en el formato compacto de telemetría.
- void [GpsData_to_json](#) (char *buffer, size_t buffer_size, [GpsDataPacket_t](#) *data)
Serializa el paquete GPS en el formato compacto de telemetría.
- void [GpsDataPacket_Init](#) ([GpsDataPacket_t](#) *packet)
Inicializa un paquete GPS a un estado conocido.
- void [ImuDataPacket_Init](#) ([ImuDataPacket_t](#) *packet)
Inicializa un paquete IMU a un estado conocido.
- void [EnvDataPacket_Init](#) ([EnvDataPacket_t](#) *packet)
Inicializa un paquete ambiental a un estado conocido.

8.27.1. Descripción detallada

Conversión de paquetes de telemetría a cadenas de texto para publicación.

Autor

Lorenzo

Fecha

15 may 2026

Este módulo centraliza el formateo de los datos adquiridos antes de enviarlos por el canal de comunicaciones. Se mantiene un formato compacto basado en separadores para reducir el tamaño del payload y simplificar su transporte por MQTT/AT, evitando caracteres que anteriormente podían interferir con el encapsulado de comandos del módulo WiFi.

8.27.2. Documentación de funciones

8.27.2.1. EnvData_to_json()

```
void EnvData_to_json (
    char * buffer,
    size_t buffer_size,
    EnvDataPacket_t * data)
```

Serializa el paquete ambiental en el formato compacto de telemetría.

Serializa un paquete ambiental en formato JSON.

Parámetros

out	<i>buffer</i>	Buffer donde se escribe la cadena generada.
in	<i>buffer_size</i>	Tamaño total disponible en <i>buffer</i> .
in	<i>data</i>	Paquete ambiental con rumbo, temperatura y presión.

El formato se mantiene coherente con el resto de tramas del sistema para que el receptor pueda aplicar una única lógica de separación de campos, sin tratar este paquete como un caso especial.

Precondición

buffer debe apuntar a memoria válida con al menos *buffer_size* bytes.

data debe apuntar a un paquete ambiental válido.

Serializa el paquete ambiental en el formato compacto de telemetría.

Parámetros

out	<i>buffer</i>	Buffer de salida donde se escribe la cadena JSON.
in	<i>buffer_size</i>	Tamaño disponible en el buffer de salida.
in	<i>data</i>	Paquete ambiental que contiene la última muestra normalizada.

Precondición

buffer debe apuntar a memoria válida y *buffer_size* debe ser suficiente para almacenar el JSON generado.

data debe apuntar a un paquete inicializado.

Gráfico de llamadas a esta función:

8.27.2.2. EnvDataPacket_Init()

```
void EnvDataPacket_Init (
    EnvDataPacket_t * packet)
```

Inicializa un paquete ambiental a un estado conocido.

Inicializa un paquete ambiental con valores seguros.

Parámetros

out	<i>packet</i>	Paquete ambiental que se reinicia.
-----	---------------	------------------------------------

Se utiliza el mismo patrón que en el resto de paquetes para mantener una semántica uniforme entre productores y consumidores de datos en FreeRTOS.

Precondición

packet debe apuntar a una estructura válida.

Inicializa un paquete ambiental a un estado conocido.

Parámetros

out	<i>packet</i>	Paquete ambiental a inicializar.
-----	---------------	----------------------------------

Postcondición

El paquete queda preparado para ser usado por el pipeline antes de la primera lectura barométrica o magnética.

Gráfico de llamadas a esta función:

8.27.2.3. GpsData_to_json()

```
void GpsData_to_json (
    char * buffer,
    size_t buffer_size,
    GpsDataPacket_t * data)
```

Serializa el paquete GPS en el formato compacto de telemetría.

Serializa un paquete GPS en formato JSON.

Parámetros

out	<i>buffer</i>	Buffer donde se escribe la cadena generada.
in	<i>buffer_size</i>	Tamaño total disponible en <i>buffer</i> .
in	<i>data</i>	Paquete GPS con altitud, latitud y longitud.

Las coordenadas se limitan a cuatro decimales para contener el tamaño del mensaje y porque la precisión resultante es suficiente para la visualización general de posición usada por la interfaz del proyecto.

Precondición

`buffer` debe apuntar a memoria válida con al menos `buffer_size` bytes.
`data` debe apuntar a un paquete GPS válido.

Serializa el paquete GPS en el formato compacto de telemetría.

Parámetros

out	<i>buffer</i>	Buffer de salida donde se escribe la cadena JSON.
in	<i>buffer_size</i>	Tamaño disponible en el buffer de salida.
in	<i>data</i>	Paquete GPS que contiene la última muestra normalizada.

Precondición

`buffer` debe apuntar a memoria válida y `buffer_size` debe ser suficiente para almacenar el JSON generado.
`data` debe apuntar a un paquete inicializado.

Nota

La función está pensada para preparar payloads MQTT sin exponer al publicador los detalles internos de la estructura GPS.

Gráfico de llamadas a esta función:

8.27.2.4. GpsDataPacket_Init()

```
void GpsDataPacket_Init (
    GpsDataPacket_t * packet)
```

Inicializa un paquete GPS a un estado conocido.

Inicializa un paquete GPS con valores seguros.

Parámetros

out	<i>packet</i>	Paquete GPS que se reinicia.
-----	---------------	------------------------------

Se borra toda la estructura antes de fijar los flags de control para evitar que una lectura parcial o antigua se interprete como nueva telemetría válida.

Precondición

`packet` debe apuntar a una estructura válida.

Inicializa un paquete GPS a un estado conocido.

Parámetros

out	<i>packet</i>	Paquete GPS a inicializar.
-----	---------------	----------------------------

Postcondición

El paquete queda preparado para ser usado por el pipeline antes de que llegue la primera trama GPS válida.

Gráfico de llamadas a esta función:

8.27.2.5. ImuData_to_json()

```
void ImuData_to_json (  
    char * buffer,  
    size_t buffer_size,  
    ImuDataPacket_t * data)
```

Serializa el paquete inercial en el formato compacto de telemetría.

Serializa un paquete IMU en formato JSON.

Parámetros

out	<i>buffer</i>	Buffer donde se escribe la cadena generada.
in	<i>buffer_size</i>	Tamaño total disponible en <i>buffer</i> .
in	<i>data</i>	Paquete IMU con acelerómetro, giróscopo y magnetómetro.

El formato usa claves delimitadas con `|` y campos separados por `/` para evitar comillas dobles dentro del payload. Esta decisión reduce problemas al insertar la cadena dentro de comandos AT, donde las comillas forman parte de la sintaxis del propio comando.

Precondición

`buffer` debe apuntar a memoria válida con al menos `buffer_size` bytes.

`data` debe apuntar a un paquete IMU válido.

Nota

La función no devuelve estado; si el buffer no es suficiente, `snprintf` truncará la salida manteniendo la terminación nula cuando el tamaño lo permita.

Serializa el paquete inercial en el formato compacto de telemetría.

Parámetros

out	<i>buffer</i>	Buffer de salida donde se escribe la cadena JSON.
in	<i>buffer_size</i>	Tamaño disponible en el buffer de salida.
in	<i>data</i>	Paquete IMU que contiene la última muestra normalizada.

Precondición

`buffer` debe apuntar a memoria válida y `buffer_size` debe ser suficiente para almacenar el JSON generado.

`data` debe apuntar a un paquete inicializado.

Nota

Centralizar la conversión evita duplicar el formato de telemetría en las tareas de FreeRTOS.

Gráfico de llamadas a esta función:

8.27.2.6. ImuDataPacket_Init()

```
void ImuDataPacket_Init (  
    ImuDataPacket_t * packet)
```

Inicializa un paquete IMU a un estado conocido.

Inicializa un paquete IMU con valores seguros.

Parámetros

out	<i>packet</i>	Paquete IMU que se reinicia.
-----	---------------	------------------------------

El reinicio explícito del flag de actualización permite que las tareas consumidoras distingan entre datos recién publicados y memoria simplemente reservada o reutilizada.

Precondición

`packet` debe apuntar a una estructura válida.

Inicializa un paquete IMU a un estado conocido.

Parámetros

out	<i>packet</i>	Paquete IMU a inicializar.
-----	---------------	----------------------------

Postcondición

El paquete queda preparado para ser usado por el pipeline antes de la primera lectura del sensor.

Gráfico de llamadas a esta función:

8.28. Referencia del archivo Core/Src/freertos.c

Planificación FreeRTOS de adquisición, empaquetado y envío MQTT de telemetría.

```
#include "cmsis_os.h"
#include "sensor10DoF.h"
#include "gps.h"
#include "wifi.h"
#include "main.h"
#include "formatter_data.h"
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
```

Gráfico de dependencias incluidas en freertos.c:

defines

- `#define WIFI_TEST_SSID "Livebox7-640F"`
SSID de la red WiFi usada por el sistema durante la ejecución.
- `#define WIFI_TEST_PASS "6s42rTNbXDvd"`
Contraseña de la red WiFi usada por el sistema durante la ejecución.
- `#define SERVER_HOST "192.168.1.12"`
Dirección IP del broker MQTT local.
- `#define PAYLOAD LENGHT 128`
Tamaño reservado para cada payload JSON de telemetría.

Funciones

- `void StartImuTask (void *argument)`
Tarea FreeRTOS de adquisición del sensor IMU/10DoF.
- `void StartGpsTask (void *argument)`
Tarea FreeRTOS de inicialización y procesamiento periódico del GPS.
- `void StartPacketTask (void *argument)`
Tarea FreeRTOS de fusión lógica y serialización de telemetría.
- `void StartWifiTask (void *argument)`
Tarea FreeRTOS de conexión WiFi y publicación MQTT.
- `void MX_FREERTOS_Init (void)`
Inicializa los recursos FreeRTOS definidos por la aplicación.

Variables

- UART_HandleTypeDef [huart2](#)
UART asociada al módulo WiFi.
- UART_HandleTypeDef [huart3](#)
UART asociada al módulo GPS.
- static [Sensor10DoF_t](#) [imu_data](#)
Última lectura válida del sensor IMU/10DoF.
- static [GpsDataPacket_t](#) [gpsPacket](#)
Paquete de telemetría GPS preparado para serialización.
- static [ImuDataPacket_t](#) [imuPacket](#)
Paquete de telemetría inercial preparado para serialización.
- static [EnvDataPacket_t](#) [envPacket](#)
Paquete de telemetría ambiental preparado para serialización.
- static char [payload_gpsPacket](#) [PAYLOAD_LENGTH] = {0}
Buffer JSON del paquete GPS pendiente de envío.
- static char [payload_imuPacket](#) [PAYLOAD_LENGTH] = {0}
Buffer JSON del paquete IMU pendiente de envío.
- static char [payload_envPacket](#) [PAYLOAD_LENGTH] = {0}
Buffer JSON del paquete ambiental pendiente de envío.
- static uint8_t [imuDataUpdated](#) = 0
Indica que existe una nueva muestra IMU disponible para empaquetado.
- static uint8_t [gpsDataUpdated](#) = 0
Indica que existe una nueva muestra GPS disponible para empaquetado.
- static uint8_t [imuPacketToUpdate](#) = 0
Solicita reconstruir los paquetes derivados de la última muestra IMU.
- static uint8_t [gpsPacketToUpdate](#) = 0
Solicita reconstruir el paquete derivado de la última muestra GPS.
- static uint8_t [imuPacketToSend](#) = 0
Indica que el payload IMU/ambiental está listo para publicación MQTT.
- static uint8_t [gpsPacketToSend](#) = 0
Indica que el payload GPS está listo para publicación MQTT.
- volatile [wifi_status_t](#) [wifi_st](#)
Último estado reportado por el driver WiFi.
- volatile [gps_status_t](#) [gps_st](#)
Último estado reportado por el driver GPS.
- osMutexId_t [imuMutex](#)
Mutex de protección para el buffer compartido de datos IMU.
- osMutexId_t [gpsMutex](#)
Mutex de protección para el buffer compartido de datos GPS.
- osMutexId_t [packetMutex](#)
Mutex de protección para payloads JSON y flags de publicación.
- osThreadId_t [imuTaskHandle](#)
Handle de la tarea de adquisición IMU.
- osThreadId_t [gpsTaskHandle](#)
Handle de la tarea de adquisición GPS.
- osThreadId_t [packetTaskHandle](#)
Handle de la tarea de construcción de paquetes de telemetría.
- osThreadId_t [wifiTaskHandle](#)
Handle de la tarea de conexión WiFi y publicación MQTT.
- const osThreadAttr_t [imuTask_attributes](#)

Atributos RTOS de la tarea de adquisición IMU.

- const osThreadAttr_t [wifiTask_attributes](#)

Atributos RTOS de la tarea WiFi/MQTT.

- const osThreadAttr_t [gpsTask_attributes](#)

Atributos RTOS de la tarea GPS.

- const osThreadAttr_t [packetTask_attributes](#)

Atributos RTOS de la tarea de empaquetado.

8.28.1. Descripción detallada

Planificación FreeRTOS de adquisición, empaquetado y envío MQTT de telemetría.

Este módulo organiza el firmware como una arquitectura desacoplada tipo "data pipeline": cada tarea tiene una responsabilidad única y se comunica con el resto mediante buffers globales protegidos por mutex y flags de estado.

La separación entre adquisición, construcción de paquetes y publicación MQTT evita que una operación de red bloquee directamente la lectura de sensores. Esta decisión mejora la robustez temporal del sistema en un entorno embebido donde las comunicaciones WiFi pueden introducir retardos variables.

Autor

Lorenzo

Fecha

14 May 2026

8.28.2. Documentación de «define»

8.28.2.1. PAYLOAD_LENGTH

```
#define PAYLOAD_LENGTH 128
```

Tamaño reservado para cada payload JSON de telemetría.

El valor se dimensiona para mensajes MQTT compactos, reduciendo consumo de RAM sin recurrir a reserva dinámica de memoria.

8.28.2.2. SERVER_HOST

```
#define SERVER_HOST "192.168.1.12"
```

Dirección IP del broker MQTT local.

El uso de dirección IP directa evita depender de resolución DNS en el módulo WiFi y reduce una posible fuente de fallo durante las pruebas locales.

8.28.2.3. WIFI_TEST_PASS

```
#define WIFI_TEST_PASS "6s42rTNbXDvd"
```

Contraseña de la red WiFi usada por el sistema durante la ejecución.

8.28.2.4. WIFI_TEST_SSID

```
#define WIFI_TEST_SSID "Livebox7-640F"
```

SSID de la red WiFi usada por el sistema durante la ejecución.

Se mantiene como macro para simplificar la configuración de conexión en un entorno embebido sin almacenamiento persistente de credenciales.

8.28.3. Documentación de funciones

8.28.3.1. MX_FREERTOS_Init()

```
void MX_FREERTOS_Init (
    void )
```

Inicializa los recursos FreeRTOS definidos por la aplicación.

Crea los mutex compartidos y lanza las tareas que forman la cadena de adquisición, empaquetado y transmisión. La función conserva el patrón esperado por STM32CubeMX para integrarse con la inicialización global del proyecto. Gráfico de llamadas de esta función:

8.28.3.2. StartGpsTask()

```
void StartGpsTask (
    void * argument)
```

Tarea FreeRTOS de inicialización y procesamiento periódico del GPS.

Parámetros

<i>argument</i>	Argumento genérico proporcionado por CMSIS-RTOS. No se utiliza.
-----------------	---

Reintenta la inicialización del GPS hasta obtener un estado válido. Este patrón evita que un arranque lento del módulo GPS impida la ejecución del resto del firmware.

Durante la ejecución periódica, la tarea procesa la entrada NMEA/UART mediante el driver GPS y marca la muestra como disponible para la etapa de empaquetado. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.28.3.3. StartImuTask()

```
void StartImuTask (
    void * argument)
```

Tarea FreeRTOS de adquisición del sensor IMU/10DoF.

Parámetros

<i>argument</i>	Argumento genérico proporcionado por CMSIS-RTOS. No se utiliza.
-----------------	---

Inicializa el sensor y mantiene actualizada la última muestra disponible. Si una lectura falla, se solicita un reset del sensor para recuperar el bus o el estado interno del driver sin detener el resto del sistema.

La tarea solo mantiene bloqueado el mutex durante la sección crítica de lectura y actualización del buffer compartido. Esto reduce la contención con la tarea de empaquetado. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.28.3.4. StartPacketTask()

```
void StartPacketTask (
    void * argument)
```

Tarea FreeRTOS de fusión lógica y serialización de telemetría.

Parámetros

<i>argument</i>	Argumento genérico proporcionado por CMSIS-RTOS. No se utiliza.
-----------------	---

Esta tarea actúa como etapa intermedia entre adquisición y comunicación. Copia muestras recientes bajo mutex, libera rápidamente los recursos compartidos y construye los paquetes fuera de las secciones críticas siempre que sea posible.

El diseño evita que la generación de JSON o la publicación MQTT accedan directamente a las estructuras internas de los drivers, reduciendo acoplamiento y facilitando cambios de formato en la telemetría. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.28.3.5. StartWifiTask()

```
void StartWifiTask (
    void * argument)
```

Tarea FreeRTOS de conexión WiFi y publicación MQTT.

Parámetros

<i>argument</i>	Argumento genérico proporcionado por CMSIS-RTOS. No se utiliza.
-----------------	---

Inicializa el módulo WiFi, garantiza la conexión al punto de acceso y configura una sesión MQTT antes de entrar en el bucle de publicación.

Los payloads se copian a buffers temporales antes de llamar al driver WiFi para minimizar el tiempo de bloqueo de [packetMutex](#). Esta decisión evita que una publicación lenta impida a la tarea de empaquetado actualizar nuevos mensajes. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.28.4. Documentación de variables

8.28.4.1. envPacket

```
EnvDataPacket_t envPacket [static]
```

Paquete de telemetría ambiental preparado para serialización.

8.28.4.2. gps_st

```
volatile gps_status_t gps_st
```

Último estado reportado por el driver GPS.

Se conserva global para facilitar diagnóstico y reutilización del estado de inicialización del módulo GPS.

8.28.4.3. gpsDataUpdated

```
uint8_t gpsDataUpdated = 0 [static]
```

Indica que existe una nueva muestra GPS disponible para empaquetado.

8.28.4.4. gpsMutex

```
osMutexId_t gpsMutex
```

Mutex de protección para el buffer compartido de datos GPS.

8.28.4.5. gpsPacket

```
GpsDataPacket_t gpsPacket [static]
```

Paquete de telemetría GPS preparado para serialización.

8.28.4.6. gpsPacketToSend

```
uint8_t gpsPacketToSend = 0 [static]
```

Indica que el payload GPS está listo para publicación MQTT.

8.28.4.7. gpsPacketToUpdate

```
uint8_t gpsPacketToUpdate = 0 [static]
```

Solicita reconstruir el paquete derivado de la última muestra GPS.

8.28.4.8. `gpsTask_attributes`

```
const osThreadAttr_t gpsTask_attributes
```

Valor inicial:

```
    =  
{  
    .name = "gpsTask",  
    .stack_size = 512 * 4,  
    .priority = (osPriority_t) osPriorityBelowNormal  
}
```

Atributos RTOS de la tarea GPS.

El GPS se ejecuta con menor prioridad porque su frecuencia útil de actualización es inferior a la de la IMU y no requiere una cadencia tan estricta.

8.28.4.9. `gpsTaskHandle`

```
osThreadId_t gpsTaskHandle
```

Handle de la tarea de adquisición GPS.

8.28.4.10. `huart2`

```
UART_HandleTypeDef huart2 [extern]
```

UART asociada al módulo WiFi.

Se declara como externa porque la inicialización física del periférico UART se realiza en el código generado por STM32CubeMX. Este módulo solo necesita la referencia para inicializar y operar el driver WiFi.

8.28.4.11. `huart3`

```
UART_HandleTypeDef huart3 [extern]
```

UART asociada al módulo GPS.

Se reutiliza el manejador generado por STM32CubeMX para mantener separada la configuración de bajo nivel del periférico y la lógica de adquisición GPS.

8.28.4.12. `imu_data`

```
Sensor10DoF_t imu_data [static]
```

Última lectura válida del sensor IMU/10DoF.

Actúa como buffer compartido entre la tarea de adquisición IMU y la tarea de empaquetado. Su acceso se protege mediante [imuMutex](#).

8.28.4.13. imuDataUpdated

```
uint8_t imuDataUpdated = 0 [static]
```

Indica que existe una nueva muestra IMU disponible para empaquetado.

Permite desacoplar la frecuencia de lectura del sensor de la frecuencia de construcción de paquetes sin bloquear la tarea de adquisición más tiempo del necesario.

8.28.4.14. imuMutex

```
osMutexId_t imuMutex
```

Mutex de protección para el buffer compartido de datos IMU.

8.28.4.15. imuPacket

```
ImuDataPacket_t imuPacket [static]
```

Paquete de telemetría inercial preparado para serialización.

8.28.4.16. imuPacketToSend

```
uint8_t imuPacketToSend = 0 [static]
```

Indica que el payload IMU/ambiental está listo para publicación MQTT.

8.28.4.17. imuPacketToUpdate

```
uint8_t imuPacketToUpdate = 0 [static]
```

Solicita reconstruir los paquetes derivados de la última muestra IMU.

8.28.4.18. imuTask_attributes

```
const osThreadAttr_t imuTask_attributes
```

Valor inicial:

```
=  
{  
    .name = "imuTask",  
    .stack_size = 1024 * 4,  
    .priority = (osPriority_t) osPriorityNormal  
}
```

Atributos RTOS de la tarea de adquisición IMU.

La pila se dimensiona con margen para las llamadas al driver 10DoF y la prioridad normal permite que tareas de comunicación puedan ejecutarse sin bloquear indefinidamente la adquisición.

8.28.4.19. imuTaskHandle

```
osThreadId_t imuTaskHandle
```

Handle de la tarea de adquisición IMU.

8.28.4.20. packetMutex

```
osMutexId_t packetMutex
```

Mutex de protección para payloads JSON y flags de publicación.

8.28.4.21. packetTask_attributes

```
const osThreadAttr_t packetTask_attributes
```

Valor inicial:

```
=
{
    .name = "packetTask",
    .stack_size = 1024 * 4,
    .priority = (osPriority_t) osPriorityAboveNormal
}
```

Atributos RTOS de la tarea de empaquetado.

Su prioridad superior permite transformar datos recientes en payloads JSON con baja latencia antes de que la tarea WiFi los publique.

8.28.4.22. packetTaskHandle

```
osThreadId_t packetTaskHandle
```

Handle de la tarea de construcción de paquetes de telemetría.

8.28.4.23. payload_envPacket

```
char payload_envPacket[PAYLOAD_LENGTH] = {0} [static]
```

Buffer JSON del paquete ambiental pendiente de envío.

8.28.4.24. payload_gpsPacket

```
char payload_gpsPacket[PAYLOAD_LENGTH] = {0} [static]
```

Buffer JSON del paquete GPS pendiente de envío.

Se usa un buffer estático para evitar fragmentación de memoria y mantener un comportamiento determinista en FreeRTOS.

8.28.4.25. payload_imuPacket

```
char payload_imuPacket[PAYLOAD_LENGTH] = {0} [static]
```

Buffer JSON del paquete IMU pendiente de envío.

El buffer queda desacoplado del paquete binario para que la tarea WiFi pueda copiar una versión estable antes de publicar por MQTT.

8.28.4.26. wifi_st

```
volatile wifi_status_t wifi_st
```

Último estado reportado por el driver WiFi.

Se declara como volatile porque su valor representa estado externo del módulo de comunicaciones y se consulta de forma recurrente desde la tarea WiFi.

8.28.4.27. wifiTask_attributes

```
const osThreadAttr_t wifiTask_attributes
```

Valor inicial:

```
=
{
    .name = "wifiTask",
    .stack_size = 1024 * 4,
    .priority = (osPriority_t) osPriorityAboveNormal
}
```

Atributos RTOS de la tarea WiFi/MQTT.

La prioridad superior reduce el riesgo de acumulación de payloads cuando la publicación MQTT necesita atenderse con rapidez frente a tareas de menor criticidad temporal.

8.28.4.28. wifiTaskHandle

```
osThreadId_t wifiTaskHandle
```

Handle de la tarea de conexión WiFi y publicación MQTT.

8.29. Referencia del archivo Core/Src/gps.c

Driver no bloqueante para recepción y procesamiento de tramas NMEA GPS.

```
#include "gps.h"
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
```

Gráfico de dependencias incluidas en gps.c:

Funciones

- static void [gps_rx_push](#) (uint8_t byte)
Inserta un byte recibido en el búfer circular.
- static bool [gps_rx_pop](#) (uint8_t *byte)
Extrae un byte pendiente del búfer circular.
- static float [gps_nmea_to_decimal](#) (const char *raw, char dir)
Convierte una coordenada NMEA en grados decimales.
- static void [gps_parse_gga](#) (char *nmea)
Decodifica una sentencia GGA y actualiza posición, fix, satélites y altitud.
- static void [gps_parse_rmc](#) (char *nmea)
Decodifica una sentencia RMC y actualiza la velocidad.
- [gps_status_t](#) [GPS_Init](#) (UART_HandleTypeDef *huart)
Inicializa el driver GPS y arranca la recepción UART por interrupción.
- void [GPS_UARTCallback](#) (UART_HandleTypeDef *huart)
Callback que debe llamarse al completarse la recepción UART del GPS.
- void [GPS_Process](#) (void)
Procesa los bytes pendientes y actualiza los datos GPS disponibles.
- [gps_data_t](#) [GPS_GetData](#) (void)
Devuelve una copia del último estado GPS decodificado.
- bool [GPS_DataValid](#) (void)
Indica si ya se ha decodificado una sentencia GPS válida.
- void [GPS_RestartReceive](#) (void)
Rearma manualmente la recepción UART del GPS.

Variables

- static UART_HandleTypeDef * [gps_uart](#)
UART asociada al módulo GPS inicializado.
- static uint8_t [uart_rx_byte](#)
Byte temporal usado por la recepción UART por interrupción.
- static uint8_t [gps_rx_buffer](#) [[GPS_RX_BUFFER_SIZE](#)]
Búfer circular de recepción para no perder bytes entre llamadas a [GPS_Process\(\)](#).
- static volatile uint16_t [rx_head](#) = 0
Índice de escritura del búfer circular, actualizado desde la ISR.
- static volatile uint16_t [rx_tail](#) = 0
Índice de lectura del búfer circular, consumido por el parser.
- static char [sentence](#) [[GPS_SENTENCE_MAX_LENGTH](#)]
Sentencia NMEA en construcción antes de identificar su tipo.
- static uint16_t [sentence_index](#) = 0
Posición actual dentro de [sentence](#).
- static [gps_data_t](#) [gps_data](#)
Último estado GPS decodificado y disponible para el resto del sistema.

8.29.1. Descripción detallada

Driver no bloqueante para recepción y procesado de tramas NMEA GPS.

Autor

Lorenzo

Fecha

13 may 2026

El módulo recibe bytes por interrupción UART y los desacopla del parser usando un búfer circular. Esta arquitectura evita realizar trabajo pesado dentro de la ISR y permite que el procesado de sentencias NMEA se ejecute desde una tarea o bucle principal con menor impacto temporal.

8.29.2. Documentación de funciones

8.29.2.1. GPS_DataValid()

```
bool GPS_DataValid (
    void )
```

Indica si ya se ha decodificado una sentencia GPS válida.

Indica si la última muestra GPS disponible es válida.

Valores devueltos

<i>true</i>	Hay datos GPS procesados.
<i>false</i>	Aún no hay datos GPS válidos disponibles.

Indica si ya se ha decodificado una sentencia GPS válida.

Valores devueltos

<i>true</i>	Existe una muestra válida con fix aceptable.
<i>false</i>	La muestra actual no debe usarse como posición fiable.

8.29.2.2. GPS_GetData()

```
gps_data_t GPS_GetData (
    void )
```

Devuelve una copia del último estado GPS decodificado.

Devuelve una copia de la última muestra GPS procesada.

Devuelve

Estructura con los últimos datos GPS disponibles.

Se devuelve por valor para evitar que otros módulos modifiquen directamente el estado interno del driver.

Devuelve una copia del último estado GPS decodificado.

Devuelve

Estructura `gps_data_t` con el último estado conocido del receptor.

Nota

La devolución por valor evita que el consumidor modifique accidentalmente el estado interno del driver.

Gráfico de llamadas a esta función:

8.29.2.3. GPS_Init()

```
gps_status_t GPS_Init (
    UART_HandleTypeDef * huart)
```

Inicializa el driver GPS y arranca la recepción UART por interrupción.

Inicializa el módulo GPS y asocia la UART usada por el receptor.

Parámetros

in	huart	UART conectada al módulo GPS.
----	-------	-------------------------------

Valores devueltos

<i>GPS_OK</i>	La recepción inicial se configuró correctamente.
<i>GPS_ERROR</i>	No se pudo iniciar la recepción por interrupción.

La recepción se inicia byte a byte para poder reconstruir sentencias NMEA sin depender de DMA ni de longitudes fijas, ya que el GPS emite tramas ASCII de tamaño variable.

Inicializa el driver GPS y arranca la recepción UART por interrupción.

Parámetros

in	<i>huart</i>	Handler HAL de la UART conectada al módulo GPS.
----	--------------	---

Devuelve

GPS_OK si la inicialización se completa correctamente.

GPS_ERROR si no es posible preparar la recepción.

Precondición

huart debe estar inicializado por la capa HAL antes de llamar a esta función.

Gráfico de llamadas a esta función:

8.29.2.4. gps_nmea_to_decimal()

```
float gps_nmea_to_decimal (
    const char * raw,
    char dir) [static]
```

Convierte una coordenada NMEA en grados decimales.

Parámetros

in	<i>raw</i>	Coordenada en formato NMEA dddmm.mmmm o dddmm.mmmm.
in	<i>dir</i>	Hemisferio asociado a la coordenada (N, S, E o W).

Devuelve

Coordenada en grados decimales con signo geográfico.

El GPS entrega latitud y longitud en grados/minutos. La conversión se realiza aquí para que el resto del sistema trabaje siempre con un formato directo de representar en mapas o interfaces gráficas. Gráfico de llamadas a esta función:

8.29.2.5. gps_parse_gga()

```
void gps_parse_gga (
    char * nmea) [static]
```

Decodifica una sentencia GGA y actualiza posición, fix, satélites y altitud.

Parámetros

in, out	<i>nmea</i>	Sentencia NMEA modificable.
---------	-------------	-----------------------------

Se usa GGA porque concentra la información de calidad de fix y altitud. La sentencia se tokeniza in situ para evitar buffers adicionales en RAM.

Atención

`strtok()` modifica la cadena recibida; no se debe reutilizar la sentencia original después de llamar a esta función.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.29.2.6. `gps_parse_rmc()`

```
void gps_parse_rmc (
    char * nmea) [static]
```

Decodifica una sentencia RMC y actualiza la velocidad.

Parámetros

<code>in, out</code>	<code>nmea</code>	Sentencia NMEA modificable.
----------------------	-------------------	-----------------------------

La velocidad se toma de RMC porque es el campo NMEA estándar para movimiento sobre tierra. Se guarda en nudos y km/h para no repetir conversiones en las capas superiores.

Atención

`strtok()` modifica la cadena recibida.

Gráfico de llamadas a esta función:

8.29.2.7. `GPS_Process()`

```
void GPS_Process (
    void )
```

Procesa los bytes pendientes y actualiza los datos GPS disponibles.

Procesa las tramas GPS acumuladas por la recepción UART.

Esta función debe ejecutarse periódicamente desde contexto de tarea o bucle principal. Separar el parser de la ISR evita bloquear otras interrupciones y reduce el riesgo de perder datos en sistemas con FreeRTOS.

Procesa los bytes pendientes y actualiza los datos GPS disponibles.

Debe ejecutarse desde contexto de tarea o bucle principal para convertir las sentencias recibidas en una muestra `gps_data_t` consultable por la aplicación. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.29.2.8. `GPS_RestartReceive()`

```
void GPS_RestartReceive (
    void )
```

Rearma manualmente la recepción UART del GPS.

Reinicia la recepción UART del GPS.

Esta función permite recuperar la escucha si una condición externa cancela o interrumpe la recepción por interrupción. Se reutiliza el byte interno para mantener encapsulado el estado de recepción.

Rearma manualmente la recepción UART del GPS.

Se expone para recuperar la recepción tras errores o condiciones de parada sin reinicializar toda la aplicación.

8.29.2.9. gps_rx_pop()

```
bool gps_rx_pop (
    uint8_t * byte) [static]
```

Extrae un byte pendiente del búfer circular.

Parámetros

out	byte	Puntero donde se almacena el byte leído.
-----	------	--

Valores devueltos

true	Se ha extraído un byte válido.
false	No había datos pendientes.

La lectura se realiza fuera de la interrupción para mantener la ISR limitada al mínimo trabajo imprescindible. Gráfico de llamadas a esta función:

8.29.2.10. gps_rx_push()

```
void gps_rx_push (
    uint8_t byte) [static]
```

Inserta un byte recibido en el búfer circular.

Parámetros

in	byte	Byte procedente de la UART GPS.
----	------	---------------------------------

Si el búfer está lleno se descarta el byte nuevo para proteger la coherencia de los índices. Esta política prioriza que el parser no lea memoria corrupta frente a conservar una sentencia incompleta. Gráfico de llamadas a esta función:

8.29.2.11. GPS_UARTCallback()

```
void GPS_UARTCallback (
    UART_HandleTypeDef * huart)
```

Callback que debe llamarse al completarse la recepción UART del GPS.

Callback que debe invocarse desde la interrupción de recepción UART.

Parámetros

in	huart	UART que ha generado la interrupción.
----	-------	---------------------------------------

El callback filtra la UART recibida para que el driver ignore interrupciones de otros periféricos. Tras guardar el byte, rearma inmediatamente la escucha para mantener una recepción continua.

Callback que debe llamarse al completarse la recepción UART del GPS.

Parámetros

in	huart	Handler HAL que ha generado el evento de recepción.
----	-------	---

La callback mantiene la captura de caracteres separada del procesamiento completo de sentencias para reducir el trabajo realizado en contexto de interrupción. Gráfico de llamadas de esta función:

8.30. Referencia del archivo Core/Src/sensor10DoF.c

Driver de adquisición para IMU 10DoF mediante bus I2C.

```
#include "sensor10DoF.h"
#include "i2c.h"
#include "gpio.h"
#include <math.h>
#include <string.h>
```

Gráfico de dependencias incluidas en sensor10DoF.c:

defines

- #define ACC_ADDR (0x19 << 1)
- #define MAG_ADDR (0x1E << 1)
- #define GYR_ADDR (0x6B << 1)
- #define BMP_ADDR (0x77 << 1)
- #define G_CONST 9.80665f
- #define ACC_SENS_2G 0.001f
- #define GYRO_SENS_250DPS 0.00875f
- #define MAG_XY_LSB_GAUSS 1100.0f
- #define MAG_Z_LSB_GAUSS 980.0f
- #define GAUSS_TO_UT 100.0f
- #define BMP180_OSS 3
- #define BMP180_SEA_HPA 1013.25f

Funciones

- static HAL_StatusTypeDef I2C_Read (uint16_t dev, uint8_t reg, uint8_t *buf, uint16_t len)
Lee registros consecutivos de un dispositivo I2C.
- static HAL_StatusTypeDef I2C_Write (uint16_t dev, uint8_t reg, uint8_t val)
Escribe un byte en un registro de un dispositivo I2C.
- void Sensor10DoF_I2CRecover (void)
Intenta recuperar el bus I2C generando pulsos manuales en SCL.
- static SensorStatus_t ACC_Init (void)
Configura el acelerómetro para lectura continua en rango ± 2 g.
- static SensorStatus_t MAG_Init (void)
Configura el magnetómetro para medición continua.
- static SensorStatus_t GYR_Init (void)
Configura el giróscopo para habilitar sus tres ejes.
- static SensorStatus_t BMP180_ReadCalibration (void)

- Lee y almacena los coeficientes de calibración del BMP180.*
- static [SensorStatus_t BMP180_ReadUT](#) (int32_t *UT)
 - Lee la temperatura cruda del BMP180.*
- static [SensorStatus_t BMP180_ReadUP](#) (int32_t *UP)
 - Lee la presión cruda del BMP180.*
- static [SensorStatus_t Sensor10DoF_CalibrateGyro_Internal](#) (void)
 - Calcula el offset estático del giróscopo.*
- [SensorStatus_t Sensor10DoF_Init](#) (void)
 - Inicializa todos los sensores del módulo 10DoF.*
- [SensorStatus_t Sensor10DoF_Read](#) ([Sensor10DoF_t](#) *d)
 - Lee todos los sensores y rellena una muestra 10DoF completa.*
- void [Sensor10DoF_Reset](#) (void)
 - Recupera el bus I2C y reinicializa el módulo 10DoF.*
- void [Sensor10DoF_Calibrate](#) (void)
 - Ejecuta una nueva calibración del giróscopo.*

Variables

- I2C_HandleTypeDef [hi2c1](#)
 - Handle I2C generado por STM32CubeMX y compartido por los sensores 10DoF.*
- static int16_t [AC1](#)
 - Coeficientes de calibración leídos de la EEPROM interna del BMP180.*
- static int16_t [AC2](#)
- static int16_t [AC3](#)
- static int16_t [B1](#)
- static int16_t [B2](#)
- static int16_t [MB](#)
- static int16_t [MC](#)
- static int16_t [MD](#)
- static uint16_t [AC4](#)
 - Coeficientes sin signo de calibración del BMP180.*
- static uint16_t [AC5](#)
- static uint16_t [AC6](#)
- static float [gyro_x_offset](#) = 0.0f
 - Offset estático del eje X del giróscopo medido durante la calibración.*
- static float [gyro_y_offset](#) = 0.0f
 - Offset estático del eje Y del giróscopo medido durante la calibración.*
- static float [gyro_z_offset](#) = 0.0f
 - Offset estático del eje Z del giróscopo medido durante la calibración.*

8.30.1. Descripción detallada

Driver de adquisición para IMU 10DoF mediante bus I2C.

Autor

Lorenzo

Fecha

27 may 2026

Este módulo agrupa la inicialización, recuperación y lectura de acelerómetro, magnetómetro, giróscopo y barómetro. La lectura se entrega en unidades físicas para que las capas superiores no dependan de registros, escalas ni coeficientes internos de cada sensor.

8.30.2. Documentación de funciones

8.30.2.1. ACC_Init()

```
SensorStatus_t ACC_Init (
    void ) [static]
```

Configura el acelerómetro para lectura continua en rango ± 2 g.

Valores devueltos

<i>SENSOR_OK</i>	Configuración aplicada.
<i>SENSOR_ERROR</i>	Falló alguna escritura I2C.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.2. BMP180_ReadCalibration()

```
SensorStatus_t BMP180_ReadCalibration (
    void ) [static]
```

Lee y almacena los coeficientes de calibración del BMP180.

Valores devueltos

<i>SENSOR_OK</i>	Coeficientes cargados correctamente.
<i>SENSOR_ERROR</i>	No se pudo leer la EEPROM de calibración.

Los coeficientes son necesarios para compensar temperatura y presión. Se leen una sola vez durante la inicialización porque son constantes de fábrica. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.3. BMP180_ReadUP()

```
SensorStatus_t BMP180_ReadUP (
    int32_t * UP) [static]
```

Lee la presión cruda del BMP180.

Parámetros

out	<i>UP</i>	Presión sin compensar ajustada por el sobremuestreo configurado.
-----	-----------	--

Valores devueltos

<i>SENSOR_OK</i>	Lectura completada.
------------------	---------------------

<i>SENSOR_ERROR</i>	Falló la transacción I2C.
---------------------	---------------------------

El tiempo de espera depende del sobremuestreo. Con OSS alto se reduce el ruido de presión a costa de aumentar la latencia de conversión. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.4. BMP180_ReadUT()

```
SensorStatus_t BMP180_ReadUT (
    int32_t * UT) [static]
```

Lee la temperatura cruda del BMP180.

Parámetros

out	<i>UT</i>	Temperatura sin compensar.
-----	-----------	----------------------------

Valores devueltos

<i>SENSOR_OK</i>	Lectura completada.
<i>SENSOR_ERROR</i>	Falló la transacción I2C.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.5. GYR_Init()

```
SensorStatus_t GYR_Init (
    void ) [static]
```

Configura el giróscopo para habilitar sus tres ejes.

Valores devueltos

<i>SENSOR_OK</i>	Configuración aplicada.
<i>SENSOR_ERROR</i>	Falló alguna escritura I2C.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.6. I2C_Read()

```
HAL_StatusTypeDef I2C_Read (
    uint16_t dev,
    uint8_t reg,
    uint8_t * buf,
    uint16_t len) [static]
```

Lee registros consecutivos de un dispositivo I2C.

Parámetros

in	<i>dev</i>	Dirección I2C desplazada del dispositivo.
in	<i>reg</i>	Registro inicial de lectura.
out	<i>buf</i>	Buffer de destino.
in	<i>len</i>	Número de bytes a leer.

Devuelve

Estado HAL de la transacción.

La función encapsula la HAL para mantener en un único punto el timeout y el tamaño de dirección de memoria usado por todos los sensores del módulo. Gráfico de llamadas a esta función:

8.30.2.7. I2C_Write()

```
HAL_StatusTypeDef I2C_Write (
    uint16_t dev,
    uint8_t reg,
    uint8_t val) [static]
```

Escribe un byte en un registro de un dispositivo I2C.

Parámetros

in	<i>dev</i>	Dirección I2C desplazada del dispositivo.
in	<i>reg</i>	Registro de destino.
in	<i>val</i>	Valor que se escribe.

Devuelve

Estado HAL de la transacción.

Centralizar las escrituras evita duplicar parámetros de HAL y facilita que la secuencia de inicialización sea legible a nivel de registros relevantes. Gráfico de llamadas a esta función:

8.30.2.8. MAG_Init()

```
SensorStatus_t MAG_Init (
    void ) [static]
```

Configura el magnetómetro para medición continua.

Valores devueltos

<i>SENSOR_OK</i>	Configuración aplicada.
------------------	-------------------------

<i>SENSOR_ERROR</i>	Falló alguna escritura I2C.
---------------------	-----------------------------

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.9. **Sensor10DoF_Calibrate()**

```
void Sensor10DoF_Calibrate (
    void )
```

Ejecuta una nueva calibración del giróscopo.

Ejecuta la calibración del módulo 10DoF.

Permite recalcular los offsets si cambia la temperatura, la alimentación o se observa deriva tras el arranque. El sensor debe permanecer inmóvil durante la llamada para que el promedio represente únicamente el sesgo estático.

Ejecuta una nueva calibración del giróscopo.

La calibración reduce offsets sistemáticos, especialmente en el giróscopo, y permite que las magnitudes publicadas sean estables cuando el sistema está en reposo. Gráfico de llamadas de esta función:

8.30.2.10. **Sensor10DoF_CalibrateGyro_Internal()**

```
SensorStatus_t Sensor10DoF_CalibrateGyro_Internal (
    void ) [static]
```

Calcula el offset estático del giróscopo.

Valores devueltos

<i>SENSOR_OK</i>	Calibración completada.
<i>SENSOR_ERROR</i>	No se pudieron leer las muestras necesarias.

El promedio de varias muestras en reposo compensa el sesgo observado en los ejes del giróscopo. Esta corrección es necesaria para que las lecturas en quietud se aproximen a cero y no propaguen deriva a la telemetría.

Precondición

El sensor debe permanecer inmóvil durante la calibración.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.11. **Sensor10DoF_I2CRecover()**

```
void Sensor10DoF_I2CRecover (
    void )
```

Intenta recuperar el bus I2C generando pulsos manuales en SCL.

Ejecuta una recuperación manual del bus I2C usado por el módulo.

Algunos esclavos I2C pueden dejar SDA retenida tras un reset o una transferencia interrumpida. La recuperación conmuta temporalmente SCL/SDA como GPIO para liberar el bus antes de reinicializar el periférico I2C.

Intenta recuperar el bus I2C generando pulsos manuales en SCL.

Esta operación está pensada para liberar el bus cuando un dispositivo mantiene líneas bloqueadas tras una transacción incompleta o un reinicio inesperado. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.12. Sensor10DoF_Init()

```
SensorStatus_t Sensor10DoF_Init (
    void )
```

Inicializa todos los sensores del módulo 10DoF.

Inicializa los sensores y prepara el bus de comunicación.

Valores devueltos

<i>SENSOR_OK</i>	Todos los dispositivos respondieron y quedaron configurados.
<i>SENSOR_ERROR</i>	Algún dispositivo no respondió, no pudo configurarse o falló la calibración.

La función verifica primero la presencia de todos los esclavos I2C antes de configurar registros. Esto permite detectar fallos de cableado o bloqueo de bus antes de producir lecturas parciales incoherentes.

Inicializa todos los sensores del módulo 10DoF.

Devuelve

SENSOR_OK si todos los dispositivos quedan listos para lectura.

SENSOR_ERROR si algún dispositivo no responde o no queda configurado.

SENSOR_TIMEOUT si el bus no responde dentro del tiempo esperado.

Debe llamarse antes de solicitar lecturas para asegurar que las escalas, registros y compensaciones iniciales están configuradas de forma coherente. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.13. Sensor10DoF_Read()

```
SensorStatus_t Sensor10DoF_Read (
    Sensor10DoF_t * data)
```

Lee todos los sensores y rellena una muestra 10DoF completa.

Lee una muestra completa del módulo 10DoF.

Parámetros

out	<i>d</i>	Estructura de destino con aceleración, orientación, campo magnético, velocidad angular, temperatura, presión y altitud.
-----	----------	---

Valores devueltos

<i>SENSOR_OK</i>	Muestra actualizada correctamente.
<i>SENSOR_ERROR</i>	Error de argumento, comunicación o compensación inválida.

La función entrega magnitudes ya convertidas para aislar al resto del proyecto de escalas de registros. También calcula roll, pitch y heading compensado por inclinación para que la interfaz pueda consumir directamente valores físicos.

Precondición

`Sensor10DoF_Init()` debe haberse ejecutado correctamente.

Lee todos los sensores y rellena una muestra 10DoF completa.

Parámetros

out	data	Estructura donde se almacenan las magnitudes convertidas.
-----	------	---

Devuelve

SENSOR_OK si la lectura se completa correctamente.

SENSOR_ERROR si alguna magnitud no puede actualizarse.

SENSOR_TIMEOUT si un dispositivo no responde a tiempo.

SENSOR_BUSY si el módulo no puede atender la lectura en ese instante.

Precondición

`data` debe apuntar a una estructura válida.

La función entrega una muestra agregada para reducir el acoplamiento entre las tareas consumidoras y los distintos sensores físicos que forman el módulo. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.2.14. Sensor10DoF_Reset()

```
void Sensor10DoF_Reset (
    void )
```

Recupera el bus I2C y reinicializa el módulo 10DoF.

Reinicia el estado interno del módulo 10DoF.

Se proporciona como rutina de recuperación ante errores de comunicación. La función intenta dejar el bus y los sensores en un estado operativo sin exponer a las capas superiores los detalles eléctricos de recuperación.

Recupera el bus I2C y reinicializa el módulo 10DoF.

Se utiliza como mecanismo de recuperación cuando la secuencia de lectura queda en un estado no fiable y conviene volver a una condición conocida. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.30.3. Documentación de variables**8.30.3.1. AC1**

```
int16_t AC1 [static]
```

Coeficientes de calibración leídos de la EEPROM interna del BMP180.

8.30.3.2. AC2

```
int16_t AC2 [static]
```

8.30.3.3. AC3

```
int16_t AC3 [static]
```

8.30.3.4. AC4

```
uint16_t AC4 [static]
```

Coefficientes sin signo de calibración del BMP180.

8.30.3.5. AC5

```
uint16_t AC5 [static]
```

8.30.3.6. AC6

```
uint16_t AC6 [static]
```

8.30.3.7. B1

```
int16_t B1 [static]
```

8.30.3.8. B2

```
int16_t B2 [static]
```

8.30.3.9. gyro_x_offset

```
float gyro_x_offset = 0.0f [static]
```

Offset estático del eje X del giróscopo medido durante la calibración.

8.30.3.10. gyro_y_offset

```
float gyro_y_offset = 0.0f [static]
```

Offset estático del eje Y del giróscopo medido durante la calibración.

8.30.3.11. gyro_z_offset

```
float gyro_z_offset = 0.0f [static]
```

Offset estático del eje Z del giróscopo medido durante la calibración.

8.30.3.12. hi2c1

```
I2C_HandleTypeDef hi2c1 [extern]
```

Handle I2C generado por STM32CubeMX y compartido por los sensores 10DoF.

8.30.3.13. MB

```
int16_t MB [static]
```

8.30.3.14. MC

```
int16_t MC [static]
```

8.30.3.15. MD

```
int16_t MD [static]
```

8.31. Referencia del archivo Core/Src/stm32f4xx_hal_msp.c

This file provides code for the MSP Initialization and de-Initialization codes.

```
#include "main.h"
```

Gráfico de dependencias incluidas en stm32f4xx_hal_msp.c:

Funciones

- void [HAL_MspInit](#) (void)

8.31.1. Descripción detallada

This file provides code for the MSP Initialization and de-Initialization codes.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.31.2. Documentación de funciones

8.31.2.1. HAL_MspInit()

```
void HAL_MspInit (
    void )
```

Initializes the Global MSP.

8.32. Referencia del archivo Core/Src/stm32f4xx_hal_timebase_tim.c

HAL time base based on the hardware TIM.

```
#include "stm32f4xx_hal.h"
#include "stm32f4xx_hal_tim.h"
```

Gráfico de dependencias incluidas en stm32f4xx_hal_timebase_tim.c:

Funciones

- HAL_StatusTypeDef [HAL_InitTick](#) (uint32_t TickPriority)
This function configures the TIM2 as a time base source. The time source is configured to have 1ms time base with a dedicated Tick interrupt priority.
- void [HAL_SuspendTick](#) (void)
Suspend Tick increment.
- void [HAL_ResumeTick](#) (void)
Resume Tick increment.

Variables

- TIM_HandleTypeDef [htim2](#)

8.32.1. Descripción detallada

HAL time base based on the hardware TIM.

Atención

Copyright (c) 2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.32.2. Documentación de funciones

8.32.2.1. HAL_InitTick()

```
HAL_StatusTypeDef HAL_InitTick (
    uint32_t TickPriority)
```

This function configures the TIM2 as a time base source. The time source is configured to have 1ms time base with a dedicated Tick interrupt priority.

Nota

This function is called automatically at the beginning of program after reset by HAL_Init() or at any time when clock is configured, by HAL_RCC_ClockConfig().

Parámetros

<i>TickPriority</i>	Tick interrupt priority.
---------------------	--------------------------

Valores devueltos

<i>HAL</i>	status
------------	--------

8.32.2.2. HAL_ResumeTick()

```
void HAL_ResumeTick (
    void )
```

Resume Tick increment.

Nota

Enable the tick increment by Enabling TIM2 update interrupt.

Parámetros

<i>None</i>	
-------------	--

Valores devueltos

<i>None</i>	
-------------	--

8.32.2.3. HAL_SuspendTick()

```
void HAL_SuspendTick (
    void )
```

Suspend Tick increment.

Nota

Disable the tick increment by disabling TIM2 update interrupt.

Parámetros

None	
------	--

Valores devueltos

None	
------	--

8.32.3. Documentación de variables

8.32.3.1. htim2

```
TIM_HandleTypeDef htim2
```

8.33. Referencia del archivo Core/Src/syscalls.c

STM32CubeIDE Minimal System calls file.

```
#include <sys/stat.h>
#include <stdlib.h>
#include <errno.h>
#include <stdio.h>
#include <signal.h>
#include <time.h>
#include <sys/time.h>
#include <sys/times.h>
```

Gráfico de dependencias incluidas en syscalls.c:

Funciones

- `int __io_putchar (int ch) __attribute__((weak))`
- `int __io_getchar (void)`
- `void initialise_monitor_handles ()`
- `int _getpid (void)`
- `int _kill (int pid, int sig)`
- `void _exit (int status)`
- `__attribute__((weak))`
- `int _close (int file)`
- `int _fstat (int file, struct stat *st)`
- `int _isatty (int file)`
- `int _lseek (int file, int ptr, int dir)`
- `int _open (char *path, int flags,...)`
- `int _wait (int *status)`
- `int _unlink (char *name)`
- `int _times (struct tms *buf)`
- `int _stat (char *file, struct stat *st)`
- `int _link (char *old, char *new)`
- `int _fork (void)`
- `int _execve (char *name, char **argv, char **env)`

Variables

- `char ** environ = __env`

8.33.1. Descripción detallada

STM32CubeIDE Minimal System calls file.

Autor

Auto-generated by STM32CubeIDE

```
For more information about which c-functions
need which of these lowlevel functions
please consult the Newlib libc-manual
```

Atención

Copyright (c) 2020-2026 STMicroelectronics. All rights reserved.

This software is licensed under terms that can be found in the LICENSE file in the root directory of this software component. If no LICENSE file comes with this software, it is provided AS-IS.

8.33.2. Documentación de funciones

8.33.2.1. __attribute__((weak))

```
__attribute__((weak))
```

Gráfico de llamadas de esta función:

8.33.2.2. __io_getchar()

```
int __io_getchar (  
    void ) [extern]
```

Gráfico de llamadas a esta función:

8.33.2.3. __io_putchar()

```
int __io_putchar (  
    int ch) [extern]
```

8.33.2.4. _close()

```
int _close (  
    int file)
```

8.33.2.5. _execve()

```
int _execve (  
    char * name,  
    char ** argv,  
    char ** env)
```

8.33.2.6. _exit()

```
void _exit (  
    int status)
```

Gráfico de llamadas de esta función:

8.33.2.7. _fork()

```
int _fork (  
    void )
```

8.33.2.8. _fstat()

```
int _fstat (  
    int file,  
    struct stat * st)
```

8.33.2.9. _getpid()

```
int _getpid (  
    void )
```

8.33.2.10. _isatty()

```
int _isatty (
    int file)
```

8.33.2.11. _kill()

```
int _kill (
    int pid,
    int sig)
```

Gráfico de llamadas a esta función:

8.33.2.12. _link()

```
int _link (
    char * old,
    char * new)
```

8.33.2.13. _lseek()

```
int _lseek (
    int file,
    int ptr,
    int dir)
```

8.33.2.14. _open()

```
int _open (
    char * path,
    int flags,
    ...)
```

8.33.2.15. _stat()

```
int _stat (
    char * file,
    struct stat * st)
```

8.33.2.16. _times()

```
int _times (
    struct tms * buf)
```

8.33.2.17. `_unlink()`

```
int _unlink (  
    char * name)
```

8.33.2.18. `_wait()`

```
int _wait (  
    int * status)
```

8.33.2.19. `initialise_monitor_handles()`

```
void initialise_monitor_handles ()
```

8.33.3. Documentación de variables

8.33.3.1. `environ`

```
char** environ = __env
```

8.34. Referencia del archivo Core/Src/wifi.c

Driver WiFi/MQTT basado en comandos AT con RX DMA circular y TX DMA sincronizado.

```
#include "cmsis_os2.h"  
#include "wifi.h"  
#include <string.h>  
#include <stdio.h>  
#include <stdlib.h>
```

Gráfico de dependencias incluidas en wifi.c:

defines

- `#define RING_BUFFER_SIZE 2048`
Tamaño del búfer circular usado por la recepción DMA continua.

Funciones

- static uint32_t [get_dma_write_index](#) (void)
Obtiene la posición actual de escritura del DMA de recepción.
- static void [flush_ring_buffer](#) (void)
Descarta las respuestas acumuladas hasta el instante actual.
- static int [search_in_ring_buffer](#) (const char *target)
Busca una cadena objetivo en los datos recibidos desde la última lectura.
- void [HAL_UART_TxCpltCallback](#) (UART_HandleTypeDef *huart)
Callback HAL ejecutado al finalizar una transmisión UART por DMA.
- static wifi_status_t [wifi_send_cmd](#) (const char *cmd, const char *expected, uint32_t timeout)
Envía un comando AT y espera una respuesta esperada.
- wifi_status_t [WIFI_Init](#) (UART_HandleTypeDef *huart)
Inicializa el driver WiFi y prepara la recepción DMA circular.
- wifi_status_t [WIFI_Test](#) (void)
Comprueba que el módulo WiFi responde a comandos AT.
- wifi_status_t [WIFI_Reset](#) (void)
Reinicia el módulo WiFi mediante comando AT.
- wifi_status_t [WIFI_SetStationMode](#) (void)
Configura el módulo WiFi en modo estación.
- wifi_status_t [WIFI_ConnectAP](#) (const char *ssid, const char *pass)
Conecta el módulo a un punto de acceso WiFi.
- wifi_status_t [WIFI_DisconnectAP](#) (void)
Desconecta el módulo del punto de acceso actual.
- wifi_state_t [WIFI_GetState](#) (void)
Devuelve el estado lógico actual del módulo WiFi/MQTT.
- wifi_status_t [WIFI_MQTT_UserConfig](#) (const char *client_id)
Configura el identificador de cliente MQTT.
- wifi_status_t [WIFI_MQTT_Connect](#) (const char *broker, uint16_t port)
Abre la conexión MQTT contra el broker indicado.
- wifi_status_t [WIFI_MQTT_Publish](#) (const char *topic, const char *payload, uint8_t qos, uint8_t retain)
Publica un payload en un topic MQTT.
- wifi_status_t [WIFI_MQTT_Disconnect](#) (void)
Cierra la sesión MQTT actual.
- wifi_status_t [WIFI_Reconnect](#) (void)
Reconstruye la conexión WiFi y MQTT usando la configuración almacenada.

Variables

- static UART_HandleTypeDef * [wifi_uart](#)
UART asociada al módulo WiFi inicializado.
- static wifi_state_t [wifi_state](#) = [WIFI_DISCONNECTED](#)
Estado lógico actual de la conexión WiFi/MQTT.
- static uint8_t [ring_buffer](#) [[RING_BUFFER_SIZE](#)]
Búfer circular donde el DMA deposita respuestas y eventos del módulo WiFi.
- static uint32_t [read_index](#) = 0
Índice de lectura software dentro del búfer circular DMA.
- static osSemaphoreId_t [wifi_tx_sem](#) = NULL
Semáforo usado para despertar a la tarea cuando termina la transmisión DMA.
- static char [cached_ssid](#) [64] = {0}

SSID almacenado para permitir reconexión automática.

- static char `cached_pass` [64] = {0}

Contraseña almacenada para permitir reconexión automática.

- static char `cached_broker` [64] = {0}

Broker MQTT almacenado para reconstruir la sesión si se pierde.

- static uint16_t `cached_port` = 1883

Puerto MQTT almacenado para reconstruir la sesión si se pierde.

- static char `cached_client_id` [64] = {0}

Identificador MQTT almacenado para reconfigurar el cliente tras una reconexión.

8.34.1. Descripción detallada

Driver WiFi/MQTT basado en comandos AT con RX DMA circular y TX DMA sincronizado.

La arquitectura separa recepción y transmisión en dos canales de hardware: recepción DMA circular permanente para no perder respuestas asíncronas del módulo WiFi, y transmisión DMA sincronizada mediante semáforo para que las tareas FreeRTOS esperen sin ocupar CPU durante el envío físico por UART.

8.34.2. Documentación de «define»

8.34.2.1. RING_BUFFER_SIZE

```
#define RING_BUFFER_SIZE 2048
```

Tamaño del búfer circular usado por la recepción DMA continua.

8.34.3. Documentación de funciones

8.34.3.1. flush_ring_buffer()

```
void flush_ring_buffer (
    void ) [static]
```

Descarta las respuestas acumuladas hasta el instante actual.

Antes de enviar un comando nuevo se sincroniza el índice de lectura con el DMA para que la búsqueda posterior se asocie a la respuesta del comando actual y no a mensajes antiguos que hayan quedado en el búfer. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.2. get_dma_write_index()

```
uint32_t get_dma_write_index (
    void ) [static]
```

Obtiene la posición actual de escritura del DMA de recepción.

Devuelve

Índice donde el DMA escribirá el siguiente byte dentro del búfer circular.

La HAL expone el contador restante del DMA, por lo que se convierte a índice de escritura para comparar directamente con el índice software de lectura. Gráfico de llamadas a esta función:

8.34.3.3. HAL_UART_TxCpltCallback()

```
void HAL_UART_TxCpltCallback (
    UART_HandleTypeDef * huart)
```

Callback HAL ejecutado al finalizar una transmisión UART por DMA.

Parámetros

in	<i>huart</i>	UART que ha completado la transmisión.
----	--------------	--

La ISR solo libera el semáforo de transmisión para despertar a la tarea que envió el comando. Mantener el callback mínimo reduce latencia y evita procesar respuestas AT dentro de contexto de interrupción.

8.34.3.4. search_in_ring_buffer()

```
int search_in_ring_buffer (
    const char * target) [static]
```

Busca una cadena objetivo en los datos recibidos desde la última lectura.

Parámetros

in	<i>target</i>	Texto que debe aparecer en la respuesta del módulo WiFi.
----	---------------	--

Valores devueltos

1	Se encontró la cadena y se avanzó el índice de lectura.
0	La cadena no está disponible en los datos pendientes.

La búsqueda se realiza sobre una copia lineal para poder usar `strstr()` aunque el DMA esté trabajando sobre un búfer circular. Solo se consume hasta el final de la coincidencia para no descartar respuestas posteriores. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.5. WIFI_ConnectAP()

```
wifi_status_t WIFI_ConnectAP (
    const char * ssid,
    const char * pass)
```

Conecta el módulo a un punto de acceso WiFi.

Conecta el módulo WiFi a un punto de acceso.

Parámetros

in	<i>ssid</i>	Nombre de la red WiFi.
in	<i>pass</i>	Contraseña de la red WiFi.

Valores devueltos

<i>WIFI_OK</i>	Conexión establecida y dirección IP obtenida.
<i>WIFI_ERROR</i>	Error de conexión o transmisión.
<i>WIFI_TIMEOUT</i>	No se obtuvo confirmación dentro del tiempo límite.

Las credenciales se almacenan localmente para que [WIFI_Reconnect\(\)](#) pueda reconstruir la conexión sin depender de que la aplicación vuelva a pasarlas.

Conecta el módulo a un punto de acceso WiFi.

Parámetros

in	<i>ssid</i>	Nombre de la red WiFi.
in	<i>pass</i>	Contraseña de la red WiFi.

Devuelve

WIFI_OK si la asociación se completa correctamente.

WIFI_TIMEOUT si el punto de acceso no responde a tiempo.

WIFI_ERROR si las credenciales o la negociación fallan.

Precondición

ssid y *pass* deben apuntar a cadenas válidas terminadas en cero.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.6. WIFI_DisconnectAP()

```
wifi_status_t WIFI_DisconnectAP (  
    void )
```

Desconecta el módulo del punto de acceso actual.

Valores devueltos

<i>WIFI_OK</i>	Desconexión aceptada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Devuelve

WIFI_OK si la desconexión se completa correctamente.

WIFI_ERROR si el módulo no acepta la orden.

Gráfico de llamadas de esta función:

8.34.3.7. WIFI_GetState()

```
wifi_state_t WIFI_GetState (
    void )
```

Devuelve el estado lógico actual del módulo WiFi/MQTT.

Devuelve el estado lógico actual del driver WiFi.

Devuelve

Estado interno de conexión.

Devuelve el estado lógico actual del módulo WiFi/MQTT.

Devuelve

Estado de conexión almacenado por el módulo.

Nota

El estado permite a la aplicación evitar publicaciones cuando todavía no existe conectividad suficiente.

8.34.3.8. WIFI_Init()

```
wifi_status_t WIFI_Init (
    UART_HandleTypeDef * huart)
```

Inicializa el driver WiFi y prepara la recepción DMA circular.

Inicializa el driver WiFi y asocia la UART del módulo.

Parámetros

in	<i>huart</i>	UART conectada al módulo WiFi.
----	--------------	--------------------------------

Valores devueltos

<i>WIFI_OK</i>	El módulo respondió y quedó en modo estación.
<i>WIFI_ERROR</i>	No se pudo comunicar o configurar el módulo.

La recepción DMA se arranca antes de probar el módulo para capturar cualquier respuesta o evento que emita el ESP. Se desactivan interrupciones de mitad y fin de recepción porque el consumo se realiza por software consultando el índice del DMA.

Inicializa el driver WiFi y prepara la recepción DMA circular.

Parámetros

in	<i>huart</i>	Handler HAL de la UART conectada al módulo WiFi.
----	--------------	--

Devuelve

WIFI_OK si el módulo queda preparado para recibir comandos.

WIFI_ERROR si la UART o el módulo no pueden inicializarse.

Precondición

huart debe estar inicializado por la capa HAL.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.9. WIFI_MQTT_Connect()

```
wifi_status_t WIFI_MQTT_Connect (
    const char * broker,
    uint16_t port)
```

Abre la conexión MQTT contra el broker indicado.

Establece la conexión MQTT con el broker.

Parámetros

in	<i>broker</i>	Dirección IP o nombre del broker MQTT.
in	<i>port</i>	Puerto TCP del broker MQTT.

Valores devueltos

<i>WIFI_OK</i>	Conexión MQTT establecida.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

El broker y puerto se conservan para recuperar la sesión tras pérdidas de red sin obligar al resto de tareas a conocer la secuencia AT de reconexión.

Abre la conexión MQTT contra el broker indicado.

Parámetros

in	<i>broker</i>	Nombre DNS o dirección IP del broker MQTT.
in	<i>port</i>	Puerto TCP del broker MQTT.

Devuelve

WIFI_OK si la sesión MQTT queda establecida.

WIFI_TIMEOUT si el broker no responde.

WIFI_ERROR si falla la negociación MQTT.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.10. WIFI_MQTT_Disconnect()

```
wifi_status_t WIFI_MQTT_Disconnect (
    void )
```

Cierra la sesión MQTT actual.

Cierra la sesión MQTT activa.

Valores devueltos

<i>WIFI_OK</i>	Sesión MQTT liberada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Cierra la sesión MQTT actual.

Devuelve

WIFI_OK si la desconexión se completa correctamente.

WIFI_ERROR si el módulo no confirma el cierre de sesión.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.11. WIFI_MQTT_Publish()

```
wifi_status_t WIFI_MQTT_Publish (
    const char * topic,
    const char * payload,
    uint8_t qos,
    uint8_t retain)
```

Publica un payload en un topic MQTT.

Parámetros

in	<i>topic</i>	Topic MQTT de destino.
in	<i>payload</i>	Cadena que se publica como contenido del mensaje.
in	<i>qos</i>	Nivel QoS solicitado al módulo WiFi.
in	<i>retain</i>	Flag retain del mensaje MQTT.

Valores devueltos

<i>WIFI_OK</i>	Publicación aceptada por el módulo.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo confirmación dentro del tiempo límite.

Si la publicación falla, el estado se degrada a WIFI_CONNECTED porque la red puede seguir activa aunque la sesión MQTT haya quedado inválida. Esta decisión permite que la capa superior intente reconstruir solo MQTT antes de reiniciar toda la conexión WiFi.

Parámetros

in	<i>topic</i>	Topic MQTT de destino.
in	<i>payload</i>	Contenido que se publicará.
in	<i>qos</i>	Nivel QoS solicitado para la publicación.
in	<i>retain</i>	Flag retain enviado al broker.

Devuelve

WIFI_OK si el módulo acepta la publicación.

WIFI_NOT_CONNECTED si no existe sesión MQTT activa.

WIFI_ERROR si el comando de publicación falla.

La función permite a las tareas publicar telemetría sin conocer el comando AT concreto ni la sintaxis interna del módulo WiFi. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.12. WIFI_MQTT_UserConfig()

```
wifi_status_t WIFI_MQTT_UserConfig (
    const char * client_id)
```

Configura el identificador de cliente MQTT.

Parámetros

in	<i>client↔ _id</i>	Identificador usado por el cliente MQTT del módulo.
----	------------------------	---

Valores devueltos

<i>WIFI_OK</i>	Configuración aceptada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

El identificador se guarda para poder repetir la configuración después de una reconexión WiFi o reinicio del módulo.

Parámetros

in	<i>client↔ _id</i>	Identificador usado por el broker para reconocer el nodo.
----	------------------------	---

Devuelve

WIFI_OK si la configuración se acepta correctamente.

WIFI_ERROR si el módulo rechaza el identificador.

Precondición

`client_id` debe apuntar a una cadena válida terminada en cero.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.13. WIFI_Reconnect()

```
wifi_status_t WIFI_Reconnect (
    void )
```

Reconstruye la conexión WiFi y MQTT usando la configuración almacenada.

Ejecuta la secuencia de reconexión WiFi definida por el driver.

Valores devueltos

WIFI_OK	Reconexión completada hasta MQTT.
WIFI_ERROR	No se pudo restaurar WiFi o MQTT.

La rutina distingue entre pérdida total de WiFi y pérdida solo de MQTT para no reiniciar más capas de las necesarias. Esto reduce tiempo de recuperación y evita cortes innecesarios cuando el punto de acceso sigue disponible.

Reconstruye la conexión WiFi y MQTT usando la configuración almacenada.

Devuelve

WIFI_OK si se recupera la conectividad.

WIFI_ERROR si la reconexión falla.

Centralizar la reconexión evita que varias tareas intenten recuperar el módem con secuencias distintas. Gráfico de llamadas de esta función:

8.34.3.14. WIFI_Reset()

```
wifi_status_t WIFI_Reset (
    void )
```

Reinicia el módulo WiFi mediante comando AT.

Reinicia el módulo WiFi.

Valores devueltos

<i>WIFI_OK</i>	El módulo confirmó el reinicio con <i>ready</i> .
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Reinicia el módulo WiFi mediante comando AT.

Devuelve

WIFI_OK si el reinicio se acepta correctamente.

WIFI_TIMEOUT si el módulo no responde.

WIFI_ERROR si la secuencia de reinicio falla.

Se usa para devolver el módem a un estado conocido cuando la comunicación AT o la conexión de red dejan de ser fiables. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.15. wifi_send_cmd()

```
wifi_status_t wifi_send_cmd (
    const char * cmd,
    const char * expected,
    uint32_t timeout) [static]
```

Envía un comando AT y espera una respuesta esperada.

Parámetros

in	<i>cmd</i>	Comando AT completo, incluyendo terminación \r\n.
in	<i>expected</i>	Cadena que confirma la operación esperada.
in	<i>timeout</i>	Tiempo máximo de espera en ticks del kernel.

Valores devueltos

<i>WIFI_OK</i>	La respuesta esperada se recibió dentro del tiempo límite.
<i>WIFI_ERROR</i>	Falló la transmisión o el módulo devolvió <i>ERROR</i> .
<i>WIFI_TIMEOUT</i>	No llegó respuesta concluyente antes del timeout.

La función limpia errores UART y sincroniza el búfer RX antes de transmitir para asociar la respuesta al comando actual. El envío usa DMA para no bloquear la CPU, pero la tarea queda dormida hasta que el hardware confirma el fin de transmisión. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.16. WIFI_SetStationMode()

```
wifi_status_t WIFI_SetStationMode (  
    void )
```

Configura el módulo WiFi en modo estación.

Valores devueltos

<i>WIFI_OK</i>	Configuración aceptada.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Devuelve

WIFI_OK si el modo queda configurado correctamente.

WIFI_ERROR si el módulo rechaza la configuración.

El modo estación es necesario para que el dispositivo se conecte a la red local y pueda alcanzar el broker MQTT o servicios HTTP externos. Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.3.17. WIFI_Test()

```
wifi_status_t WIFI_Test (  
    void )
```

Comprueba que el módulo WiFi responde a comandos AT.

Comprueba la comunicación básica con el módulo WiFi.

Valores devueltos

<i>WIFI_OK</i>	El módulo respondió con OK.
<i>WIFI_ERROR</i>	Error devuelto por el módulo o fallo de transmisión.
<i>WIFI_TIMEOUT</i>	No hubo respuesta dentro del tiempo límite.

Comprueba que el módulo WiFi responde a comandos AT.

Devuelve

WIFI_OK si el módulo responde al comando de prueba.

WIFI_TIMEOUT si no hay respuesta.

WIFI_ERROR si la respuesta recibida no es válida.

Gráfico de llamadas de esta función: Gráfico de llamadas a esta función:

8.34.4. Documentación de variables

8.34.4.1. cached_broker

```
char cached_broker[64] = {0} [static]
```

Broker MQTT almacenado para reconstruir la sesión si se pierde.

8.34.4.2. cached_client_id

```
char cached_client_id[64] = {0} [static]
```

Identificador MQTT almacenado para reconfigurar el cliente tras una reconexión.

8.34.4.3. cached_pass

```
char cached_pass[64] = {0} [static]
```

Contraseña almacenada para permitir reconexión automática.

8.34.4.4. cached_port

```
uint16_t cached_port = 1883 [static]
```

Puerto MQTT almacenado para reconstruir la sesión si se pierde.

8.34.4.5. cached_ssid

```
char cached_ssid[64] = {0} [static]
```

SSID almacenado para permitir reconexión automática.

8.34.4.6. read_index

```
uint32_t read_index = 0 [static]
```

Índice de lectura software dentro del búfer circular DMA.

8.34.4.7. ring_buffer

```
uint8_t ring_buffer[RING_BUFFER_SIZE] [static]
```

Búfer circular donde el DMA deposita respuestas y eventos del módulo WiFi.

8.34.4.8. `wifi_state`

```
wifi_state_t wifi_state = WIFI_DISCONNECTED [static]
```

Estado lógico actual de la conexión WiFi/MQTT.

8.34.4.9. `wifi_tx_sem`

```
osSemaphoreId_t wifi_tx_sem = NULL [static]
```

Semáforo usado para despertar a la tarea cuando termina la transmisión DMA.

El semáforo evita esperas activas durante el envío de comandos AT y mantiene determinista la transición entre comando enviado y búsqueda de respuesta.

8.34.4.10. `wifi_uart`

```
UART_HandleTypeDef* wifi_uart [static]
```

UART asociada al módulo WiFi inicializado.

Índice alfabético

`__attribute__`
 syscalls.c, [144](#)
`__io_getchar`
 syscalls.c, [144](#)
`__io_putchar`
 syscalls.c, [145](#)
`_close`
 syscalls.c, [145](#)
`_execve`
 syscalls.c, [145](#)
`_exit`
 syscalls.c, [145](#)
`_fork`
 syscalls.c, [145](#)
`_fstat`
 syscalls.c, [145](#)
`_getpid`
 syscalls.c, [145](#)
`_isatty`
 syscalls.c, [145](#)
`_kill`
 syscalls.c, [146](#)
`_link`
 syscalls.c, [146](#)
`_lseek`
 syscalls.c, [146](#)
`_open`
 syscalls.c, [146](#)
`_stat`
 syscalls.c, [146](#)
`_times`
 syscalls.c, [146](#)
`_unlink`
 syscalls.c, [146](#)
`_wait`
 syscalls.c, [147](#)

AC1
 sensor10DoF.c, [138](#)
AC2
 sensor10DoF.c, [138](#)
AC3
 sensor10DoF.c, [139](#)
AC4
 sensor10DoF.c, [139](#)
AC5
 sensor10DoF.c, [139](#)
AC6
 sensor10DoF.c, [139](#)

ACC_ADDR
 Direcciones I2C, [11](#)
ACC_Init
 sensor10DoF.c, [133](#)
ACC_SENS_2G
 Constantes de conversión, [12](#)
accel
 ImuDataPacket_t, [23](#)
accel_x_ms2
 Sensor10DoF_t, [25](#)
accel_y_ms2
 Sensor10DoF_t, [25](#)
accel_z_ms2
 Sensor10DoF_t, [25](#)
altitude
 gps_data_t, [19](#)
 GpsDataPacket_t, [21](#)
altitude_m
 Sensor10DoF_t, [25](#)
assert_param
 stm32f4xx_hal_conf.h, [73](#)
Audio_RST_GPIO_Port
 main.h, [57](#)
Audio_RST_Pin
 main.h, [57](#)

B1
 sensor10DoF.c, [139](#)
B1_GPIO_Port
 main.h, [57](#)
B1_Pin
 main.h, [57](#)
B2
 sensor10DoF.c, [139](#)
BMP180_OSS
 Constantes de conversión, [12](#)
BMP180_ReadCalibration
 sensor10DoF.c, [133](#)
BMP180_ReadUP
 sensor10DoF.c, [133](#)
BMP180_ReadUT
 sensor10DoF.c, [134](#)
BMP180_SEA_HPA
 Constantes de conversión, [12](#)
BMP_ADDR
 Direcciones I2C, [11](#)
BOOT1_GPIO_Port
 main.h, [58](#)
BOOT1_Pin

main.h, [58](#)
 cached_broker
 wifi.c, [159](#)
 cached_client_id
 wifi.c, [159](#)
 cached_pass
 wifi.c, [159](#)
 cached_port
 wifi.c, [159](#)
 cached_ssid
 wifi.c, [159](#)
 checksum
 EnvDataPacket_t, [17](#)
 GpsDataPacket_t, [21](#)
 ImuDataPacket_t, [23](#)
 CLK_IN_GPIO_Port
 main.h, [58](#)
 CLK_IN_Pin
 main.h, [58](#)
 CMSIS_device_header
 FreeRTOSConfig.h, [38](#)
 configASSERT
 FreeRTOSConfig.h, [38](#)
 configCPU_CLOCK_HZ
 FreeRTOSConfig.h, [39](#)
 configENABLE_FPU
 FreeRTOSConfig.h, [39](#)
 configENABLE_MPU
 FreeRTOSConfig.h, [39](#)
 configKERNEL_INTERRUPT_PRIORITY
 FreeRTOSConfig.h, [39](#)
 configLIBRARY_LOWEST_INTERRUPT_PRIORITY
 FreeRTOSConfig.h, [39](#)
 configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY
 FreeRTOSConfig.h, [39](#)
 configMAX_CO_ROUTINE_PRIORITIES
 FreeRTOSConfig.h, [39](#)
 configMAX_PRIORITIES
 FreeRTOSConfig.h, [39](#)
 configMAX_SYSCALL_INTERRUPT_PRIORITY
 FreeRTOSConfig.h, [39](#)
 configMAX_TASK_NAME_LEN
 FreeRTOSConfig.h, [40](#)
 configMESSAGE_BUFFER_LENGTH_TYPE
 FreeRTOSConfig.h, [40](#)
 configMINIMAL_STACK_SIZE
 FreeRTOSConfig.h, [40](#)
 configPRIO_BITS
 FreeRTOSConfig.h, [40](#)
 configQUEUE_REGISTRY_SIZE
 FreeRTOSConfig.h, [40](#)
 configSUPPORT_DYNAMIC_ALLOCATION
 FreeRTOSConfig.h, [40](#)
 configSUPPORT_STATIC_ALLOCATION
 FreeRTOSConfig.h, [40](#)
 configTICK_RATE_HZ
 FreeRTOSConfig.h, [40](#)
 configTIMER_QUEUE_LENGTH
 FreeRTOSConfig.h, [40](#)
 configTIMER_TASK_PRIORITY
 FreeRTOSConfig.h, [40](#)
 configTIMER_TASK_STACK_DEPTH
 FreeRTOSConfig.h, [41](#)
 configTOTAL_HEAP_SIZE
 FreeRTOSConfig.h, [41](#)
 configUSE_16_BIT_TICKS
 FreeRTOSConfig.h, [41](#)
 configUSE_CO_ROUTINES
 FreeRTOSConfig.h, [41](#)
 configUSE_COUNTING_SEMAPHORES
 FreeRTOSConfig.h, [41](#)
 configUSE_IDLE_HOOK
 FreeRTOSConfig.h, [41](#)
 configUSE_MUTEXES
 FreeRTOSConfig.h, [41](#)
 configUSE_NEWLIB_REENTRANT
 FreeRTOSConfig.h, [41](#)
 configUSE_OS2_EVENTFLAGS_FROM_ISR
 FreeRTOSConfig.h, [41](#)
 configUSE_OS2_MUTEX
 FreeRTOSConfig.h, [41](#)
 configUSE_OS2_THREAD_ENUMERATE
 FreeRTOSConfig.h, [42](#)
 configUSE_OS2_THREAD_FLAGS
 FreeRTOSConfig.h, [42](#)
 configUSE_OS2_THREAD_SUSPEND_RESUME
 FreeRTOSConfig.h, [42](#)
 configUSE_OS2_TIMER
 FreeRTOSConfig.h, [42](#)
 configUSE_PORT_OPTIMISED_TASK_SELECTION
 FreeRTOSConfig.h, [42](#)
 configUSE_PREEMPTION
 FreeRTOSConfig.h, [42](#)
 configUSE_RECURSIVE_MUTEXES
 FreeRTOSConfig.h, [42](#)
 configUSE_TICK_HOOK
 FreeRTOSConfig.h, [42](#)
 configUSE_TIMERS
 FreeRTOSConfig.h, [42](#)
 configUSE_TRACE_FACILITY
 FreeRTOSConfig.h, [42](#)
 Constantes de conversión, [11](#)
 ACC_SENS_2G, [12](#)
 BMP180_OSS, [12](#)
 BMP180_SEA_HPA, [12](#)
 G_CONST, [12](#)
 GAUSS_TO_UT, [12](#)
 GYRO_SENS_250DPS, [12](#)
 MAG_XY_LSB_GAUSS, [13](#)
 MAG_Z_LSB_GAUSS, [13](#)
 Core/Inc/dma.h, [29](#), [30](#)
 Core/Inc/formatter_data.h, [30](#), [36](#)
 Core/Inc/FreeRTOSConfig.h, [37](#), [45](#)
 Core/Inc/gpio.h, [47](#), [48](#)
 Core/Inc/gps.h, [48](#), [53](#)
 Core/Inc/i2c.h, [54](#), [55](#)

- Core/Inc/main.h, [55](#), [64](#)
- Core/Inc/sensor10DoF.h, [65](#), [70](#)
- Core/Inc/stm32f4xx_hal_conf.h, [70](#), [84](#)
- Core/Inc/tim.h, [89](#), [91](#)
- Core/Inc/usart.h, [91](#), [93](#)
- Core/Inc/wifi.h, [93](#), [106](#)
- Core/Inc/wifi_types.h, [107](#), [108](#)
- Core/Src/formatter_data.c, [109](#)
- Core/Src/freertos.c, [115](#)
- Core/Src/gps.c, [124](#)
- Core/Src/sensor10DoF.c, [131](#)
- Core/Src/stm32f4xx_hal_msp.c, [140](#)
- Core/Src/stm32f4xx_hal_timebase_tim.c, [141](#)
- Core/Src/syscalls.c, [143](#)
- Core/Src/wifi.c, [147](#)
- CS_I2C_SPI_GPIO_Port
 - main.h, [58](#)
- CS_I2C_SPI_Pin
 - main.h, [58](#)
- DATA_CACHE_ENABLE
 - stm32f4xx_hal_conf.h, [73](#)
- Direcciones I2C, [10](#)
 - ACC_ADDR, [11](#)
 - BMP_ADDR, [11](#)
 - GYR_ADDR, [11](#)
 - MAG_ADDR, [11](#)
- dma.h
 - MX_DMA_Init, [29](#)
- DP83848_PHY_ADDRESS
 - stm32f4xx_hal_conf.h, [73](#)
- env_timestamp
 - EnvDataPacket_t, [17](#)
- env_updated
 - EnvDataPacket_t, [18](#)
- EnvData_to_json
 - formatter_data.c, [110](#)
 - formatter_data.h, [31](#)
- EnvDataPacket_Init
 - formatter_data.c, [110](#)
 - formatter_data.h, [32](#)
- EnvDataPacket_t, [17](#)
 - checksum, [17](#)
 - env_timestamp, [17](#)
 - env_updated, [18](#)
 - heading, [18](#)
 - pressure, [18](#)
 - temperature, [18](#)
- environ
 - syscalls.c, [147](#)
- envPacket
 - freertos.c, [120](#)
- Error_Handler
 - main.h, [64](#)
- Estado interno del driver GPS, [9](#)
 - gps_data, [9](#)
 - gps_rx_buffer, [9](#)
 - gps_uart, [10](#)
 - rx_head, [10](#)
 - rx_tail, [10](#)
 - sentence, [10](#)
 - sentence_index, [10](#)
 - uart_rx_byte, [10](#)
- ETH_RX_BUF_SIZE
 - stm32f4xx_hal_conf.h, [73](#)
- ETH_RXBUFNB
 - stm32f4xx_hal_conf.h, [73](#)
- ETH_TX_BUF_SIZE
 - stm32f4xx_hal_conf.h, [73](#)
- ETH_TXBUFNB
 - stm32f4xx_hal_conf.h, [74](#)
- EXTERNAL_CLOCK_VALUE
 - stm32f4xx_hal_conf.h, [74](#)
- fix
 - gps_data_t, [19](#)
- flush_ring_buffer
 - wifi.c, [149](#)
- formatter_data.c
 - EnvData_to_json, [110](#)
 - EnvDataPacket_Init, [110](#)
 - GpsData_to_json, [111](#)
 - GpsDataPacket_Init, [112](#)
 - ImuData_to_json, [113](#)
 - ImuDataPacket_Init, [114](#)
- formatter_data.h
 - EnvData_to_json, [31](#)
 - EnvDataPacket_Init, [32](#)
 - GpsData_to_json, [33](#)
 - GpsDataPacket_Init, [34](#)
 - ImuData_to_json, [34](#)
 - ImuDataPacket_Init, [35](#)
- freertos.c
 - envPacket, [120](#)
 - gps_st, [120](#)
 - gpsDataUpdated, [120](#)
 - gpsMutex, [120](#)
 - gpsPacket, [120](#)
 - gpsPacketToSend, [120](#)
 - gpsPacketToUpdate, [120](#)
 - gpsTask_attributes, [120](#)
 - gpsTaskHandle, [121](#)
 - huart2, [121](#)
 - huart3, [121](#)
 - imu_data, [121](#)
 - imuDataUpdated, [121](#)
 - imuMutex, [122](#)
 - imuPacket, [122](#)
 - imuPacketToSend, [122](#)
 - imuPacketToUpdate, [122](#)
 - imuTask_attributes, [122](#)
 - imuTaskHandle, [122](#)
 - MX_FREERTOS_Init, [118](#)
 - packetMutex, [123](#)
 - packetTask_attributes, [123](#)
 - packetTaskHandle, [123](#)
 - payload_envPacket, [123](#)

- payload_gpsPacket, 123
- payload_imuPacket, 123
- PAYLOAD_LENGTH, 117
- SERVER_HOST, 117
- StartGpsTask, 118
- StartImuTask, 118
- StartPacketTask, 119
- StartWifiTask, 119
- wifi_st, 124
- WIFI_TEST_PASS, 117
- WIFI_TEST_SSID, 118
- wifiTask_attributes, 124
- wifiTaskHandle, 124
- FreeRTOSConfig.h
 - CMSIS_device_header, 38
 - configASSERT, 38
 - configCPU_CLOCK_HZ, 39
 - configENABLE_FPU, 39
 - configENABLE_MPU, 39
 - configKERNEL_INTERRUPT_PRIORITY, 39
 - configLIBRARY_LOWEST_INTERRUPT_PRIORITY, 39
 - configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY, 39
 - configMAX_CO_ROUTINE_PRIORITIES, 39
 - configMAX_PRIORITIES, 39
 - configMAX_SYSCALL_INTERRUPT_PRIORITY, 39
 - configMAX_TASK_NAME_LEN, 40
 - configMESSAGE_BUFFER_LENGTH_TYPE, 40
 - configMINIMAL_STACK_SIZE, 40
 - configPRIO_BITS, 40
 - configQUEUE_REGISTRY_SIZE, 40
 - configSUPPORT_DYNAMIC_ALLOCATION, 40
 - configSUPPORT_STATIC_ALLOCATION, 40
 - configTICK_RATE_HZ, 40
 - configTIMER_QUEUE_LENGTH, 40
 - configTIMER_TASK_PRIORITY, 40
 - configTIMER_TASK_STACK_DEPTH, 41
 - configTOTAL_HEAP_SIZE, 41
 - configUSE_16_BIT_TICKS, 41
 - configUSE_CO_ROUTINES, 41
 - configUSE_COUNTING_SEMAPHORES, 41
 - configUSE_IDLE_HOOK, 41
 - configUSE_MUTEXES, 41
 - configUSE_NEWLIB_REENTRANT, 41
 - configUSE_OS2_EVENTFLAGS_FROM_ISR, 41
 - configUSE_OS2_MUTEX, 41
 - configUSE_OS2_THREAD_ENUMERATE, 42
 - configUSE_OS2_THREAD_FLAGS, 42
 - configUSE_OS2_THREAD_SUSPEND_RESUME, 42
 - configUSE_OS2_TIMER, 42
 - configUSE_PORT_OPTIMISED_TASK_SELECTION, 42
 - configUSE_PREEMPTION, 42
 - configUSE_RECURSIVE_MUTEXES, 42
 - configUSE_TICK_HOOK, 42
 - configUSE_TIMERS, 42
 - configUSE_TRACE_FACILITY, 42
 - INCLUDE_eTaskGetState, 43
 - INCLUDE_uxTaskGetStackHighWaterMark, 43
 - INCLUDE_uxTaskPriorityGet, 43
 - INCLUDE_vTaskCleanUpResources, 43
 - INCLUDE_vTaskDelay, 43
 - INCLUDE_vTaskDelayUntil, 43
 - INCLUDE_vTaskDelete, 43
 - INCLUDE_vTaskPrioritySet, 43
 - INCLUDE_vTaskSuspend, 43
 - INCLUDE_xQueueGetMutexHolder, 43
 - INCLUDE_xTaskGetCurrentTaskHandle, 44
 - INCLUDE_xTaskGetSchedulerState, 44
 - INCLUDE_xTimerPendFunctionCall, 44
 - USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION, 44
 - USE_FreeRTOS_HEAP_4, 44
 - vPortSVCHandler, 44
 - xPortPendSVHandler, 44
- G_CONST
 - Constantes de conversión, 12
- GAUSS_TO_UT
 - Constantes de conversión, 12
- get_dma_write_index
 - wifi.c, 149
- gpio.h
 - MX_GPIO_Init, 47
- gps.c
 - GPS_DataValid, 126
 - GPS_GetData, 126
 - GPS_Init, 127
 - gps_nmea_to_decimal, 128
 - gps_parse_gga, 128
 - gps_parse_rmc, 128
 - GPS_Process, 129
 - GPS_RestartReceive, 129
 - gps_rx_pop, 129
 - gps_rx_push, 130
 - GPS_UARTCallback, 130
- gps.h
 - GPS_DataValid, 50
 - GPS_ERROR, 50
 - GPS_FIX, 50
 - gps_fix_t, 50
 - GPS_GetData, 51
 - GPS_Init, 51
 - GPS_NO_FIX, 50
 - GPS_OK, 50
 - GPS_Process, 52
 - GPS_RestartReceive, 52
 - GPS_RX_BUFFER_SIZE, 49
 - GPS_SENTENCE_MAX_LENGTH, 49
 - gps_status_t, 50
 - GPS_TIMEOUT, 50
 - GPS_UARTCallback, 52
- gps_data
 - Estado interno del driver GPS, 9

- gps_data_t, 18
 - altitude, 19
 - fix, 19
 - latitude, 19
 - longitude, 19
 - satellites, 20
 - speed_kmh, 20
 - speed_knots, 20
 - valid, 20
- GPS_DataValid
 - gps.c, 126
 - gps.h, 50
- GPS_ERROR
 - gps.h, 50
- GPS_FIX
 - gps.h, 50
- gps_fix_t
 - gps.h, 50
- GPS_GetData
 - gps.c, 126
 - gps.h, 51
- GPS_Init
 - gps.c, 127
 - gps.h, 51
- gps_nmea_to_decimal
 - gps.c, 128
- GPS_NO_FIX
 - gps.h, 50
- GPS_OK
 - gps.h, 50
- gps_parse_gga
 - gps.c, 128
- gps_parse_rmc
 - gps.c, 128
- GPS_Process
 - gps.c, 129
 - gps.h, 52
- GPS_RestartReceive
 - gps.c, 129
 - gps.h, 52
- gps_rx_buffer
 - Estado interno del driver GPS, 9
- GPS_RX_BUFFER_SIZE
 - gps.h, 49
- gps_rx_pop
 - gps.c, 129
- gps_rx_push
 - gps.c, 130
- GPS_SENTENCE_MAX_LENGTH
 - gps.h, 49
- gps_st
 - freertos.c, 120
- gps_status_t
 - gps.h, 50
- GPS_TIMEOUT
 - gps.h, 50
- gps_timestamp
 - GpsDataPacket_t, 21
- gps_uart
 - Estado interno del driver GPS, 10
- GPS_UARTCallback
 - gps.c, 130
 - gps.h, 52
- gps_updated
 - GpsDataPacket_t, 21
- GpsData_to_json
 - formatter_data.c, 111
 - formatter_data.h, 33
- GpsDataPacket_Init
 - formatter_data.c, 112
 - formatter_data.h, 34
- GpsDataPacket_t, 20
 - altitude, 21
 - checksum, 21
 - gps_timestamp, 21
 - gps_updated, 21
 - latitude, 21
 - longitude, 22
- gpsDataUpdated
 - freertos.c, 120
- gpsMutex
 - freertos.c, 120
- gpsPacket
 - freertos.c, 120
- gpsPacketToSend
 - freertos.c, 120
- gpsPacketToUpdate
 - freertos.c, 120
- gpsTask_attributes
 - freertos.c, 120
- gpsTaskHandle
 - freertos.c, 121
- GYR_ADDR
 - Direcciones I2C, 11
- GYR_Init
 - sensor10DoF.c, 134
- gyro
 - ImuDataPacket_t, 23
- GYRO_SENS_250DPS
 - Constantes de conversión, 12
- gyro_x_dps
 - Sensor10DoF_t, 25
- gyro_x_offset
 - sensor10DoF.c, 139
- gyro_y_dps
 - Sensor10DoF_t, 25
- gyro_y_offset
 - sensor10DoF.c, 139
- gyro_z_dps
 - Sensor10DoF_t, 25
- gyro_z_offset
 - sensor10DoF.c, 139
- HAL_CORTEX_MODULE_ENABLED
 - stm32f4xx_hal_conf.h, 74
- HAL_DMA_MODULE_ENABLED
 - stm32f4xx_hal_conf.h, 74

HAL_EXTI_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [74](#)
 HAL_FLASH_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [74](#)
 HAL_GPIO_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [74](#)
 HAL_I2C_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [74](#)
 HAL_InitTick
 stm32f4xx_hal_timebase_tim.c, [142](#)
 HAL_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [74](#)
 HAL_Mspltinit
 stm32f4xx_hal_msp.c, [141](#)
 HAL_PWR_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [75](#)
 HAL_RCC_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [75](#)
 HAL_ResumeTick
 stm32f4xx_hal_timebase_tim.c, [142](#)
 HAL_SuspendTick
 stm32f4xx_hal_timebase_tim.c, [142](#)
 HAL_TIM_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [75](#)
 HAL_UART_MODULE_ENABLED
 stm32f4xx_hal_conf.h, [75](#)
 HAL_UART_TxCpltCallback
 wifi.c, [149](#)
 heading
 EnvDataPacket_t, [18](#)
 heading_deg
 Sensor10DoF_t, [25](#)
 hi2c1
 i2c.h, [55](#)
 sensor10DoF.c, [140](#)
 HSE_STARTUP_TIMEOUT
 stm32f4xx_hal_conf.h, [75](#)
 HSE_VALUE
 stm32f4xx_hal_conf.h, [75](#)
 HSI_VALUE
 stm32f4xx_hal_conf.h, [75](#)
 htim1
 tim.h, [90](#)
 htim2
 stm32f4xx_hal_timebase_tim.c, [143](#)
 htim3
 tim.h, [90](#)
 huart2
 freertos.c, [121](#)
 usart.h, [92](#)
 huart3
 freertos.c, [121](#)
 usart.h, [92](#)
 i2c.h
 hi2c1, [55](#)
 MX_I2C1_Init, [55](#)
 I2C1_SCL_IMU_GPIO_Port
 main.h, [58](#)
 I2C1_SCL_IMU_Pin
 main.h, [58](#)
 I2C1_SDA_IMU_GPIO_Port
 main.h, [58](#)
 I2C1_SDA_IMU_Pin
 main.h, [58](#)
 I2C_Read
 sensor10DoF.c, [134](#)
 I2C_Write
 sensor10DoF.c, [135](#)
 I2S3_MCK_GPIO_Port
 main.h, [59](#)
 I2S3_MCK_Pin
 main.h, [59](#)
 I2S3_SCK_GPIO_Port
 main.h, [59](#)
 I2S3_SCK_Pin
 main.h, [59](#)
 I2S3_SD_GPIO_Port
 main.h, [59](#)
 I2S3_SD_Pin
 main.h, [59](#)
 I2S3_WS_GPIO_Port
 main.h, [59](#)
 I2S3_WS_Pin
 main.h, [59](#)
 imu_data
 freertos.c, [121](#)
 imu_timestamp
 ImuDataPacket_t, [23](#)
 imu_updated
 ImuDataPacket_t, [23](#)
 ImuData_to_json
 formatter_data.c, [113](#)
 formatter_data.h, [34](#)
 ImuDataPacket_Init
 formatter_data.c, [114](#)
 formatter_data.h, [35](#)
 ImuDataPacket_t, [22](#)
 accel, [23](#)
 checksum, [23](#)
 gyro, [23](#)
 imu_timestamp, [23](#)
 imu_updated, [23](#)
 mag, [23](#)
 imuDataUpdated
 freertos.c, [121](#)
 imuMutex
 freertos.c, [122](#)
 imuPacket
 freertos.c, [122](#)
 imuPacketToSend
 freertos.c, [122](#)
 imuPacketToUpdate
 freertos.c, [122](#)
 imuTask_attributes
 freertos.c, [122](#)
 imuTaskHandle

- freertos.c, [122](#)
- INCLUDE_eTaskGetState
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_uxTaskGetStackHighWaterMark
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_uxTaskPriorityGet
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_vTaskCleanUpResources
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_vTaskDelay
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_vTaskDelayUntil
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_vTaskDelete
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_vTaskPrioritySet
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_vTaskSuspend
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_xQueueGetMutexHolder
 - FreeRTOSConfig.h, [43](#)
- INCLUDE_xTaskGetCurrentTaskHandle
 - FreeRTOSConfig.h, [44](#)
- INCLUDE_xTaskGetSchedulerState
 - FreeRTOSConfig.h, [44](#)
- INCLUDE_xTimerPendFunctionCall
 - FreeRTOSConfig.h, [44](#)
- initialise_monitor_handles
 - syscalls.c, [147](#)
- INSTRUCTION_CACHE_ENABLE
 - stm32f4xx_hal_conf.h, [75](#)
- latitude
 - gps_data_t, [19](#)
 - GpsDataPacket_t, [21](#)
- LD3_GPIO_Port
 - main.h, [59](#)
- LD3_Pin
 - main.h, [59](#)
- LD4_GPIO_Port
 - main.h, [60](#)
- LD4_Pin
 - main.h, [60](#)
- LD5_GPIO_Port
 - main.h, [60](#)
- LD5_Pin
 - main.h, [60](#)
- LD6_GPIO_Port
 - main.h, [60](#)
- LD6_Pin
 - main.h, [60](#)
- longitude
 - gps_data_t, [19](#)
 - GpsDataPacket_t, [22](#)
- LSE_STARTUP_TIMEOUT
 - stm32f4xx_hal_conf.h, [76](#)
- LSE_VALUE
 - stm32f4xx_hal_conf.h, [76](#)
- LSI_VALUE
 - stm32f4xx_hal_conf.h, [76](#)
- MAC_ADDR0
 - stm32f4xx_hal_conf.h, [76](#)
- MAC_ADDR1
 - stm32f4xx_hal_conf.h, [76](#)
- MAC_ADDR2
 - stm32f4xx_hal_conf.h, [76](#)
- MAC_ADDR3
 - stm32f4xx_hal_conf.h, [76](#)
- MAC_ADDR4
 - stm32f4xx_hal_conf.h, [76](#)
- MAC_ADDR5
 - stm32f4xx_hal_conf.h, [77](#)
- mag
 - ImuDataPacket_t, [23](#)
- MAG_ADDR
 - Direcciones I2C, [11](#)
- MAG_Init
 - sensor10DoF.c, [135](#)
- mag_x_uT
 - Sensor10DoF_t, [26](#)
- MAG_XY_LSB_GAUSS
 - Constantes de conversión, [13](#)
- mag_y_uT
 - Sensor10DoF_t, [26](#)
- MAG_Z_LSB_GAUSS
 - Constantes de conversión, [13](#)
- mag_z_uT
 - Sensor10DoF_t, [26](#)
- main.h
 - Audio_RST_GPIO_Port, [57](#)
 - Audio_RST_Pin, [57](#)
 - B1_GPIO_Port, [57](#)
 - B1_Pin, [57](#)
 - BOOT1_GPIO_Port, [58](#)
 - BOOT1_Pin, [58](#)
 - CLK_IN_GPIO_Port, [58](#)
 - CLK_IN_Pin, [58](#)
 - CS_I2C_SPI_GPIO_Port, [58](#)
 - CS_I2C_SPI_Pin, [58](#)
 - Error_Handler, [64](#)
 - I2C1_SCL_IMU_GPIO_Port, [58](#)
 - I2C1_SCL_IMU_Pin, [58](#)
 - I2C1_SDA_IMU_GPIO_Port, [58](#)
 - I2C1_SDA_IMU_Pin, [58](#)
 - I2S3_MCK_GPIO_Port, [59](#)
 - I2S3_MCK_Pin, [59](#)
 - I2S3_SCK_GPIO_Port, [59](#)
 - I2S3_SCK_Pin, [59](#)
 - I2S3_SD_GPIO_Port, [59](#)
 - I2S3_SD_Pin, [59](#)
 - I2S3_WS_GPIO_Port, [59](#)
 - I2S3_WS_Pin, [59](#)
 - LD3_GPIO_Port, [59](#)
 - LD3_Pin, [59](#)
 - LD4_GPIO_Port, [60](#)
 - LD4_Pin, [60](#)
 - LD5_GPIO_Port, [60](#)

- LD5_Pin, [60](#)
- LD6_GPIO_Port, [60](#)
- LD6_Pin, [60](#)
- MEMS_INT2_GPIO_Port, [60](#)
- MEMS_INT2_Pin, [60](#)
- OTG_FS_DM_GPIO_Port, [60](#)
- OTG_FS_DM_Pin, [60](#)
- OTG_FS_DP_GPIO_Port, [61](#)
- OTG_FS_DP_Pin, [61](#)
- OTG_FS_ID_GPIO_Port, [61](#)
- OTG_FS_ID_Pin, [61](#)
- OTG_FS_OverCurrent_GPIO_Port, [61](#)
- OTG_FS_OverCurrent_Pin, [61](#)
- OTG_FS_PowerSwitchOn_GPIO_Port, [61](#)
- OTG_FS_PowerSwitchOn_Pin, [61](#)
- PC14_OSC32_IN_GPIO_Port, [61](#)
- PC14_OSC32_IN_Pin, [61](#)
- PC15_OSC32_OUT_GPIO_Port, [62](#)
- PC15_OSC32_OUT_Pin, [62](#)
- PDM_OUT_GPIO_Port, [62](#)
- PDM_OUT_Pin, [62](#)
- PH0_OSC_IN_GPIO_Port, [62](#)
- PH0_OSC_IN_Pin, [62](#)
- PH1_OSC_OUT_GPIO_Port, [62](#)
- PH1_OSC_OUT_Pin, [62](#)
- SPI1_MISO_GPIO_Port, [62](#)
- SPI1_MISO_Pin, [62](#)
- SPI1_MOSI_GPIO_Port, [63](#)
- SPI1_MOSI_Pin, [63](#)
- SPI1_SCK_GPIO_Port, [63](#)
- SPI1_SCK_Pin, [63](#)
- SWCLK_GPIO_Port, [63](#)
- SWCLK_Pin, [63](#)
- SWDIO_GPIO_Port, [63](#)
- SWDIO_Pin, [63](#)
- SWO_GPIO_Port, [63](#)
- SWO_Pin, [63](#)
- VBUS_FS_GPIO_Port, [64](#)
- VBUS_FS_Pin, [64](#)
- MB
 - sensor10DoF.c, [140](#)
- MC
 - sensor10DoF.c, [140](#)
- MD
 - sensor10DoF.c, [140](#)
- MEMS_INT2_GPIO_Port
 - main.h, [60](#)
- MEMS_INT2_Pin
 - main.h, [60](#)
- MX_DMA_Init
 - dma.h, [29](#)
- MX_FREERTOS_Init
 - freertos.c, [118](#)
- MX_GPIO_Init
 - gpio.h, [47](#)
- MX_I2C1_Init
 - i2c.h, [55](#)
- MX_TIM1_Init
 - tim.h, [90](#)
- MX_TIM3_Init
 - tim.h, [90](#)
- MX_USART2_UART_Init
 - usart.h, [92](#)
- MX_USART3_UART_Init
 - usart.h, [92](#)
- OTG_FS_DM_GPIO_Port
 - main.h, [60](#)
- OTG_FS_DM_Pin
 - main.h, [60](#)
- OTG_FS_DP_GPIO_Port
 - main.h, [61](#)
- OTG_FS_DP_Pin
 - main.h, [61](#)
- OTG_FS_ID_GPIO_Port
 - main.h, [61](#)
- OTG_FS_ID_Pin
 - main.h, [61](#)
- OTG_FS_OverCurrent_GPIO_Port
 - main.h, [61](#)
- OTG_FS_OverCurrent_Pin
 - main.h, [61](#)
- OTG_FS_PowerSwitchOn_GPIO_Port
 - main.h, [61](#)
- OTG_FS_PowerSwitchOn_Pin
 - main.h, [61](#)
- packetMutex
 - freertos.c, [123](#)
- packetTask_attributes
 - freertos.c, [123](#)
- packetTaskHandle
 - freertos.c, [123](#)
- payload_envPacket
 - freertos.c, [123](#)
- payload_gpsPacket
 - freertos.c, [123](#)
- payload_imuPacket
 - freertos.c, [123](#)
- PAYLOAD_LENIGHT
 - freertos.c, [117](#)
- PC14_OSC32_IN_GPIO_Port
 - main.h, [61](#)
- PC14_OSC32_IN_Pin
 - main.h, [61](#)
- PC15_OSC32_OUT_GPIO_Port
 - main.h, [62](#)
- PC15_OSC32_OUT_Pin
 - main.h, [62](#)
- PDM_OUT_GPIO_Port
 - main.h, [62](#)
- PDM_OUT_Pin
 - main.h, [62](#)
- PH0_OSC_IN_GPIO_Port
 - main.h, [62](#)
- PH0_OSC_IN_Pin
 - main.h, [62](#)

- PH1_OSC_OUT_GPIO_Port
 - main.h, [62](#)
- PH1_OSC_OUT_Pin
 - main.h, [62](#)
- PHY_AUTONEGO_COMPLETE
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_AUTONEGOTIATION
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_BCR
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_BSR
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_CONFIG_DELAY
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_DUPLEX_STATUS
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_FULLDUPLEX_100M
 - stm32f4xx_hal_conf.h, [77](#)
- PHY_FULLDUPLEX_10M
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_HALFDUPLEX_100M
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_HALFDUPLEX_10M
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_ISOLATE
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_JABBER_DETECTION
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_LINKED_STATUS
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_LOOPBACK
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_POWERDOWN
 - stm32f4xx_hal_conf.h, [78](#)
- PHY_READ_TO
 - stm32f4xx_hal_conf.h, [79](#)
- PHY_RESET
 - stm32f4xx_hal_conf.h, [79](#)
- PHY_RESET_DELAY
 - stm32f4xx_hal_conf.h, [79](#)
- PHY_RESTART_AUTONEGOTIATION
 - stm32f4xx_hal_conf.h, [79](#)
- PHY_SPEED_STATUS
 - stm32f4xx_hal_conf.h, [79](#)
- PHY_SR
 - stm32f4xx_hal_conf.h, [79](#)
- PHY_WRITE_TO
 - stm32f4xx_hal_conf.h, [79](#)
- pitch_deg
 - Sensor10DoF_t, [26](#)
- PREFETCH_ENABLE
 - stm32f4xx_hal_conf.h, [79](#)
- pressure
 - EnvDataPacket_t, [18](#)
- pressure_atm
 - Sensor10DoF_t, [26](#)
- pressure_hpa
 - Sensor10DoF_t, [26](#)
- pressure_pa
 - Sensor10DoF_t, [26](#)
- read_index
 - wifi.c, [159](#)
- Referencia del directorio Core, [15](#)
- Referencia del directorio Core/Inc, [15](#)
- Referencia del directorio Core/Src, [16](#)
- ring_buffer
 - wifi.c, [159](#)
- RING_BUFFER_SIZE
 - wifi.c, [149](#)
- roll_deg
 - Sensor10DoF_t, [26](#)
- rx_head
 - Estado interno del driver GPS, [10](#)
- rx_tail
 - Estado interno del driver GPS, [10](#)
- satellites
 - gps_data_t, [20](#)
- search_in_ring_buffer
 - wifi.c, [150](#)
- sensor10DoF.c
 - AC1, [138](#)
 - AC2, [138](#)
 - AC3, [139](#)
 - AC4, [139](#)
 - AC5, [139](#)
 - AC6, [139](#)
 - ACC_Init, [133](#)
 - B1, [139](#)
 - B2, [139](#)
 - BMP180_ReadCalibration, [133](#)
 - BMP180_ReadUP, [133](#)
 - BMP180_ReadUT, [134](#)
 - GYR_Init, [134](#)
 - gyro_x_offset, [139](#)
 - gyro_y_offset, [139](#)
 - gyro_z_offset, [139](#)
 - hi2c1, [140](#)
 - I2C_Read, [134](#)
 - I2C_Write, [135](#)
 - MAG_Init, [135](#)
 - MB, [140](#)
 - MC, [140](#)
 - MD, [140](#)
 - Sensor10DoF_Calibrate, [136](#)
 - Sensor10DoF_CalibrateGyro_Internal, [136](#)
 - Sensor10DoF_I2CRecover, [136](#)
 - Sensor10DoF_Init, [136](#)
 - Sensor10DoF_Read, [137](#)
 - Sensor10DoF_Reset, [138](#)
- sensor10DoF.h
 - Sensor10DoF_Calibrate, [67](#)
 - Sensor10DoF_I2CRecover, [67](#)
 - Sensor10DoF_Init, [67](#)
 - Sensor10DoF_Read, [68](#)
 - Sensor10DoF_Reset, [69](#)

- SENSOR_BUSY, [67](#)
- SENSOR_ERROR, [67](#)
- SENSOR_OK, [67](#)
- SENSOR_TIMEOUT, [67](#)
- SensorStatus_t, [66](#)
- Sensor10DoF_Calibrate
 - sensor10DoF.c, [136](#)
 - sensor10DoF.h, [67](#)
- Sensor10DoF_CalibrateGyro_Internal
 - sensor10DoF.c, [136](#)
- Sensor10DoF_I2CRecover
 - sensor10DoF.c, [136](#)
 - sensor10DoF.h, [67](#)
- Sensor10DoF_Init
 - sensor10DoF.c, [136](#)
 - sensor10DoF.h, [67](#)
- Sensor10DoF_Read
 - sensor10DoF.c, [137](#)
 - sensor10DoF.h, [68](#)
- Sensor10DoF_Reset
 - sensor10DoF.c, [138](#)
 - sensor10DoF.h, [69](#)
- Sensor10DoF_t, [23](#)
 - accel_x_ms2, [25](#)
 - accel_y_ms2, [25](#)
 - accel_z_ms2, [25](#)
 - altitude_m, [25](#)
 - gyro_x_dps, [25](#)
 - gyro_y_dps, [25](#)
 - gyro_z_dps, [25](#)
 - heading_deg, [25](#)
 - mag_x_uT, [26](#)
 - mag_y_uT, [26](#)
 - mag_z_uT, [26](#)
 - pitch_deg, [26](#)
 - pressure_atm, [26](#)
 - pressure_hpa, [26](#)
 - pressure_pa, [26](#)
 - roll_deg, [26](#)
 - temperature_c, [27](#)
 - timestamp_ms, [27](#)
- SENSOR_BUSY
 - sensor10DoF.h, [67](#)
- SENSOR_ERROR
 - sensor10DoF.h, [67](#)
- SENSOR_OK
 - sensor10DoF.h, [67](#)
- SENSOR_TIMEOUT
 - sensor10DoF.h, [67](#)
- SensorStatus_t
 - sensor10DoF.h, [66](#)
- sentence
 - Estado interno del driver GPS, [10](#)
- sentence_index
 - Estado interno del driver GPS, [10](#)
- SERVER_HOST
 - freertos.c, [117](#)
- speed_kmh
 - gps_data_t, [20](#)
- speed_knots
 - gps_data_t, [20](#)
- SPI1_MISO_GPIO_Port
 - main.h, [62](#)
- SPI1_MISO_Pin
 - main.h, [62](#)
- SPI1_MOSI_GPIO_Port
 - main.h, [63](#)
- SPI1_MOSI_Pin
 - main.h, [63](#)
- SPI1_SCK_GPIO_Port
 - main.h, [63](#)
- SPI1_SCK_Pin
 - main.h, [63](#)
- StartGpsTask
 - freertos.c, [118](#)
- StartImuTask
 - freertos.c, [118](#)
- StartPacketTask
 - freertos.c, [119](#)
- StartWifiTask
 - freertos.c, [119](#)
- stm32f4xx_hal_conf.h
 - assert_param, [73](#)
 - DATA_CACHE_ENABLE, [73](#)
 - DP83848_PHY_ADDRESS, [73](#)
 - ETH_RX_BUF_SIZE, [73](#)
 - ETH_RXBUFNB, [73](#)
 - ETH_TX_BUF_SIZE, [73](#)
 - ETH_TXBUFNB, [74](#)
 - EXTERNAL_CLOCK_VALUE, [74](#)
 - HAL_CORTEX_MODULE_ENABLED, [74](#)
 - HAL_DMA_MODULE_ENABLED, [74](#)
 - HAL_EXTI_MODULE_ENABLED, [74](#)
 - HAL_FLASH_MODULE_ENABLED, [74](#)
 - HAL_GPIO_MODULE_ENABLED, [74](#)
 - HAL_I2C_MODULE_ENABLED, [74](#)
 - HAL_MODULE_ENABLED, [74](#)
 - HAL_PWR_MODULE_ENABLED, [75](#)
 - HAL_RCC_MODULE_ENABLED, [75](#)
 - HAL_TIM_MODULE_ENABLED, [75](#)
 - HAL_UART_MODULE_ENABLED, [75](#)
 - HSE_STARTUP_TIMEOUT, [75](#)
 - HSE_VALUE, [75](#)
 - HSI_VALUE, [75](#)
 - INSTRUCTION_CACHE_ENABLE, [75](#)
 - LSE_STARTUP_TIMEOUT, [76](#)
 - LSE_VALUE, [76](#)
 - LSI_VALUE, [76](#)
 - MAC_ADDR0, [76](#)
 - MAC_ADDR1, [76](#)
 - MAC_ADDR2, [76](#)
 - MAC_ADDR3, [76](#)
 - MAC_ADDR4, [76](#)
 - MAC_ADDR5, [77](#)
 - PHY_AUTONEGO_COMPLETE, [77](#)
 - PHY_AUTONEGOTIATION, [77](#)

- PHY_BCR, [77](#)
- PHY_BSR, [77](#)
- PHY_CONFIG_DELAY, [77](#)
- PHY_DUPLEX_STATUS, [77](#)
- PHY_FULLDUPLEX_100M, [77](#)
- PHY_FULLDUPLEX_10M, [78](#)
- PHY_HALFDUPLEX_100M, [78](#)
- PHY_HALFDUPLEX_10M, [78](#)
- PHY_ISOLATE, [78](#)
- PHY_JABBER_DETECTION, [78](#)
- PHY_LINKED_STATUS, [78](#)
- PHY_LOOPBACK, [78](#)
- PHY_POWERDOWN, [78](#)
- PHY_READ_TO, [79](#)
- PHY_RESET, [79](#)
- PHY_RESET_DELAY, [79](#)
- PHY_RESTART_AUTONEGOTIATION, [79](#)
- PHY_SPEED_STATUS, [79](#)
- PHY_SR, [79](#)
- PHY_WRITE_TO, [79](#)
- PREFETCH_ENABLE, [79](#)
- TICK_INT_PRIORITY, [80](#)
- USE_HAL_ADC_REGISTER_CALLBACKS, [80](#)
- USE_HAL_CAN_REGISTER_CALLBACKS, [80](#)
- USE_HAL_CEC_REGISTER_CALLBACKS, [80](#)
- USE_HAL_CRYPT_REGISTER_CALLBACKS, [80](#)
- USE_HAL_DAC_REGISTER_CALLBACKS, [80](#)
- USE_HAL_DCMI_REGISTER_CALLBACKS, [80](#)
- USE_HAL_DFSDM_REGISTER_CALLBACKS, [80](#)
- USE_HAL_DMA2D_REGISTER_CALLBACKS, [80](#)
- USE_HAL_DSI_REGISTER_CALLBACKS, [80](#)
- USE_HAL_ETH_REGISTER_CALLBACKS, [81](#)
- USE_HAL_FMPI2C_REGISTER_CALLBACKS, [81](#)
- USE_HAL_FMPSMBUS_REGISTER_CALLBACKS, [81](#)
- USE_HAL_HASH_REGISTER_CALLBACKS, [81](#)
- USE_HAL_HCD_REGISTER_CALLBACKS, [81](#)
- USE_HAL_I2C_REGISTER_CALLBACKS, [81](#)
- USE_HAL_I2S_REGISTER_CALLBACKS, [81](#)
- USE_HAL_IRDA_REGISTER_CALLBACKS, [81](#)
- USE_HAL_LPTIM_REGISTER_CALLBACKS, [81](#)
- USE_HAL_LTDC_REGISTER_CALLBACKS, [81](#)
- USE_HAL_MMC_REGISTER_CALLBACKS, [82](#)
- USE_HAL_NAND_REGISTER_CALLBACKS, [82](#)
- USE_HAL_NOR_REGISTER_CALLBACKS, [82](#)
- USE_HAL_PCCARD_REGISTER_CALLBACKS, [82](#)
- USE_HAL_PCD_REGISTER_CALLBACKS, [82](#)
- USE_HAL_QSPI_REGISTER_CALLBACKS, [82](#)
- USE_HAL_RNG_REGISTER_CALLBACKS, [82](#)
- USE_HAL_RTC_REGISTER_CALLBACKS, [82](#)
- USE_HAL_SAI_REGISTER_CALLBACKS, [82](#)
- USE_HAL_SD_REGISTER_CALLBACKS, [82](#)
- USE_HAL_SDRAM_REGISTER_CALLBACKS, [83](#)
- USE_HAL_SMARTCARD_REGISTER_CALLBACKS, [83](#)
- USE_HAL_SMBUS_REGISTER_CALLBACKS, [83](#)
- USE_HAL_SPDIFRX_REGISTER_CALLBACKS, [83](#)
- USE_HAL_SPI_REGISTER_CALLBACKS, [83](#)
- USE_HAL_SRAM_REGISTER_CALLBACKS, [83](#)
- USE_HAL_TIM_REGISTER_CALLBACKS, [83](#)
- USE_HAL_UART_REGISTER_CALLBACKS, [83](#)
- USE_HAL_USART_REGISTER_CALLBACKS, [83](#)
- USE_HAL_WWDG_REGISTER_CALLBACKS, [83](#)
- USE_RTOS, [84](#)
- USE_SPI_CRC, [84](#)
- VDD_VALUE, [84](#)
- stm32f4xx_hal_msp.c
 - HAL_Mspltinit, [141](#)
- stm32f4xx_hal_timebase_tim.c
 - HAL_InitTick, [142](#)
 - HAL_ResumeTick, [142](#)
 - HAL_SuspendTick, [142](#)
- htim2, [143](#)
- SWCLK_GPIO_Port
 - main.h, [63](#)
- SWCLK_Pin
 - main.h, [63](#)
- SWDIO_GPIO_Port
 - main.h, [63](#)
- SWDIO_Pin
 - main.h, [63](#)
- SWO_GPIO_Port
 - main.h, [63](#)
- SWO_Pin
 - main.h, [63](#)
- syscalls.c
 - __attribute__, [144](#)
 - __io_getchar, [144](#)
 - __io_putchar, [145](#)
 - _close, [145](#)
 - _execve, [145](#)
 - _exit, [145](#)
 - _fork, [145](#)
 - _fstat, [145](#)
 - _getpid, [145](#)
 - _isatty, [145](#)
 - _kill, [146](#)
 - _link, [146](#)
 - _lseek, [146](#)
 - _open, [146](#)
 - _stat, [146](#)
 - _times, [146](#)
 - _unlink, [146](#)
 - _wait, [147](#)
 - environ, [147](#)
 - initialise_monitor_handles, [147](#)
- temperature
 - EnvDataPacket_t, [18](#)
- temperature_c
 - Sensor10DoF_t, [27](#)
- TICK_INT_PRIORITY
 - stm32f4xx_hal_conf.h, [80](#)
- tim.h

- htim1, [90](#)
- htim3, [90](#)
- MX_TIM1_Init, [90](#)
- MX_TIM3_Init, [90](#)
- timestamp_ms
 - Sensor10DoF_t, [27](#)
- uart_rx_byte
 - Estado interno del driver GPS, [10](#)
- usart.h
 - huart2, [92](#)
 - huart3, [92](#)
 - MX_USART2_UART_Init, [92](#)
 - MX_USART3_UART_Init, [92](#)
- USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION
 - FreeRTOSConfig.h, [44](#)
- USE_FreeRTOS_HEAP_4
 - FreeRTOSConfig.h, [44](#)
- USE_HAL_ADC_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_CAN_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_CEC_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_Cryp_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_DAC_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_DCMI_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_DFSDM_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_DMA2D_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_DSI_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [80](#)
- USE_HAL_ETH_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_FMPI2C_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_FMPSPMBUS_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_HASH_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_HCD_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_I2C_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_I2S_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_IRDA_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_LPTIM_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_LTDC_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [81](#)
- USE_HAL_MMC_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_NAND_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_NOR_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_PCCARD_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_PCD_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_QSPI_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_RNG_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_RTC_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_SAI_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_SD_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [82](#)
- USE_HAL_SDRAM_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_SMARTCARD_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_SMBUS_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_SPDIFRX_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_SPI_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_SRAM_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_TIM_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_UART_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_USART_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_HAL_WWDG_REGISTER_CALLBACKS
 - stm32f4xx_hal_conf.h, [83](#)
- USE_RTOS
 - stm32f4xx_hal_conf.h, [84](#)
- USE_SPI_CRC
 - stm32f4xx_hal_conf.h, [84](#)
- valid
 - gps_data_t, [20](#)
- VBUS_FS_GPIO_Port
 - main.h, [64](#)
- VBUS_FS_Pin
 - main.h, [64](#)
- VDD_VALUE
 - stm32f4xx_hal_conf.h, [84](#)
- vPortSVCHandler
 - FreeRTOSConfig.h, [44](#)
- wifi.c
 - cached_broker, [159](#)
 - cached_client_id, [159](#)
 - cached_pass, [159](#)
 - cached_port, [159](#)
 - cached_ssid, [159](#)

- flush_ring_buffer, 149
- get_dma_write_index, 149
- HAL_UART_TxCpltCallback, 149
- read_index, 159
- ring_buffer, 159
- RING_BUFFER_SIZE, 149
- search_in_ring_buffer, 150
- WIFI_ConnectAP, 150
- WIFI_DisconnectAP, 151
- WIFI_GetState, 151
- WIFI_Init, 152
- WIFI_MQTT_Connect, 153
- WIFI_MQTT_Disconnect, 153
- WIFI_MQTT_Publish, 154
- WIFI_MQTT_UserConfig, 155
- WIFI_Reconnect, 156
- WIFI_Reset, 156
- wifi_send_cmd, 157
- WIFI_SetStationMode, 157
- wifi_state, 159
- WIFI_Test, 158
- wifi_tx_sem, 160
- wifi_uart, 160
- wifi.h
 - WIFI_ConnectAP, 95
 - WIFI_DisconnectAP, 95
 - WIFI_GetIP, 96
 - WIFI_GetRSSI, 96
 - WIFI_GetState, 97
 - WIFI_HTTP_PostJSON, 97
 - WIFI_Init, 98
 - WIFI_MQTT_Connect, 99
 - WIFI_MQTT_Disconnect, 100
 - WIFI_MQTT_Publish, 100
 - WIFI_MQTT_Subscribe, 101
 - WIFI_MQTT_UserConfig, 101
 - WIFI_Process, 102
 - WIFI_Reconnect, 102
 - WIFI_Reset, 103
 - WIFI_SendJSON, 104
 - WIFI_SendRaw, 104
 - WIFI_SetStationMode, 104
 - WIFI_TCPClose, 105
 - WIFI_TCPConnect, 105
 - WIFI_Test, 105
- WIFI_BUSY
 - wifi_types.h, 108
- WIFI_ConnectAP
 - wifi.c, 150
 - wifi.h, 95
- WIFI_CONNECTED
 - wifi_types.h, 108
- WIFI_DisconnectAP
 - wifi.c, 151
 - wifi.h, 95
- WIFI_DISCONNECTED
 - wifi_types.h, 108
- WIFI_ERROR
 - wifi_types.h, 108
- WIFI_GetIP
 - wifi.h, 96
- WIFI_GetRSSI
 - wifi.h, 96
- WIFI_GetState
 - wifi.c, 151
 - wifi.h, 97
- WIFI_GOT_IP
 - wifi_types.h, 108
- WIFI_HTTP_PostJSON
 - wifi.h, 97
- WIFI_Init
 - wifi.c, 152
 - wifi.h, 98
- WIFI_MQTT_Connect
 - wifi.c, 153
 - wifi.h, 99
- WIFI_MQTT_CONNECTED
 - wifi_types.h, 108
- WIFI_MQTT_Disconnect
 - wifi.c, 153
 - wifi.h, 100
- WIFI_MQTT_Publish
 - wifi.c, 154
 - wifi.h, 100
- WIFI_MQTT_Subscribe
 - wifi.h, 101
- WIFI_MQTT_UserConfig
 - wifi.c, 155
 - wifi.h, 101
- WIFI_NOT_CONNECTED
 - wifi_types.h, 108
- WIFI_OK
 - wifi_types.h, 108
- WIFI_Process
 - wifi.h, 102
- WIFI_Reconnect
 - wifi.c, 156
 - wifi.h, 102
- WIFI_Reset
 - wifi.c, 156
 - wifi.h, 103
- wifi_send_cmd
 - wifi.c, 157
- WIFI_SendJSON
 - wifi.h, 104
- WIFI_SendRaw
 - wifi.h, 104
- WIFI_SetStationMode
 - wifi.c, 157
 - wifi.h, 104
- wifi_st
 - freertos.c, 124
- wifi_state
 - wifi.c, 159
- wifi_state_t
 - wifi_types.h, 108

- wifi_status_t
 - wifi_types.h, [108](#)
- WIFI_TCPClose
 - wifi.h, [105](#)
- WIFI_TCPConnect
 - wifi.h, [105](#)
- WIFI_Test
 - wifi.c, [158](#)
 - wifi.h, [105](#)
- WIFI_TEST_PASS
 - freertos.c, [117](#)
- WIFI_TEST_SSID
 - freertos.c, [118](#)
- WIFI_TIMEOUT
 - wifi_types.h, [108](#)
- wifi_tx_sem
 - wifi.c, [160](#)
- wifi_types.h
 - WIFI_BUSY, [108](#)
 - WIFI_CONNECTED, [108](#)
 - WIFI_DISCONNECTED, [108](#)
 - WIFI_ERROR, [108](#)
 - WIFI_GOT_IP, [108](#)
 - WIFI_MQTT_CONNECTED, [108](#)
 - WIFI_NOT_CONNECTED, [108](#)
 - WIFI_OK, [108](#)
 - wifi_state_t, [108](#)
 - wifi_status_t, [108](#)
 - WIFI_TIMEOUT, [108](#)
- wifi_uart
 - wifi.c, [160](#)
- wifiTask_attributes
 - freertos.c, [124](#)
- wifiTaskHandle
 - freertos.c, [124](#)
- xPortPendSVHandler
 - FreeRTOSConfig.h, [44](#)